

Research Notes for “The Work a Verifier Needs”

Samuel Mausberg

1 October 2026

Contents

1	About these notes	2
2	Status of the questions and proposals	2
3	Time, recurrent state and the serving stack	6
3.1	Useful progress per unit time	6
3.2	Ranked costs	7
3.3	Ceilings under a stated execution model	7
3.4	Speculation and the recurrent state	8
3.5	How much recurrent state an exact computation needs	10
3.6	Leverage by axis	12
3.7	Interactions and the two stacks	17
3.8	A sensitivity workload for output length	17
4	The serving protocol	17
5	Drafting and repair hypotheses	22
5.1	The public block drafter	22
5.2	What a cycle costs	23
5.3	Acceptance by block position	23
5.4	Output equality and limits	24
5.5	Training a selector for rate (P6, hypothesis)	24
5.6	Exact repair of long windows (P2, P3)	25
5.7	Reusing a cached window after a rejection (P9)	29
5.8	Three external drafting proposals (P-A, P-B, P-C)	30
5.9	Verifying the recurrent state (P7b, P10, P11)	31
5.10	Selector and depth analysis not tested here	32
6	The stock engine under speculation	33
6.1	The divergence record in detail	33
6.2	Recurrent state under speculation	36
7	Further analysis	38
7.1	The directional margin	38
7.2	Beyond the head: a certified decoder tail (P1, rejected)	38
7.3	Compiling the head through the last FFN (P12, negative at its screen)	39
7.4	Sampling before the scores are known	39
7.5	An approximate mode, and why it cannot be certified harmless	40
7.6	A demand frontier over a fixed protocol	40
7.7	A cheap decision is not a cheap verification	41
7.8	A summary that supports both selection and mass	41
7.9	From skipped work to time	41
8	The exact rational reference	42

9 Evidence register	42
9.1 Evidence register	42
10 Glossary	49

1 About these notes

These notes accompany the paper “The Work a Verifier Needs” (“the paper” below). They hold the investigations made during the work that the paper’s claims do not rely on: the programme’s other questions and proposals with their tests and verdicts, the rest of the serving stack, drafting and repair, the stock engine’s recurrent state under speculation, and further analysis of sampled decisions and transport. Most of the text was moved unchanged from the paper’s appendices when the paper was edited for length; only cross-references changed, and a reference into the paper names it, as in “Appendix D.4 of the paper”.

Moved text keeps the paper’s earlier vocabulary, which differs from the paper’s present terms as Table 1 shows; Section 10 defines the other terms these notes use.

Table 1. The older words that moved text keeps, and what the paper now calls them.

In these notes	In the paper
Self-evidence	The int8 (or other low-precision) copy of the head evaluated at the target’s own head input
Envelope β_i	The error bound β_i on that copy’s score of vocabulary entry i
Row (of the head, of W)	Vocabulary entry: a token and its weight row w_i
Row (of a batch), verify row, draft row	Position: one head input and the decision made from it
Row-wise or column fallback	Per-position fallback
The programme	The research behind the paper, organized as questions H1–H7 and proposals P1–P13 and P-A to P-C (Table 2)
H1–H7, P1–P13, P-A to P-C	Labels of those questions and proposals; Table 2 gives each one, its test and its status
Kill test	A test fixed before a proposal is built, designed to reject it cheaply
First matrix, pinned matrix	The cap-16 and cap-8 runs of Appendix D.4 of the paper
Second panel	The 80 prompts of the drafter’s equality and acceptance runs (Section 5.3)

Evidence is marked as in the paper. A committed result cites its entry in the register of these notes (Section 9), printed as [N- En], and a result that was not measured is said to be so where it would appear. The paper numbers its own entries without the prefix, En , so an ID never means two things. The two registers are numbered separately: an entry that both documents cite lists the same files in both but usually has a different number, so N- En and the paper’s En for the same files differ; the paper’s numbers are in its register (Appendix F.1 of the paper). Oracle bounds, derived estimates and single runs are labelled as such where they are reported, and a refuted proposal stays with the mechanism that refuted it.

The notes build from the repository’s `paper/` directory after the paper, with `latexmk -pdf notes/research_notes.tex`.

Attribution and disclosure These notes share the paper’s authorship and its disclosure (Appendix F.4 of the paper), which reads:

Samuel Mausberg is the author of this paper, directed the work and is responsible for its claims. AI tools (Claude Opus 5.5 and GPT-6 Astra Pro) were used in developing the paper and the code. No AI system is an author. The confidential assignment that motivated the work is not reproduced or quoted, and no model weights are redistributed.

2 Status of the questions and proposals

Table 2 lists each question of the programme (H1–H7) and each later proposal that has a result or a decision (P1–P13, and the external drafting proposals P-A to P-C; P7a and P7b are the two parts of P7), with the test that decides or could damage it and its status. P13 had no committed pre-registered kill test, and its row says so. A failed hypothesis stays in the table with its mechanism.

Table 2. Questions and proposals, the test that decides or could damage each, and the status.

Question or claim	Deciding or damaging test	Status
H1: the head is a large share of decode and MTP-speculation time	Nsight attribution at several batch sizes (Appendix E.2 of the paper)	Measured: the head chain is 10.3–6.1% of a plain step ($B = 1$ –128), below its 15.1% share of the step’s weight bytes, and 17.2–12.4% of an MTP cycle ($B = 1$ –32), against about 38% of the cycle’s bytes at $B = 1$ [N-E32,N-E34]
H2: transport gives useful certificates on real hidden states, tighter than self-evidence and the static screen	Drift against δ^* ; certification on aligned pairs (Section 5.3 of the paper)	Refuted on DFlash-4B and MTP-4B for every bound family and tiling tested [N-E13,N-E14,N-E16,N-E15]
H3: a low-precision head with a rigorous envelope certifies the stock decision while reading fewer weight bytes	Candidate sets on replayed rows, then a GPU kernel against the stock head chain (Section 3 of the paper)	Envelope and contract proved; candidate sets and fallback shares measured on replayed rows [N-E1,N-E2], including MTP-4B verify and draft rows [N-E3]; for the head alone with the per-position fallback under the conservative model, the expected time from measured times and fallback rates is 0.67–0.70 of the stock head chain’s up to $M = 16$ positions, 0.81 at 32 and 0.93 at 64, and the head is slower from 128 (1.31) [N-E9,N-E10]; served, 1.2–4.6% more throughput at $c = 1$, and 1.0–3.3% less at every DFlash point above $c = 1$ where the head is active (certified batches of 28–64 rows) and for MTP at 64 (64-row draft batches); plain decoding at $c = 32$ and 64 and MTP at $c = 8$ –32 run batches of these sizes without a detectable loss [N-E73]
Certified tokens equal stock tokens	Same-shape token equality; measured cuBLAS error against the modelled G_i	No disagreement on 6,005 replayed rows [N-E2]; through the kernel, certified rows plus the fallback equal the reference on all 60,000 replayed rows [N-E8]; the stock FP32-output kernel’s largest observed error is 295 and 58 times inside the two models, and the BF16-output kernel’s is not observable [N-E6]; in SGLang, 0 of 196,734 certified rows differed from the stock head at the same batch shape, and outputs one request at a time were identical [N-E12]
Fewer bytes read means less time	Head-path time against the stock chain at each served shape	Only at small batches: the W8A16 pass alone takes 0.57–0.78 of the stock chain’s time at $M = 1$ to 64 and 1.17–1.46 at 128 and 256 [N-E11]; with its checks and expected fallbacks (best mode, conservative model) the head is faster up to $M = 64$ [N-E10]
Per-row sparsity reduces weight traffic	Batch unions of candidates; measured traffic	Candidate unions on capture batches measured [N-E1]; the implemented pass reads the whole int8 head per call, and a kernel that reads only the union was not built
Certificates save target work	Count rows, re-scoring and completion, not only decision cost	A cheap decision is not a cheap verification on constructed heads [N-E19]
H4: the certificate improves the served frontier without changing outputs	Before-and-after sweeps with output-equality checks (Appendix E.5 of the paper)	Supported under the declared rule (one primary per family, Holm-adjusted): at $c = 1$ plain decoding +2.5%, MTP +4.6% and DFlash with 16-token blocks +1.2%, and DFlash with 8-token blocks –1.2% at $c = 4$; the served envelope rises only at $c = 1$ and falls at $c = 2, 4, 8$ and 128. Token-identical to stock at $c = 1$ and in check mode (0 of 592,433 certified rows differ), and for MTP under identical batch evolution; the verify defers to the stock head for batches that request log-probabilities, penalties or a grammar [N-E73]

Question or claim	Deciding or damaging test	Status
H5: SGLang’s hybrid state stays correct under speculative verification	Targeted tests at every rejection position, aborts and prefix reuse (Section 6.2)	Truncation and stop tokens at every index of an MTP verify cycle leave the output token-identical to the untruncated prefix (397 and 400 cases; log-probabilities and state not compared), and for the tree and plain decoding identical in tokens and log-probabilities (198 and 200 cases) [N-E28]; with identical pools, MTP’s divergences from plain decoding at concurrency 1 per fragile position by the previous cycle’s commit length are compatible with one common rate (a failure to reject, positions treated as independent) and none falls at a non-fragile position [N-E27]; where a decode checkpoint was restored, warm and cold continuations agree in tokens except at two exact ties, and elsewhere differ only at exact ties or one step, as plain decoding does, though few decode checkpoints were restored [N-E28]; the declared first-cycle test on 960 fresh prompts is inconclusive (first-cycle divergences at 8 of 162 fragile positions against 842 of 11,252 later, odds ratio 0.68 with +0.5 per cell; neither one-sided lower bound above 0) [N-E30]; aborts on a four-request pool leave deterministic plain decoding’s probes token-identical (40 of 40) and MTP’s in 28 of 40, the rest at exact ties or one step, which the batch shape or request history can give alone (slot assignment not recorded); run-to-run repeats are bitwise identical (176 of 176 pairs); chunked prefill changes generation only at ties or one step, but prompt-position log-probabilities drift by up to 5.22 nats, unexplained [N-E29]
H6: other layers of the stack leave measurable headroom	Attribution and controlled ablations (Appendix E.2 of the paper and Section 3.6)	Measured shares: FP32 state 41.6% of a plain step at $B = 128$; host gaps about 21% of an unprofiled MTP cycle at $B = 1$ [N-E31,N-E36]; removing blocking planning reads and cheapening host planning shortens the MTP cycle by 5.9–8.8% in held batches and raises served per-request rate by 7.5–9.6% at $c = 1$ –32, token-identical, and by 1.0–2.2% on an untuned DFlash arm with FlashInfer draft attention (one session each) [N-E48]; faster backbone GEMM kernels would save at most 5.7% of a plain step at $M = 1$ (derived from isolated microbenchmarks), and served on tuned plain decoding the routing table is 3.4% faster at $c = 1$ and 1.0% at 128, exact up to rounding, and gains nothing material on MTP with FlashInfer or Triton attention (one session each) [N-E47]; ablations by removal not run
H7: reformulations and declared approximations multiply gains well beyond tuning	Bytes per step, quality–speed curves, end-to-end sweeps (Appendix E of the paper)	Ceilings derived [N-E39]; first single runs [N-E40]; the two declared approximations, timed alone in three sessions, are both outside the lossy budget: FP16 state serves 1.16–1.17× the throughput of the best non-lossy arm (replayed decoding, exact up to rounding) at $c = 64$ –256 for 1.0–1.4 GSM8K points, and the int4 target with its int4 drafter fails the logit probe and is slower above $c = 1$ [N-E49]; the interaction matrix and both frontiers not measured
Adaptive sampled depth stays exact	Tiny laws with candidate-dependent stopping	Settled by prior art and Theorem B.3 of the paper (its Appendix B.5)
P1: a certificate on the decision can skip the final layer’s MLP and the head	Oracle-radius replay at the cut (Section 7.2)	Rejected by its kill test [N-E4]

Question or claim	Deciding or damaging test	Status
P2: long-window exact repair from a DFlash window beats optimized DFlash substantially	Perfect-continuation oracle, then real windows (Section 5.6)	With SGLang’s FlashInfer GDN verify kernel, perfect blocks meet the $5\times$ target only at $B = 256$ with a free drafter ($5.01\times$; $4.68\times$ with DFlash’s drafting cost); with its Triton kernel (exactness not classified), from $B = 64$ ($5.21\times$) [N-E65,N-E66]; with the Triton kernel a block drafter would need to commit 61.1 of 64 tokens per cycle at $B = 64$, a per-position acceptance of at least 0.9985 if it were the same at every position, against 0.80–0.91 measured at block 16 (derived) [N-E67]; one-step recycling does not help, and exact sweeps gain about one token each [N-E68,N-E69]; in the engine each exact sweep adds 0.84–1.58 accepted drafts at $B = 16$ (1.07–1.90 at $B = 32$): rejected as a route to $5\times$ [N-E70]
P3: target-anchored residual repair is much cheaper than recomputation	Oracle and real-correction kill tests (Section 5.6)	Refuted: rank-128 bases leave a median relative error of 0.64–0.81 on held-out changes, repaired decisions agree at 37% of changed positions, and four sweeps add at most 0.11 accepted drafts [N-E69]; the two-pass design also misses the target with the FlashInfer verify kernel (ceiling $4.78\times$ at $B = 256$), though it would pass with the Triton kernel (exactness not classified) [N-E65,N-E66]
P4: writing the recurrent state less often, bit-exactly, gives at least $1.10\times$ decode throughput at batch 128	Bit-exactness check, then paired timing (Section 3.5)	Rejected: served decode rate at exactly 128 running requests $1.0042\times$ dense decoding (95% interval 1.0026–1.0058) against the required $1.10\times$; the output probe at concurrency 1 found no difference, which does not establish end-to-end exactness [N-E45]. Bit-exact at kernel level on synthetic activations; kernel $1.22\times$ at batch 128, derived about $1.08\times$ per decode step [N-E43]
The recurrent state compresses exactly	Exact witnesses; rank audit of captured keys	Witnesses committed [N-E21]; audit not run
P5: exact computation can be shared across unrelated requests	Analysis, exact witnesses, measured share of the one structural candidate	Rejected in analysis and by its pre-registered criterion (at most 0.60% of a step) [N-E38]
P6: a draft selector trained for rate beats the same selector with the strongest matched objective by $1.25\times$	Candidate-support screen, then a matched-objective comparison (Section 5.5)	Survives the screen (oracle bound 10.14 against 6.20 tokens per cycle) [N-E60]; comparison not run
P7a: every exactness claim names its reference and one of five contracts	Witnesses that separate the contracts (Appendix D.1 of the paper)	Adopted throughout; witnesses committed [N-E20]
P7b: certified block-parallel GDN verification beats strict replay	Captured-trace arms: reference, strict replay, certified fast, unchecked fast (Section 5.9)	SGLang’s chunked kernel is not a usable fast path (synthetic inputs, one request) [N-E51]; the certified path not tested
P9: reusing a cached DFlash window once after a rejection beats fresh drafting	Support oracle on the post-rejection boundaries, then the cost-charged two-cycle test with a real program and a matched refiner (Section 5.7)	For P9’s program, conditioned on the whole corrected prefix, over offline-recomputed candidate sets, with a padded block-16 verify and reuse at every possible boundary at $c = 1$, the oracle bound does not reject reuse with top-8 and top-16 sets ($\Delta = +0.22$ and $+0.99$ tokens per boundary; an upper bound, not a speed-up) and rejects it with top-1, top-2 and top-4 [N-E71]; the bounds with a free reused verify and with an oracle gate reject none of the tested sets; charging the measured verify of the remainder’s width gives $+0.39$ and $+1.17$ and still rejects top-1, top-2 and top-4; at concurrency 8 and 16, crediting one reusing request with the measured growth of the batched draft rejects every set, top-16 included; real program not built
P-A: innovation-clock drafting reaches $5\times$ at $B = 64$ to 256	Perfect-event oracle (A1), then the support of controllers built from DFlash’s candidates and the interpreter (A2); rules committed before the oracles (Section 5.8)	No-go: A1, an oracle upper bound, passes only with 32 free perfect overrides per cycle; candidate-based controllers’ support is 0.924 per position against 0.9892–0.9925 needed, measured over block positions 5–15 at the stock block-16 anchors [N-E72]

Question or claim	Deciding or damaging test	Status
P-B: prefix-isolated sparse planning reaches $5\times$ with a few anchors	First-gap bound with a perfect controller and an expander no better than DFlash at block 16 (B1)	No-go: for 2 to 16 anchors at most 14.4–37.0 tokens per cycle at $B = 64$ (44.4 needed) and 43.4 at 256 (111.6), upper bounds; after a post-run correction from 39.7 and, at $B = 64$ with 2 or 4 anchors, from 13.7, which changed no verdict [N-E72]
P-C: causal defect drafting repairs a rejected window by sweeping from the audit’s scores	One-sweep bound with point-predictor interpreters (C1)	No-go: at most 1.60–2.00 accepted drafts per boundary against 3.56 for fresh DFlash (upper bound) [N-E72]
P10: replaying the accepted tail inside the next verify kernel pays	Share of the speculative cycle in SGLang’s exact GDN fold at concurrency 8 and 16; built only if at least 9.09% (Section 5.9)	Rejected without building: 4.9% and 6.6%, so removing the fold outright would make the cycle at most $1.05\times$ and $1.07\times$ faster [N-E61]
P11: restartable prefix-demand verification removes verify work	None run (Section 5.9)	Dropped by its author in favour of P10, since rejected (row above): verify kernels keep the recurrent state in registers, so a shorter suffix does not remove per-token full-state reads (reasoning)
P12: the head compiled backward through the last FFN gives a dictionary that a static tile screen can prune	The static screen with the winner’s exact score known, an upper bound on what any screen with this bound skips (Section 7.3)	Negative at its screen: 0.077% of the rows skipped even with the winner’s score known, the share the same screen skips on the head alone [N-E5]; tighter bounds untested
P13: the GDN recurrent state can be cut to rank r in the key dimension, on an offline basis, within the lossy budget’s form	Training-free projection onto key-energy, query and product bases at $r = 32, 64$ and 96, teacher-forced on the second halves of 8 texts of 2,038 tokens (8,144 positions), against the same rule at full rank	Closed on this evidence: no tested reduction meets a criterion of the budget’s form (KL at most 0.01 nats, top-1 agreement at least 98%); the best, the energy basis at $r = 96$, gives KL 0.133 nats and 93.8% top-1, against the full-rank rotation’s 0.008 and 98.5% [N-E50]. P13 had no committed pre-registered kill test, so the closure is a decision on this evidence; the declared logit probe was not run, and ranks 97 to 127 were not tested
Results transfer	New prompts, a second model or a second GPU	Not tested

3 Time, recurrent state and the serving stack

Appendix E of the paper measures where the time of the stock engine’s decode step and speculative cycle goes and defines the serving protocol behind its frontier. This section holds the rest of that appendix: the yardstick for every lever, the ranked costs, derived ceilings for stacks of levers, how speculation interacts with the recurrent state, the proposals P4 and P5, the levers measured so far and the two stacks. Every measurement here is of the stock engine or of a lever applied to it; none uses the certified head.

3.1 Useful progress per unit time

The yardstick for every lever is useful progress per unit time. Let a cycle commit R output tokens in elapsed time $T > 0$. Under a stationary regenerative model with finite expectations the long-run rate is

$$\lambda = \frac{\mathbb{E}R}{\mathbb{E}T}, \quad (1)$$

the ratio of accumulated reward to accumulated time, not $\mathbb{E}[R/T]$, and not a queueing-latency model. A change improves the rate exactly when $\mathbb{E}R_1/\mathbb{E}R_0 > \mathbb{E}T_1/\mathbb{E}T_0$. Higher acceptance does not compensate for arbitrary overhead, and lower acceptance can be worthwhile when the saved work exceeds the lost progress. The denominator includes drafting, verification, the head, refinement, completion, state commit, non-overlapped dispatch and control. A fixed-width target pass evaluates all admitted positions even when the first proposal is rejected, and overlapped kernels require a critical-path measurement rather than a sum of kernel durations. If positive intervals enclosed the expected progress and time of a candidate and a baseline, $R_1^-/T_1^+ > R_0^+/T_0^-$ (with $T_0^- > 0$) would certify $\lambda_1 > \lambda_0$; benchmark confidence intervals are not deterministic enclosures, so the inequality says what must be estimated rather than certifying a speedup.

3.2 Ranked costs

Table 3 ranks the measured costs of the stock engine (Appendix E.2 of the paper) by the Amdahl ceiling $1/(1 - f)$ of removing each one, a bound for removing a component under otherwise unchanged execution, not a predicted gain. The other layers of the stack leave more headroom than the head (H6).

The plain step beyond the head If every weight GEMM ran at the head GEMM’s rate the step would be 469 μ s ($B = 1$) to 923 μ s ($B = 128$) shorter [N-E31]. The recurrent kernel’s time is linear in the batch, 28.9 μ s per request per step, which would be 3.47–3.48 TB/s at $B = 32$ and 128 if every state byte reached DRAM during the kernel. Nsight Compute shows that at $B = 32$ about 27 of the 67 MB it writes are still in L2 when it ends and DRAM runs at 67% of peak; at $B = 128$ the same per-request rate puts DRAM at about 82% of peak, or at 77–87% if anywhere from none to all of the 60 MB L2 still holds writes when it ends (derived) [N-E35]. In bytes the state equals the weights at $B = 84$; in time the crossover comes later, near $B = 110$ –120, because the weight GEMMs run below the peak while the state kernel’s time matches near-peak traffic for its modelled bytes, and at $B = 128$ the recurrent kernel alone (3.70 ms of wall time) already takes longer than all weight GEMMs together, head included (3.47 ms) [N-E31, N-E34]. The host is not on the critical path: each graph launch is issued 1.7 ms ($B = 1$) to 5.9 ms ($B = 128$) before the GPU starts it, and the GPU is busy 95.7–98.3% of the step. The margin is finite, though. At $B = 1$ the scheduler thread spends only 0.73 ms of a 3.54 ms step blocked on the GPU, so a GPU-side speedup much above 1.25 $\times$ at batch one would make the host the bottleneck.

Where the MTP cycle idles The idle time has one main cause [N-E36]. Outside graph replays the GPU waits about 3 ms per profiled cycle at every batch size, and the draft and verify graphs are launched only 44–46 μ s and 380–411 μ s before the GPU reaches them. NVTX ranges on the speculative worker’s host functions and samples of the scheduler put 1.2–1.5 ms of exposed host time per cycle in the attention planning for the verify pass (FlashInfer’s prefill planner, reached through the verify preparation and the graph runner’s batch loading, with blocking device-to-host copies), of which only 14% overlaps GPU work at $B = 1$, and another 0.5 ms in the draft pass’s planning; the host work of the draft-extend pass is 97% hidden behind the verify graph. Subtracting the traced GPU-busy time from the cycle measured without a profiler leaves about 21%, 20% and 15% of the cycle idle at $B = 1$, 8 and 32 (Appendix E.2 of the paper).

Table 3. Measured costs and their Amdahl ceilings $1/(1 - f)$, with f the share of the step or cycle in the stated configuration. For the idle time the parenthesized values are the share derived without the profiler and its ceiling. Measured [N-E31, N-E36].

Cost	Configuration	f	Ceiling
FP32 recurrent state read and written every step	plain, $B = 128$ ($B = 32$)	41.6% (18.0%)	1.71 $\times$ (1.22 $\times$)
GPU idle in the cycle, 35.5 of its 37.3 points in host-bound gaps outside the graph replays (attention planning)	MTP, $B = 1$	37.3% ($\approx$ 21%)	1.59 $\times$ ($\approx$ 1.27 $\times$)
GDN verify kernel and state commit, of which only the per-position FP32 state writes and the commit copy are removable	MTP, $B = 32$	27.2%	1.37 $\times$ (whole)
Weight GEMMs below the head GEMM’s bandwidth (excess time only)	plain, $B = 32$ ($B = 1$)	15.5% (13.2%)	1.18 $\times$ (1.15 $\times$)
Three full-vocabulary draft heads per cycle	MTP, $B = 1$ ($B = 32$)	12.9% (8.5%)	1.15 $\times$ (1.09 $\times$)
The target head chain	plain, $B = 1$ ($B = 128$)	10.3% (6.1%)	1.11 $\times$ (1.07 $\times$)
65 RMSNorm launches per step	plain, $B = 8$	4.8%	1.05 $\times$
Gaps between graph nodes	plain, $B = 1$	3.0%	1.03 $\times$
Eager sampling and bookkeeping after each replay	plain, $B = 1$	2.6%	1.03 $\times$

3.3 Ceilings under a stated execution model

A decode step streams the linear weights once whatever the batch size, $W = 8.41$ GB, and everything else is per request: 100.7 MB of FP32 recurrent state read and written, and 32 KiB of attention KV per context token (Table 19 of the paper). With F the floating-point operations per token (two per weight), s the per-request bytes per step, BW the measured HBM read bandwidth and P an achievable tensor-core rate, and with each layer’s weight GEMMs and its per-request kernels running in turn, as they do at the pinned commit, the step time at batch B is bounded below by

$$t(B) \geq \max\left(\frac{W}{\text{BW}}, \frac{BF}{P}\right) + \frac{Bs}{\text{BW}}, \quad (2)$$

since in that order the weight stream and the GEMM arithmetic can overlap each other while the per-request traffic adds to them. An engine that overlapped the memory-bound per-request kernels with the batch GEMMs would be bounded only by

$$t(B) \geq \max\left(\frac{W + Bs}{BW}, \frac{BF}{P}\right), \quad (3)$$

whose large-batch limit is $\min(BW/s, P/F)$. Equation (2) models the engine as it runs; Equation (3) is the overlapped roofline at the same inputs, for an engine that splits the batch to overlap the two kinds of kernel. Neither is a bound for every engine on this machine. The bandwidth input is the 3.79 TB/s measured over the head’s footprint [N-E33], and P is assumed to be 70% of the BF16 and 60% of the FP8 datasheet peaks, assumptions stated so that the ceilings can be recomputed. Two consequences follow. Under Equation (2), as B grows the throughput ceiling approaches $1/(F/P + s/BW)$, set by the per-request state and the arithmetic rather than by the weights, so halving the state’s bytes or writing it less often raises it while a faster head does not; and even with every byte removed, the arithmetic term caps an FP8 engine, so no single lever gives an order of magnitude at high concurrency. At batch one the weights bound the step, and the draft head and the target’s weight bytes are the terms to cut.

Table 4 and Figure 1 evaluate the floor for stacks of levers [N-E39]; they are derived, not measured. At batch one the floor is 2.25 ms per token (445 tokens/s), of which the weights alone account for 2.22 ms; plain decoding measured 280 tokens/s at concurrency one in a single served run [N-E40], 63% of that rate, and 289 tokens/s in the profile’s fixed-batch windows [N-E31]. At large batch the plain engine’s ceiling under Equation (2) is 23.7k tokens/s, because each request moves 114 MB per step and needs 8.4 GFLOP. Halving the state bytes, writing the state less often and moving weights and cache to FP8 raise the ceiling to between 34k and 79k tokens/s. The overlapped roofline is higher, 32.1k tokens/s for the plain engine and 141.2k for the full lossy stack at $B = 2,048$ (the plain engine’s limit as B grows is 33.3k), but under neither model does any stack reach ten times the plain engine’s ceiling: 8.4 GFLOP per token caps an FP8 engine near 140k tokens/s at the assumed rate even with every byte removed (about 235k at the datasheet FP8 peak, of which the assumed rate is 60%; derived from the same constants).

Table 4. Derived large-batch throughput ceilings under the layer-serial model of Equation (2), which assumes that per-request kernels do not overlap the weight GEMMs, and under the overlapped roofline of Equation (3), at a 334-token context, with the measured read bandwidth of 3.79 TB/s and the tensor-core rate assumed at 70% (BF16) or 60% (FP8) of the datasheet peak. The MTP rows are for the layer-serial model only. Not measurements [N-E39].

Lever stack	MB per request per step	Tokens/s at $B = 2,048$	
		layer-serial	overlapped
Plain, FP32 state	114.0	23.7k	32.1k
16-bit state	63.6	34.6k	55.9k
Replayed state writes, FP32	66.8	33.6k	53.5k
Replayed state writes, 16-bit state	40.0	44.0k	82.3k
Replayed state writes, int8 state	26.7	52.1k	82.3k
Replayed, 16-bit state, FP8 weights and KV	34.6	61.7k	102.6k
Replayed, int8 state, FP8 weights, KV and head	21.2	78.9k	141.2k
MTP (accepted length 3), stock verify, FP32	–	19.5k	–
MTP (accepted length 3), replayed verify, FP32	–	34.3k	–

Repeated engine-only and served measurements up to $B = 1,024$ for each state and weight format were not run.

3.4 Speculation and the recurrent state

The hybrid architecture changes the economics of speculation. A verify pass over D draft tokens must be able to roll back to any accepted prefix, and SGLang’s Triton verify kernel does so by writing one full FP32 state per draft position into an intermediate buffer and copying the accepted one back at commit. Per request and cycle that is one state read, D state writes and a read and write at commit: 352 MB for $D = 4$, three and a half plain steps (derived from the code path and checked against the trace). The two kinds of traffic scale differently with the batch. Speculation saves weight bytes: one verify pass streams the weights once for several committed tokens, and that stream is shared by every request in the batch, so its cost per output token falls as $1/B$. The snapshot writes are per request and per draft position, so their cost per output token does not fall with the batch at all. At batch one the weight saving dominates; at high batch, where the state dominates, speculation’s bandwidth advantage shrinks and can turn into a loss. This cost belongs to a recurrent hybrid; an attention-only model rolls back by discarding cache entries and writes no snapshots. Rollback of recurrent state is an active problem: the Mamba-in-Llama work advances one cached state lazily and recomputes after a rejection (Wang et al., 2024); STree,

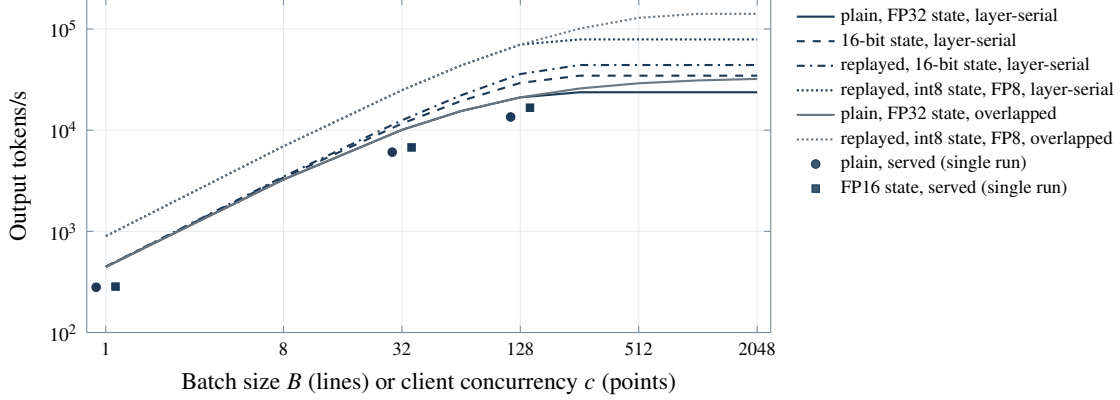


Figure 1. Derived throughput ceilings per lever stack against batch size under the layer-serial model of Equation (2) and, for two stacks, the overlapped roofline of Equation (3), at the measured bandwidth and assumed tensor-core rate (lines [N-E39]; ink for the layer-serial model, grey for the overlapped roofline, one dash pattern per lever stack), and served output throughput of single benchmark runs at client concurrency 1, 32 and 128 (points [N-E40]; the two series are offset slightly left and right of their concurrency so that both stay visible). The gap between a line and a point at the same concurrency is overhead above the layer-serial floor; at high concurrency it includes the server’s streaming path (Section 4).

TreeWY, Bole, GDN Tree-Scan and SpecLA verify trees or chains on SSM and GDN hybrids (Wu et al., 2025; Ghantasala, 2026; Wang et al., 2026a; Ma, 2026; Wang et al., 2026b), and TreeWY states that its outputs are not bit-identical to the baseline; ReplaySSM caches recent inputs instead of writing the state at every step, equivalent to ordinary decoding up to floating-point error (Liou & Dao, 2026).

The measurements agree with the mechanism. At $B = 32$ the verify kernel runs for 2.54 ms and the commit for 1.23 ms per cycle, 27% of the cycle, at an implied 3.2 and 2.6 TB/s [N-E31]. The byte model, at a context of 700 tokens and an accepted length of 2.8, has MTP moving 44% fewer bytes than plain decoding per output token at $B = 1$, 26% fewer at $B = 32$, 6% fewer at $B = 128$ and 2% more at $B = 256$ [N-E34]; the crossover moves with the context, because attention KV is read once per verify pass for several committed tokens, so longer contexts favour speculation. In the profile’s fixed-batch windows MTP’s throughput advantage over plain decoding fell from $1.33\times$ at $B = 1$ to $1.13\times$ at $B = 32$ (Appendix E.2 of the paper). Served single runs on the tuning split show the same turn: MTP with three steps and snapshot verification led plain decoding by $1.64\times$ at concurrency 1 and trailed it from concurrency 32, and replayed verification, which removes the snapshots, raised its throughput at concurrency 128 by 24% (Table 6). At that point the snapshots alone would be written at about 0.59 TB/s if each draft token’s 50.3 MB snapshot is written once per cycle, against 1.35 TB/s of state traffic for plain decoding at the same concurrency under the same assumption (derived from the measured throughput and accepted length) [N-E78]. On the confirmation split, over three sessions with matched plain baselines, every speculative family leads plain decoding through concurrency 32 and trails it from 48, with accepted length flat across concurrency (3.24 to 3.28 tokens per cycle for MTP, 4.66 to 4.81 for DFlash at block 8), so the loss is not a fall in acceptance. At concurrency 128 MTP with replayed verification delivers 0.86 of plain decoding’s throughput, and with SGLang’s snapshot verification 0.68 (one session) [N-E79].

Bytes do not explain the whole cycle at high concurrency [N-E53]. At concurrency 128 the tuning points imply an MTP cycle of 43.5 ms with snapshot verification and 35.0 ms with replayed verification (128 requests times 3.26 tokens per cycle, over the measured throughput), against 9.5 ms per plain step. The implied cycle is derived from client throughput, so it includes the prefill passes of arriving requests and the benchmark’s ramp and drain as well as the decode cycle. Derived from bytes and FLOPs at 3.8 TB/s, 650 TFLOP/s and a context of about 400 tokens, its parts add up to about 21.7 and 13.2 ms: the verify GEMMs over 512 tokens take 6.6 ms, the three draft steps about 1.7 ms, verify attention and small kernels about 1.5 ms, and the GDN state traffic 11.9 ms with snapshots (352 MB per request) or 3.4 ms with replay (about 100 MB). Replay saved about 8.5 ms per cycle, which the 252 MB per request of snapshot and commit traffic it removes accounts for, but the implied cycle is 2.0 to 2.7 times the sum of its derived parts: most of it is time the byte model does not see, in the decode cycle itself or in the prefill, ramp and drain, which this arithmetic cannot separate.

SGLang’s replay path for speculation stores each draft token’s inputs and folds the accepted prefix into the state at commit, reading and writing the state once per cycle, which would make speculation cheaper in state bytes per token than plain decoding; trees do not get this benefit yet, because the replay path handles only chains and a tree keeps one full state per node (Ghantasala, 2026; Ma, 2026). The block drafter is more exposed than the MTP layer: a 16-token DFlash block writes about 805 MB of states per request per cycle (Section 5.1).

Verifying a draft block in parallel over its positions (P7b), instead of through the sequential recurrence, would make wide blocks cheap only with a fast path that is both faster and close enough to the recurrent outputs to be certified. SGLang’s

chunked GDN kernel is neither at one request [N-E51]. On one layer, with synthetic activations and the checkpoint’s gates, its outputs over a block of 2 to 16 tokens differ from the packed decode’s in 52–55% of BF16 output words, and it takes 815–990 μ s per layer at block widths 4 to 256, against 75–378 μ s for the recurrent verify kernel with its per-position snapshots. A purpose-built block-parallel kernel is untested.

State precision is the cheapest lever and the most dangerous. DAMP reports AIME 2026 accuracy of 85.5 with FP32 recurrent state, 84.6 with FP16, 79.7 with BF16 and below 30 with FP8 or int8 on another GDN model (Zhang et al., 2026b), and LeapQuant recovers near-FP32 accuracy with 8-bit state by re-quantizing once per 16-token window (Pan et al., 2026). SGLang accepts FP16 state (`--mamba-ssm-dtype float16`). With plain and replayed decoding at concurrency 64 to 256, FP16 state is faster than the best non-lossy arm but outside the lossy budget’s GSM8K band (Section 3.6) [N-E49]. MTP with and without replayed verification at concurrency 128 to 1,024, and the logit-probe KL, divergence rate and GSM8K accuracy for BF16 and FP8 state, were not measured.

3.5 How much recurrent state an exact computation needs

The dense FP32 state is sufficient for exact decoding; nothing yet shows that it is necessary. Two questions should be kept apart: whether less information could be retained, and whether the state has to be materialized as often as it is.

For the first, fix one GDN head of one layer and treat the layer’s inputs as given. Along any permitted history the state evolves as $S_t = S_{t-1}A_t + B_t$ with $A_t = \alpha_t(I - \beta_t k_t k_t^\top)$ and $B_t = \beta_t v_t k_t^\top$, which depend on the inputs and not on the state, and the head reads $o_t = S_t q_t$. A state difference ΔS at time t therefore propagates linearly, to $\Delta S A_{t+1} \cdots A_{t+j} q_{t+j}$, and is observed as $\Delta S A_{t+1} \cdots A_{t+j} q_{t+j}$.

Proposition 3.1 (What an exact state must retain). *Let $\mathcal{S}$ be the set of states reachable over permitted histories, $\mathcal{D} = \text{span}\{S - S' : S, S' \in \mathcal{S}\}$, and $\mathcal{N}$ the set of differences ΔS with $\Delta S A_{t+1} \cdots A_{t+j} q_{t+j} = 0$ for every permitted continuation and every $j \geq 0$. Two reachable states produce the same outputs under every permitted continuation if and only if their difference lies in $\mathcal{N}$. If $\mathcal{S}$ contains a relatively open subset of its affine hull, then a linear map that separates every pair of reachable states with different outputs has rank at least $\dim \mathcal{D} - \dim(\mathcal{D} \cap \mathcal{N})$ on $\mathcal{D}$.*

Proof. Because B_t does not depend on the state, the output difference under a continuation is the linear expression above, and it vanishes for every continuation exactly when $\Delta S \in \mathcal{N}$. For the bound, take $d \in \mathcal{D}$ with $\phi(d) = 0$. By the hypothesis there are a reachable S and an $\varepsilon > 0$ with $S + \varepsilon d$ reachable; ϕ gives them the same image, so they must have the same outputs, and $\varepsilon d \in \mathcal{N}$. Hence $\ker \phi \cap \mathcal{D} \subseteq \mathcal{N} \cap \mathcal{D}$, and the rank of ϕ on $\mathcal{D}$ is at least $\dim \mathcal{D} - \dim(\mathcal{D} \cap \mathcal{N})$. $\square$

The hypothesis matters: if only one state is reachable, $\dim \mathcal{D} = 0$ and a representation with no dimensions is exact. Treating every input sequence as permitted is conservative, since in the full model a layer’s outputs influence later tokens; it is the notion that makes a reduction safe for any request. Exact witnesses show that neither $\mathcal{D}$ nor $\mathcal{N}$ is favourable without structure in the keys [N-E21]. Writes with unit coordinate keys and gates inside $(0, 1)$ reach any matrix, so the reachable set can fill the whole state space and $\mathcal{D}$ with it. A direction the current query cannot see is not unobservable: with $\alpha = \beta = \frac{1}{2}$, $k = (\frac{3}{5}, \frac{4}{5})$, $q = e_1$ and $\Delta S = u e_2^\top$ for any value vector u , the observed difference $\Delta S q$ is zero now, but one step later it is $\Delta S A q = -\frac{3}{25}u$. Each transition is a contraction, not an erasure: $\det A_t = \alpha_t^d (1 - \beta_t \|k_t\|^2) > 0$ when $\beta_t \|k_t\|^2 < 1$, so no step forgets a direction exactly. Transitions with non-orthogonal keys do not commute, so there is no common basis in which they are diagonal. And because a gated RMSNorm precedes the output projection, the null space of the projection is not unobservable: two outputs that $W_o = (1, 0)$ cannot tell apart, $(1, 1)$ and $(1, 7)$, project to 1 and $\frac{1}{5}$ after normalization.

One structural reduction is exact. If every key lies in the column space of a fixed orthonormal $U \in \mathbb{R}^{d_k \times r}$ and $S_0 = Z_0 U^\top$, then $S_t = Z_t U^\top$ with

$$Z_t = \alpha_t Z_{t-1} + \beta_t (v_t - \alpha_t Z_{t-1} \kappa_t) \kappa_t^\top, \quad \kappa_t = U^\top k_t,$$

the same recurrence in dimension r , since $U^\top A_t (I - U U^\top) = 0$ and the write $\beta_t v_t k_t^\top$ stays in the form exactly when $k_t \in \text{col}(U)$. The keys are produced by a short convolution, a SiLU and a normalization, and SiLU raises rank (SiLU(log 2 [[1, 2], [2, 4]]) has rank two), so a low-dimensional input does not imply low-dimensional keys; whether real keys are confined to a subspace is a measurement, which would have to be certified by exact or modular rank rather than an SVD threshold, and it was not made. With U fixed per layer and head and shared across requests, the per-request state is $d_v r$ values instead of $d_v d_k$, a saving for every $r < d_k$; a per-request low-rank factorization $S = A B^\top$ instead stores $d_v r + r d_k$ values, which beats the dense state only below $r = 64$ at $d_v = d_k = 128$ and halves it only at $r \leq 32$ (derived). Floating point adds a further obstruction: an exact rational reduction does not preserve bitwise results. The split form of the recurrence changed 1,481 of 1,600 FP32 outputs (Appendix D.1 of the paper), deferring a decay reassociates FP32 roundings (one ulp apart for $\alpha_1 = \alpha_2 = 0.9$, $x = 1.3$), and rounding an exactly rank-one BF16 outer product to BF16 gives a 32×32 matrix of exact rank 28 [N-E21].

Errors that do enter the state stay bounded as long as each transition is non-expansive: $(I - \beta k k^\top)$ has spectral norm at most 1 exactly when $0 \leq \beta \|k\|^2 \leq 2$ (its eigenvalues are 1 and $1 - \beta \|k\|^2$), so a perturbation η_t injected at each step obeys $\|E_t\| \leq |\alpha_t| \|E_{t-1}\| + \|\eta_t\|$; with normalized keys and $\beta \in (0, 1)$ the condition always holds [N-E20].

The second question has a better answer. A decoder that keeps a dense anchor and the recent actual FP32 operands of each step, and replays them in the reference kernel’s order, reproduces every state word bit for bit by induction, while reading the dense state once per step and writing it once every m steps. That reduces information movement, not information, and it is what replay-based rollback already does (Liou & Dao, 2026; SGLang contributors, 2026a). The paragraphs below treat it as a lever with a pre-registered test (P4), and the opposite idea, sharing exact computation across requests (P5).

Writing the recurrent state less often (P4, rejected) Rounding-preserving live replay is exact and cuts dense-state traffic per step from $2D$ (read and write) to $D + D/m$ when the state is flushed every m steps, 0.625 of the original at $m = 4$, for a few percent more storage (derived). If the recurrent kernel is a fraction f of a decode step, a kernel twice as fast gives at most $1/(1 - f/2)$, removing it entirely $1/(1 - f)$, and the $m = 4$ traffic cut alone $1/(1 - 0.375f)$. The measured share at $B = 128$ and a 418-token mean context is $f = 0.416$ (Table 20 of the paper), which gives $1.26\times$, $1.71\times$ and $1.18\times$ (derived from a measured share). The pre-registered test has two parts. A kernel check compares every state word and GDN output of the replayed and dense paths, one layer at a time, and one differing word rejects bit-exactness (below). The served timing decodes 512 tokens greedily at batch 128 from 2,048-token prompts with FP32 state and no speculation, with the threshold at $1.10\times$; through the server only the output probe below checks exactness. The served timing’s design, metric and decision rule were declared before its first run, and every later change before the run it governs [N-E44]. Dense decoding and replay at $m = 4$ run as four server pairs in the order A B B A A B B A with identical memory pools. The primary metric is the server’s decode rate at exactly 128 running requests: the harmonic mean of the scheduler’s generation-throughput windows (one per 40 decode passes) that show 128 running requests on that line and the previous one, with no prefill batch between them, inside the AIPerf profiling phase, which excludes warm-up, ramp and drain; fewer than 8 such windows in any arm voids the run. It measures decode only, which is what the lever changes, and client throughput, which includes the 2,048-token prefills, is reported beside it. The four log ratios give a 95% t interval, and the claim is rejected if its upper end is below 1.10 , supported if its lower end reaches 1.10 , and inconclusive otherwise; a supported result is a served decode throughput at 128 running requests, not an end-to-end speedup. A server output probe compares greedy tokens and top-20 log-probabilities at concurrency 1 with dense decoding, with a second dense server as the noise control: a difference refutes end-to-end exactness unless the control also differs from the first dense server, in which case the probe is undecided; a pass does not establish exactness; and a refuted probe leaves P4’s exact claim unsupported whatever the throughput interval. A run that fails its completeness, workload, pool, environment or provenance checks is void and its numbers are not reported. One check was corrected after the first run and before any verdict was computed: the sent prompts are now matched by the hashes of their texts, because the benchmark’s request ids differ from the declared ones for this workload, a check no weaker than the declared one. Three attempts have been void, because in every arm at most 127 of the 128 requests were admitted. The amendments declared after the first two, a pinned attention cache of 655,360 tokens, 132 pinned recurrent-state slots and an admission preflight that must show 128 running requests, did not change that: in the third the attention cache was at 40% use and 127 of 132 recurrent-state slots were taken. Read from the engine source, the admission check counts a request whose last prefill chunk runs in the pass twice: the request-table row it keeps from its first chunk lowers the admission limit, and it also counts against that limit as one of the pass’s requests. A wave therefore stops one short of the running limit whenever its last pass carries a chunk tail, and the 128th request runs only after the wave finishes. Every one of the 56 admission waves in the three attempts’ server logs had such a tail and stopped at 127, as predicted [N-E44]. Those waves test the prediction only at that one limit and one chunk; the discriminating test is the amended admission preflight. Before any further run the running limit was raised to 129, with the client still at 128 concurrent requests, so that at most 128 requests are ever in the server and the prediction is a peak of 128 running in both arms. The amended preflight showed exactly that, and the served test then ran (below). SGLang’s own replay path qualifies only if it passes the bit-exactness check. The replay kernel is committed as an engine patch with a one-layer test that compares every output and state word against the stock decode kernel (tests/test_gdn_exact_replay.py [N-E42]).

P4 at kernel level The replay is bit-exact at kernel level [N-E43]. Run side by side with SGLang’s packed decode for 48 steps at batch 8, on synthetic activations with the checkpoint’s gates, with rows flushing at staggered phases and the reconstructed state written out and compared at every step, it differs in none of 201,326,592 state words and none of 1,572,864 output words, at $m = 4$ and 16 on the first GDN layer and at $m = 4$ on layer 20. SGLang’s own replay path differs on the same inputs in 465,102 and 543,225 of the 1,572,864 output words (30% and 35%, layers 0 and 20), so it does not qualify. Timed on one layer under CUDA-graph replay with a cold L2, the kernel is $1.215\times$ faster than the packed decode at batch 128 and $1.227\times$ at batch 256 with $m = 4$, not the $1.6\times$ the bytes allow: at batch 128 it takes 89, 112 and 129 μ s on steps that replay zero, one and two earlier updates and 201 μ s on the step that replays three and writes the anchor, against 161.5 μ s for the packed decode; with $m = 8$ and 16 it is no faster than the packed decode. With the recurrent kernel at 41.6% of a batch-128 step, a kernel $1.22\times$ faster predicts about $1.08\times$ for the decode step, and less with the pre-registered 2,048-token prompts, where attention takes a larger share: below the $1.10\times$ the test requires (derived). The check covers one layer at a time on synthetic activations.

P4 served: rejected The served test rejects P4’s throughput claim [N-E45]. Its run passed every declared validity check: every server resolved the pinned pools (129 running requests, 655,360 attention-cache tokens, 132 recurrent-state slots), every arm completed 256 of 256 requests on the declared prompts, and each had 24 decode windows at exactly 128 running requests. Over the four pairs, replay at $m = 4$ decoded 1.0042 times as fast as dense decoding (95% interval 1.0026–1.0058), and every pair favoured replay, by 0.31 to 0.55%. The upper end is below 1.10, so the declared rule rejects the claim. Client throughput, which includes the 2,048-token prefills, was 1.0014 times dense decoding’s. The output probe found no difference: at concurrency 1, on 48 prompts of 256 greedy tokens, replay matched the dense server in every token and top-20 log-probability, and so did the second dense server. The probe can see state rounding of this kind, since SGLang’s replay path, FP16 state and BF16 state kept the dense server’s tokens in only 22, 20 and 17 of the 48 sequences, and all three changed the top-20 log-probabilities. A pass still covers only the emitted tokens and top-20 log-probabilities at concurrency 1, so it does not establish end-to-end exactness; bit-exactness is shown only at kernel level (above). The gain is also far below the derived expectation of about 1.07 to 1.08 times. The dense step at 128 running requests took 10.77 ms and the replay step 10.73 ms, a saving of 0.045 ms, where the one-layer timing predicts 0.69 ms over 24 layers (derived): about 6.5% of the kernel’s saving reached the server. Reading the engine code suggests a cause, untested. With replay enabled, stock SGLang’s ring-cursor bookkeeping runs outside the CUDA graph before every decode step and makes the host wait for the GPU twice (a boolean-mask selection and a `torch.unique`), and under the overlap scheduler such waits could leave the GPU idle for most of the saving. A trace of both arms would test this. Even the whole kernel saving would give only about 1.07 times, below the threshold.

Sharing computation across requests (P5, rejected) Factoring hidden or recurrent state across unrelated requests does not survive analysis. An exact factorization with a dense residual costs the full projection, so it saves only if $r(d+B)+z < Bd$ with a zero or sparse residual; SiLU and gating raise rank; full-rank pairwise state differences force private factors at least as large as the dense state; one gated-delta step takes four identical states to batch rank four [N-E21]; online factorization costs of order B^2 times the update; and reassociation breaks equality with the reference. The one exact candidate is structural: the first GDN layer’s input projections (12,352 outputs) depend only on the current token, so they could be read from a precomputed per-token table (5.71 GiB for the whole vocabulary) with convolution and recurrence kept private. Its derived ceiling is about 0.75% of the dense projection work per token, and at batch 128 even a 99% table hit rate would leave about 72% of batches with a miss. The pre-registered criterion rejects the table unless the stage’s measured share of a decode step allows a 1% end-to-end gain. It does not: layer 0’s input projections take 0.60%, 0.57%, 0.51% and 0.33% of a plain step at $B = 1, 8, 32$ and 128, a ceiling of $1.006\times$ [N-E38], so P5 is rejected by its criterion as well as in analysis.

3.6 Levers by axis

Table 5 lists the levers, each built as a separate server flag or environment variable so that any subset composes. A lever is *exact* when it leaves the target’s arithmetic unchanged, so every emitted token is the target’s decision at its own batch shape under the stock-kernel contract, with its measured noise floor; drafting levers are exact because the target verifies every token. A lever that changes the target’s arithmetic, a change of its attention backend included, is classed by its greedy outputs against a matched stock reference (Section 4): *exact up to rounding* if every first divergence is at an exact tie, within one BF16 step or a near tie within two steps, the classes of the stock engine’s own batch-shape divergences (Appendix D.4 of the paper), and *lossy* otherwise, in which case it carries a measured quality cost. SGLang’s replay path for plain decoding is exact up to rounding by that comparison [N-E80], although it is not bit-exact: its decode kernel reconstructs the state on tensor cores from BF16-cast operands and persists that rounding at every flush (code reading at the pinned commit), and on synthetic activations its outputs differ from the packed decode’s in 30 to 35% of words (Section 3.5). The bit-exact alternative is the rounding-preserving replay (P4), whose served decode rate was only 1.0042 times dense decoding’s (Section 3.5). Replayed verification under MTP is classed the same way: its replay fold is written as a bitwise clone of the verify kernel’s state update, but the verify outputs come from a chunked transform on TF32 tensor cores, whose rounding differs from the recurrent kernel’s by more than a reassociation; that mechanism suggested lossy, but every first divergence of the buffered MTP arm from stock MTP is rounding-level [N-E80].

The first single-run measurements on the benchmark harness (plain arm with the prefix cache on, capacity 128) separate the levers by where they act [N-E40]. FP16 recurrent state raised served throughput at concurrency 128 from 13.5k to 16.7k tokens/s (1.24 $\times$) and changed nothing at concurrency one, which is what a per-request byte term predicts; three paired sessions and its quality cost follow below. These runs streamed one token per chunk, and at that concurrency the streaming path rather than the GPU can limit a plain arm (Section 4), so the ratio may understate the lever; the paired sessions below give 1.25 $\times$ at concurrency 128 with capacity 256. FP8 KV did nothing at these short contexts (0.99 $\times$). FP8 W8A8 weights could not use their CUTLASS GEMM: the one in the installed `aarch64_sg1-kernel` aborts at run time because that binary was not compiled for `sm_90a`, as its logged error states (Hopper’s asynchronous warp-group MMA instructions exist only for that target (NVIDIA, 2026b)), and the Triton route showed no consistent gain (0.96–1.06 $\times$ across concurrencies), a negative result until a different GEMM route is wired. A first engine-only pass of decode steps at batch 512 took 26.9 ms with FP32

state, 23.0 ms with BF16 and 22.5 ms with FP16 [N-E41]; it is noisy at small batch sizes and was not repeated. All of these are single runs without repeats.

The lossy levers, measured Two lossy levers were timed against the tuned non-lossy arms in three pre-registered sessions and charged against the lossy budget of Section 3.7 [N-E49]: FP16 recurrent state at capacity 256, and the int4 quantization-aware-distilled target with its int4 DFlash drafter (Kim et al., 2026a). Neither is exact, and the two were not run together: the int4 arms keep the FP32 state and the FP16 arms the BF16 weights, so no lossy stack was measured. FP16 state is faster at each tested concurrency: 1.163, 1.158 and 1.171 times the throughput of the best non-lossy arm (replayed decoding at capacity 256, exact up to rounding) at concurrency 64, 128 and 256, every session beyond the 2% band. The plain pair’s gain matches the state traffic: FP16 saves 50.3 MB per request and step, which at the GDN kernel’s measured 3.47–3.48 TB/s is 0.93, 1.85 and 3.70 ms at concurrency 64, 128 and 256, against measured reductions of the median inter-token latency, FP16 against FP32 plain decoding, of 0.93, 1.84 and 3.52 ms (derived). Its logits change measurably but stay inside the probe’s limits. In score mode, which teacher-forces the reference’s tokens through the chunked prefill kernel and so cannot see a state’s decode-path error, the FP16 runs agree with the reference at its own noise (99.4% top-1 agreement). On the decode path they diverge four to six and a half times as often as a second launch of the reference, with about seven to eight times its mean top-20 KL (0.0002 nats, still more than forty times inside the ceiling). On GSM8K they score −0.99 and −1.29 points (plain decoding) and −1.06 and −1.36 (replayed decoding) against the two declared reference runs, with McNemar p between 0.18 and 0.36 and 95% intervals reaching about −3.3 points. That misses the declared rule (at least −1.0 against both references), so the lever is outside its band: plain decoding meets it against one reference and misses the other by 0.29 points, and replayed decoding misses both, by 0.06 and 0.36. The replayed arm’s loss equals the exact mtp-stockverify arm’s against the same references, and the two FP16 runs sit level with the lowest of the six non-lossy runs (post hoc, $p \approx 0.13$ for their mean against those runs’), so this check cannot tell the loss from run-to-run variation. Against the replayed-decoding run (plain-tuned-replayssm, exact up to rounding, the speed baseline’s family at capacity 128) the gap is about 2.1 points (undeclared, nominal p 0.03–0.05; the exact mtp-stockverify shows the same gap). The result is therefore the trade itself: 16–17% more throughput at concurrency 64 to 256 for a measured logit change and 1.0–1.4 GSM8K points against the declared references, a loss this check cannot resolve.

The int4 pair is slower in both per-request rate and throughput at every concurrency from 2 to 256 (0.715–0.870× the best non-lossy arm’s throughput; from concurrency 64, plain decoding with the int4 target). At concurrency one its throughput is 1.046×, faster under the declared rule, and its per-request rate 1.015×, within the band; why the two ratios differ there was not examined. Its accepted length is 5.0 tokens per block-16 pass, the bonus token included, against 5.7 for the BF16 pair, and its verify pass is faster only at one request (the pass time is derived, as median inter-token latency times accepted length). An isolated microbenchmark gives the direction: Marlin’s W4A16 GEMM takes 0.69 of BF16’s time on the target’s quantized projections up to 8 rows and 0.75 at 16, but 1.19–1.99 times from 32 rows, and a verify pass carries the concurrency times the block size in rows. Its predicted change in step time matches plain decoding within 2–13% and the drafted arms at concurrency 1–2 within 23%, but overpredicts the drafted arms at 4–32 by 33–87% (derived), so there it gives the sign, not the size. The int4 target also fails the probe (top-1 agreement 96.9–97.5%, mean top-20 KL 0.0120–0.0125 nats, first divergence after a median of 26–27 tokens). Its GSM8K run stopped at the hold’s time limit with 1,305 of 1,319 problems scored, so it has no declared GSM8K result, and its verdict rests on the probe.

Two levers need engine changes. Draft-vocabulary methods restrict the draft head to a subset of rows (Zhao et al., 2025; Goel et al., 2025; Williams et al., 2026; Zhang et al., 2025; Chen et al., 2026b), and FastMTP compresses only the draft vocabulary while keeping full-vocabulary verification (Cai et al., 2025); they change only the drafter, so the target’s decision is unchanged under target-only or point-mass verification. SGLang’s hot-vocabulary draft head (--speculative-token-map, after FR-Spec) does the wrong thing on Qwen3.5 as read at the pinned commit: the MTP layer’s head is the tied embedding, and set_embed_and_head ignores the truncated head in that case, so the drafter scores all 248,320 rows while the worker remaps indices as if it had scored the hot subset; the patch unties a reduced head for the drafter only. A hot map ranked by token frequency on the tuning split’s greedy outputs covers 97.5% of the 171,968 output tokens of a plain sweep on the confirmation split with 22,936 rows (every token seen in the tuning outputs), 96.3% with 16,384 and 89.1% with 4,096 [N-E52]. The second change adds a relaxed greedy rule for chains, accepting a draft token whose target logit is within g of the maximum; SGLang’s existing relaxed thresholds act only when sampling, and the rule is lossy by construction, so its quality curve is the measurement (Urbán et al., 2026). Both changes are committed as engine patches with tests that run where SGLang and a GPU are available (tests/test_moonshot_levers.py [N-E42]); their effect and the relaxed rule’s quality curve were not measured.

The host gap of the speculative cycle at low concurrency (Appendix E.2 of the paper) has a configuration-level lever [N-E46]. With Triton attention for the target and the draft, neither attention backend needs host-side sequence lengths, so the overlap scheduler stops copying them to the host and FlashInfer’s planning leaves the cycle. MTP with three steps then gains 1.18× in per-request rate at concurrency 1 and 1.14× in throughput at concurrency 4, with unchanged accepted length; Triton attention for the draft alone gains 1.08× and 1.06× (single runs). The draft-only change is exact, because it changes only which tokens are proposed. Triton attention for the target changes the target’s arithmetic and is exact up to rounding [N-E80].

SGLang’s overlapped planning stream fails at the first verify on this hybrid model, whose GDN backend does not implement a call the stream needs.

Removing the blocking reads from attention planning With FlashInfer attention the host gap has an exact lever as well [N-E48]. Without a profiler, stock SGLang leaves the GPU idle 1.39 to 1.81 ms of each cycle of the tuned MTP arm at $B = 1$ to 128, 20% of the cycle at $B = 1$ and 9% at $B = 128$ (derived: the untraced stock cycle less the GPU busy time per cycle in a host trace of the same configuration; stock and patched traces agree on that busy time within 0.02 ms at $B \leq 64$). For a DFlash cycle at block 8 with FlashInfer draft attention the estimate is 1.43 to 1.53 ms, 22% at $B = 1$ and 10% at $B = 32$ (from the patched engine’s trace, assuming the same GPU work; an earlier session gave 1.36 to 1.46 ms). In host traces the largest site is FlashInfer’s planning of the verify pass for MTP, which reads a mask size and three index arrays back to the host on every verify, and the drafter’s attention planning for DFlash. Engine patches (engine/sglang/patches/hostgap/0001–0005) compute those values on the host from lengths the scheduler already holds and leave the plan unchanged. A fourth patch makes that host arithmetic cheaper and uploads the mask offsets from pinned memory, which also removes a last blocking copy in FlashInfer’s mask setup. A fifth follows FlashInfer’s split-KV rule for NVFP4 caches; it changes nothing with the BF16 cache here, and NVFP4 is untested. On the GPU the plans and a replayed attention graph’s output bits matched FlashInfer’s own on random batches at every tested size. In the serving engine a validation mode compared at least 8,192 plans of each kind with the stock path and found no difference. Greedy outputs were token-identical to the stock engine in the same configuration, with equal verify-step counts, for all 192 requests (64 prompts at concurrency 1, 8 and 32, top- k 1 chains, prefix cache off, running limit and recurrent-state slots pinned): for MTP and for DFlash at block 8 with FlashInfer draft attention, both with all five patches, the DFlash servers with the attention cache pinned at 240,000 tokens. Trees, sampling and the prefix cache are not covered. In held decode batches without a profiler the five patches shorten the MTP cycle by 5.9 to 8.8% (0.50 to 1.20 ms), 36 to 71% of the idle. The first three alone, which only remove the reads, gave 2.7 to 4.8% in a session that ran stock first; the five ran patched first. So at low batch most of the idle was host work rather than waiting on a read. Served on the tuned MTP arm, interleaved as stock, patched, patched, stock, the five patches raise per-request rate by 9.4 to 9.6% at concurrency 1 to 8, 8.9% at 16 and 7.5% at 32. All twelve stock–patched pairs fall between $1.067\times$ and $1.101\times$, with unchanged accepted length at concurrency 1 to 16 (one session). What is left is the host path before the draft: the host still needs 1.67 to 1.82 ms (traced) from the previous verify’s completion to the next draft’s launch, while 0.54 to 0.92 ms of GPU work remains queued. The patches change nothing on the tuned DFlash arm, whose draft uses FA4 attention and whose verify already plans without blocking. With FlashInfer draft attention, on an untuned DFlash arm, the five patches gain much less: the per-request rate rises by 1.0 to 2.2% at concurrency 1 to 32 in one interleaved session (all twelve pairs between $1.004\times$ and $1.028\times$), and held batches shorten by 0.9 to 1.7%, 4 to 13% of that arm’s idle. There the drafter’s six blocking reads come after the host has already waited for the previous verify’s lengths, so they barely wait, and the drafter’s planning still runs while the GPU idles. The tuned arm serves 4 to 12% more per request than the patched one (derived, across days), so the DFlash frontier does not move. On the tuned arm the one synchronization per cycle that remains matches this wait for the previous verify’s lengths in count and size, which suggests (it was not traced there) that its host gap of at most about 0.66 ms at concurrency 1 [N-E37] is host work after that wait rather than a blocking read.

Backbone GEMMs and RMSNorm The weight GEMMs’ time above the head GEMM’s bandwidth and the 65 RMSNorm launches are the fourth and seventh rows of Table 3. Isolated microbenchmarks under CUDA graphs locate the slow GEMMs and test faster kernels that read the same BF16 weights [N-E47]. Each timed call multiplies every layer’s own checkpoint weight by its own input, so the weights stream from HBM as in the model; times are medians of 30 timings of 10 graph replays each, in one exclusive session, include any split- K reduction and the gap to the next kernel, and are compared only within these files. The three projections with 2,560 outputs, the GDN and attention output projections and the MLP down-projection, run as split- K kernels followed by a reduction kernel up to $M = 32$ (128 for the down-projection) and reach 49% (output projections) and 68% (down-projection) of the measured 3.79 TB/s read bandwidth [N-E33] at $M = 1$, against 80–87% for the large projections; disabling cuBLAS’s reduced-precision reduction changes neither their time nor any output bit. At $M = 1$ SGLang’s own Hopper GEMV is the fastest kernel for every projection: it takes 0.77 of cuBLAS’s time on the output projections (8.57 against 11.20 μ s per call for the GDN one), 0.91 on the down-projection, 0.95–0.97 on the other large projections and 0.51 on the 64-row GDN projection, which cuBLAS runs on a single block. At the pinned commit it is unreachable: --bf16-gemm-backend gemv loads it, but the unquantized linear layers call the dispatcher only for another backend (code reading; patch engine/sglang/patches/backbone/0001 lets them reach it). At $M = 2$ –16 a Triton skinny GEMM pays only with programmatic dependent launch, at 0.88–0.92 of cuBLAS’s time on the output projections and 0.96–0.99 on the GDN input and MLP projections, with no gain on the attention QKV projection; in this benchmark each call overlaps its successor, which in the model is a different kernel. From $M = 64$ cuBLAS, or a cuBLASLt algorithm within 2–5% of it, is best, except on the down-projection, where PyTorch’s cuBLASLt choice takes 0.92 of the time. None of these routes leaves the target’s arithmetic unchanged in general: the GEMV and the Triton kernel are bitwise equal to cuBLAS on 99.6 to 100% of output elements, and the cuBLASLt choice differs from it on the down-projection from $M = 16$, while all arms lie within the same distance of a float64 product. If every projection ran at its best isolated time, a

plain step’s backbone GEMMs would take 197 μ s less at $M = 1$, at most 5.7% of the 3,460 μ s step, and 92 to 120 μ s less at $M = 2$ –16 (derived from the microbenchmarks, not a served result).

SGLang’s packed GDN input projection, one GEMM over the concatenated $12,352 \times 2,560$ weight instead of a 12,288-row and a 64-row GEMM, gives every output element bitwise equal to the separate pair’s at each tested M , the powers of two from 1 to 1,024, with the real weights of all 24 GDN layers and random BF16 inputs, under cuBLAS 13.1 on this GPU [N-E47]. That is a statement about one isolated GEMM, not about served tokens. Against the stock pair, which runs the 64-row GEMM on a side stream, it is as fast up to $M = 16$, 3% slower at 32 and 5 to 13% faster from 64. The stock fused add-RMSNorm takes 1.47 μ s per launch at $M = 1$ and 2.0 to 2.2 μ s at $M = 4$ –32 inside a graph of 64 norms. Folding the norm and the SiLU into the next GEMM’s prologue reproduces the stock norm bit for bit up to $M = 32$ (one element in 10^5 is one BF16 step away at $M = 64$ and 128) and the stock SiLU bit for bit, but as built it makes each layer slower in MLP and GDN layer skeletons, by 12 to 90 μ s up to $M = 32$, with or without programmatic dependent launch, and by 82 to 543 μ s at $M = 64$ and 128, far more than the two launches of about 1.5 to 2 μ s that it removes. Two further variants, a single-pass norm prologue that scales the product and the SiLU fold alone, also lose: from $M = 2$ to 16 they take 103 to 257 μ s per layer against 32 to 52 μ s for the stock and Triton layers, and at $M = 1$ only the SiLU fold ties. In the same skeletons the Triton kernel with programmatic dependent launch saves 1.2 to 2.3 μ s per MLP layer and 1.4 to 1.8 μ s per GDN layer up to $M = 16$, except the GDN layer at $M = 4$, and loses at 32. The engine patches that route each projection by a table are committed and off by default. Served, each switch was compared with stock plain decoding on the 320 divergence prompts (256 greedy tokens, top-five log-probabilities, prefix cache on) [N-E47]. With every switch off, and with the packed GDN input projection alone, all 320 prompts return the same output tokens and the same top-five log-probability arrays as stock at concurrency 1; for the packed projection that covers prefill only, since decode at concurrency 1 stays below its 64-row cutoff. SGLang’s Hopper GEMV backend diverges at 3.75 per 1,000 compared tokens (164 ties, 8 one-step and 2 near events, none large), against the 3.42 floor at a cap of 16, and the routing table (the Hopper GEMV at $M = 1$, the Triton kernel with programmatic dependent launch at $M = 2$ –16, the packed projection from 64 rows), with pools pinned at a cap of 8, diverges at 3.40 at concurrency 1 and 3.82 at 32, five to six times the 0.63 floor at that cap. Both are exact up to rounding, and no run committed a token that was not its own top-1. In one session of paired serving on tuned plain decoding (order B A A B, the routing table against the same engine with every switch off), the routing table delivered 1.035 and 1.034 times the throughput at concurrency 1, a decode step 116 μ s shorter against the 197 μ s the microbenchmarks bound; 1.004 in both pairs at concurrency 8, where the decode step is 10 μ s shorter against 90 to 98 μ s predicted; 1.003 and 1.001 at 32, about the 0.11% spread of the two baseline runs, so no claim; and 1.010 in both pairs at 128, where only the packed projection acts and the step is 86 μ s shorter against 81 predicted. These are two pairs from one session. A trace of plain decoding under Nsight Systems (node-level graph trace, one 2-second window per point) accounts for part of the shortfall at concurrency 8 [N-E47]. Every route dispatches as tabled: 152 Hopper GEMV calls per decode step at $M = 1$ and 120 Triton calls at $M = 16$. But the GPU time of a decode step, the span of its graph replay, shrinks by only 102 μ s at $M = 1$ and 45 μ s at $M = 16$, 52% and 37% of the isolated predictions of 197 and 120 μ s, and one unprofiled window per point shortened the step by 111 and 15 μ s; why the step at concurrency 16 keeps only a third of the GPU saving is not traced. Per-kernel durations do not add up under programmatic dependent launch, because an early-launched norm or SiLU kernel waits inside itself, so only replay spans compare steps. Against MTP with FlashInfer attention (mtp-tuned, the arm the benchmark tunes for high concurrency), in one session of the same design, the routing table gains nothing. At concurrency 1 it delivers 0.993 and 0.989 of the throughput; about 0.2% of that loss is fewer tokens per verify cycle (3.273 against 3.279), and the rest is not traced. At 8 and 32 the two pairs straddle 1, while the baseline’s own two runs differ by 1.3 and 2.4%; at 128 both pairs are below 1 (0.996 and 0.984, one beyond the baseline’s 0.81% spread), so no claim. Against mtp-tuned-triton, the faster MTP arm at concurrency 1 to 32, one session of the same design at concurrency 1, 8 and 32 gains nothing material either: 1.0006 and 1.0007 of the throughput at concurrency 1, below the 0.15% spread of the benchmark’s three stock sessions of that arm, and pairs that straddle 1 at 8 and 32 (0.9997–1.0011 and 0.9873–1.0093) [N-E47]. The routing table does change greedy outputs under MTP. At concurrency 1 and 8 each arm reproduces its own streamed text across its two launches, but the two arms’ texts differ on 7 of 64 prompts at concurrency 1 and 15 of 64 at 8; at 32 two runs of the same arm already differ, so that point says nothing about the table. This compares streamed text, not token ids or log-probabilities, so the routing table’s exactness class under MTP is not established.

What did not work FP8 W8A8 weights, whose CUTLASS GEMM in the installed aarch64 sgl-kernel aborts at run time and whose Triton fallback showed no consistent gain in single runs; FP8 KV cache, which saves nothing at short contexts; BF16 recurrent state, which moves the same bytes as FP16 and is worse on quality in published measurements on another GDN model (Zhang et al., 2026b); folding the RMSNorm and the SiLU into the next GEMM’s prologue, 12–90 μ s per layer slower up to $M = 32$ and 82–543 μ s at 64 and 128 as built, with the single-pass and SiLU-only variants slower at every M from 2 to 16, in isolated layer skeletons [N-E47]; the int4 target with its int4 drafter, slower than BF16 from concurrency 2, where its W4A16 GEMMs lose to BF16 in isolation (from 32 rows) [N-E49]; and 2:4 sparsity, which the engine does not support at the pinned commit (its compressed-tensors loader raises an error for 2:4 schemes; code reading, python/sglang/srt/layers/quantization/compressed_tensors/compressed_tensors.py). An MTP server with BF16 state

Table 5. Levers by axis.

Axis	Lever	Class	Status
Progress per verify pass	MTP and DFlash drafting, deeper chains, trees where the GDN path allows, n-gram drafting	exact	Chosen in single runs on the tuning split: MTP with three steps, DFlash blocks 16, 8 and 4 by concurrency (Table 6); trees and n-gram drafting not searched; served comparison [N-E79]
Bytes per verify pass	Relaxed greedy acceptance within g logits	lossy	Not measured
	Certified int8 head (Section 3 of the paper)	exact	Head alone (best mode, conservative model): 0.62–0.69 of the stock chain’s time up to $M = 16$, slower from 128 [N-E10]; served: +1.2–4.6% at $c = 1$; –1.0–3.3% for DFlash above $c = 1$ and MTP at 64 [N-E73]
	Replayed verification of the recurrent state	exact up to rounding	MTP, three steps: 0.86× plain decoding at $c = 128$ on the confirmation split, against 0.68× with snapshot verification (one session) [N-E79]; outputs rounding-level against stock MTP [N-E80]; no detectable GSM8K loss [N-E81]
	Fold-every-commit verification for DFlash	exact	Bitwise with matched pools; one session: +6–12% at $c = 8$ –32, –3.2% at $c = 1$ [N-E61]; narrow value tiles in its verify remove the $c = 1$ loss (+2.0%, block 16) but cut the $c = 8$ gain to +1.8%, untimed above $c = 8$ [N-E62]
Bytes per draft step Concurrency and state	FP8 W8A8 weights, FP8 KV; int4 target with a matched drafter (Kim et al., 2026a)	lossy	FP8 weights: no consistent gain; FP8 KV: no gain (single runs); int4 pair: 0.715–0.870× throughput at $c = 2$ –256; at $c = 1$, 1.046× (faster), 1.015× per request (within the band); fails the probe [N-E49]
	Hot-vocabulary or low-precision draft head	exact	Not measured
	One state slot per request (lifts the 133-request cap)	exact	Capacity 256 served at $c = 64$ –256 (lossy study, three sessions) [N-E49]; no A/B against a capped arm; default limits not measured
Backbone GEMMs and norms	FP16 state; replayed decoding; rounding-preserving replay (P4)	lossy, exact up to rounding, exact	FP16: 1.16–1.17× the best non-lossy arm at $c = 64$ –256 (three sessions), probe passed, GSM8K 1.0–1.4 points lower: outside the band [N-E49]; replayed decoding: 1.08× at $c = 128$ on the confirmation split in one session [N-E79], outputs rounding-level [N-E80]; P4: kernel 1.22× at $B = 128$, derived about 1.08× per decode step [N-E43]; served decode rate 1.0042× at 128 running requests, rejected [N-E45]
	Kernel routing per projection and M (Hopper GEMV, Triton with programmatic dependent launch, cuBLASLt); packed GDN input projection; norm and SiLU folded into the GEMM	exact up to rounding (packed projection alone: identical outputs at $c = 1$)	Best isolated kernel up to $M = 16$: 0.77–0.92 of cuBLAS’s time on the output projections, 0.91–1.00 on the other large ones; no folding variant pays; served on tuned plain decoding, one session: 3.4% faster at $c = 1$, 0.4% at 8, 1.0% at 128 [N-E47]; no material gain on MTP with FlashInfer or Triton attention; with Triton its streamed text differs on 7 of 64 prompts at $c = 1$, exactness class not established [N-E47]
System overheads	Host gaps and synchronizations, fusion, attention backend per shape	exact; Triton target attention exact up to rounding	Measured shares in Table 3; Triton attention: MTP 1.18× at $c = 1$, draft only 1.08× (single runs [N-E46]); blocking planning reads removed with token-identical outputs: tuned MTP 1.075–1.096× per-request rate served at $c = 1$ –32 (interleaved, one session), held cycle 5.9–8.8% shorter [N-E48]; untuned DFlash (FlashInfer draft) 1.01–1.02× (one session) [N-E48]

at 512 concurrent requests also failed to capture its verify graph, on a FlashInfer workspace overflow (Section 4). Three proposals were rejected by their pre-registered tests: writing the recurrent state less often (P4, above), whose bit-exact replay decoded only 1.0042 times as fast as dense decoding at 128 running requests against the required 1.10; sharing computation across requests (P5, above); and a certified decoder tail (P1, Section 7.2). None of these single runs was repeated.

3.7 Interactions and the two stacks

Speedups from separate levers do not multiply, and quality losses need not add. Three interactions are predictable from the mechanism. A lossy target lowers acceptance, because the drafter was trained against the BF16 target; the published int4 recipe fine-tunes its drafter against the int4 target for exactly this reason (Kim et al., 2026a), and here the int4 pair’s accepted length is 5.0 tokens per block-16 pass against 5.7 for the BF16 pair [N-E49], with target and drafter both quantized, so the two effects are not separated. A cheaper verifier makes drafting relatively more expensive, which raises the value of a cheap draft head. And every lever that shrinks per-request bytes also raises capacity, so its measured gain depends on whether the server was allowed to use that capacity. Each pair is therefore measured in the baseline, A , B , $A+B$ pattern rather than by multiplying isolated speedups.

The *exact stack* admits only levers that keep the target’s decisions up to rounding-level near ties: capacity and scheduling changes, drafting levers, the levers whose greedy outputs are exact up to rounding (replayed verification among them), and the certified head. The *lossy stack* admits everything, under a quality budget declared before measuring: at most 1.0 point of GSM8K accuracy below the reference on the full test split with thinking on (paired, with an exact McNemar test), teacher-forced top-1 agreement of at least 98% and mean top-20 KL divergence of at most 0.01 nats on a fixed probe set of 48 prompts and 256 tokens, each reported next to the reference’s own run-to-run noise. The combination’s quality is measured on the combined configuration, never summed from its parts. Proposition 7.4 explains why the budget is empirical: a per-token logit error small enough to certify a long output as close in total variation is far below what any low-precision lever delivers. The interaction matrix (speed at concurrency 1 and at peak, acceptance and quality for each measured pair) and both frontiers, from one server launch per arm, were not measured; the two lossy levers were timed alone (Section 3.6).

3.8 A sensitivity workload for output length

Sensitivity to output length With a closed-loop client and equal output lengths, plain requests start and finish in synchronized waves, while speculative arms advance requests at rates that vary with acceptance, so their completions spread out into more, smaller prefill passes and a drain at the end of a point. A sensitivity workload, declared before any arm was timed on it, removes the equal lengths and nothing else: each confirmation prompt is given its own greedy completion length under plain decoding with natural stopping, capped at 2,048 tokens, so that every arm generates the same number of tokens per request. It runs at concurrency 32 and 128 with $\max(64, 8c)$ measured requests, the ordered prefix of the confirmation split (256 and 1,024 prompts). The arms follow a rule fixed before any sensitivity run: for each speculative family and concurrency, the arm with the highest mean y over the three confirmation sessions, invalid points excluded and only arms with a valid point in all three eligible, together with the family’s runner-up if it is within 2% of that mean. Each runs with its matched plain baseline, in three sessions. The fixed-length panel stays the primary result; where the sensitivity workload changes a ranking, both numbers are reported. It was declared but not run.

4 The serving protocol

Appendix E.3 of the paper gives the stock serving frontier and what it measures. This section holds the protocol behind it in full: the workload and its splits, the metrics, the launch checks and capacity, the configuration search, the output comparisons against stock references, the GSM8K quality check, the frontiers and the measurement hazards. Every arm in it is stock or exact up to rounding; none uses the certified head.

Every served speedup is to be measured against a server, not a kernel; the only served runs so far are of stock and exact-up-to-rounding arms. This section defines the measurement: what runs, on which prompts, how the latency-throughput frontier is computed, how the baselines are made strong before anything is compared with them, and which quality check a change must pass.

Setup and workload All runs use the machine and engine of Section 4.1 of the paper, serving the BF16 target at its pinned revision with FlashInfer attention (Ye et al., 2025); the vision encoder, which text requests never reach, uses Triton attention. The client is AIPerf (NVIDIA, 2026a). Speculative acceptance depends on what the model is writing, so random-token prompts would measure the wrong thing. The benchmark uses a frozen set of 1,857 public prompts in three domains: chat (the first turns of the 80 MT-Bench questions (Zheng et al., 2023) and English root prompts from OASST1 (Köpf et al., 2023)), code (the 164 HumanEval problems (Chen et al., 2021) and MBPP test tasks with their assertions (Austin et al.,

2021)) and mathematics (GSM8K training questions with a request to box the final answer (Cobbe et al., 2021); the GSM8K test split is reserved for the quality check, so no workload prompt is a quality problem). Each source is pinned to a Hugging Face revision and the builder reproduces the files byte for byte. Each domain is cut into disjoint warm-up (43 prompts), tuning (192) and confirmation (384) sets, and every split interleaves the domains so that any prefix of it is balanced, which matters because low-concurrency points use only a prefix. Configurations are chosen on the tuning split and reported on the confirmation split. Templated prompts are short: the confirmation split has a mean of 87.9 prompt tokens (median 78, 99th percentile 283), with chat shorter (mean 47) than code (130) and mathematics (86) [N-E74].

Requests use the model’s chat template with thinking enabled (the template’s default) and no reasoning parser, so the thinking text is streamed and counted as output. Decoding is greedy. The throughput panel fixes the output at 512 tokens with `ignore_eos`, and every point checks that each response has exactly that length. A fixed length is a faithful sample of the model’s behaviour only while the model has not chosen to stop, so a natural-stopping run on 576 tuning prompts measured where greedy outputs end. Only 3.8% stopped before 512 tokens and 0.7% had entered a detected repetition loop by then (a 256-token window with fewer than 30% distinct 8-grams), so almost all of the fixed-length panel is text the model would have produced anyway. The same run shows why greedy decoding is not used for the quality check: 142 of the 576 outputs (25%; 36% in mathematics, 32% in chat, 6% in code) ran to the 16,384-token limit. Qwen’s model card recommends sampling parameters for thinking mode and a presence penalty to reduce endless repetitions (Qwen, 2026). The run used an earlier version of the workload, whose mathematics prompts came from the GSM8K test split, and other processes used up to 24 CPU cores during it, so it is a length measurement, not a timing [N-E75].

Metrics For request i at a sweep point let n_i be the number of generated tokens the server reports in the `completion_tokens` field of its usage record, s_i the time the client sends it, f_i the arrival of its first non-empty streamed chunk and e_i the interval from s_i to its last non-empty chunk. Over the N measured requests the frontier plots

$$x = \frac{1}{N} \sum_{i=1}^N \frac{n_i}{e_i}, \quad y = \frac{\sum_i n_i}{\max_i (s_i + e_i) - \min_i s_i}, \quad (4)$$

so x is the mean per-request output rate *including* the time to first token, and y is output tokens per second on the one GPU. Two companions are reported and never substituted for them. The decode rate $x_{\text{dec}} = \frac{1}{N} \sum_i (n_i - c_i) / (e_i - (f_i - s_i))$, where c_i is the exact token count of the first chunk from per-chunk usage (a speculative step streams several tokens per chunk), excludes queueing and prefill; one token per chunk is never assumed. The steady-state throughput counts only the tokens streamed between the first and the last send, when the client holds its full concurrency, and so excludes the drain at the end of a point. Each point also records time-to-first-token, inter-token and end-to-end latency percentiles, failures, the per-request speculative statistics the server attaches to each stream (verify steps, accepted drafts, and accepted length as output tokens per verify step with the bonus token included), and deltas of the server’s counters, among them the fraction of decode passes that replayed a CUDA graph. The scheduler’s logged generation rate while the batch is nearly full is recorded as a diagnostic of what the GPU sustains; clients do not see it, and it is never a frontier metric.

Protocol and baselines A server arm is a model revision plus a declarative set of launch flags. The harness starts it once, verifies it, and sweeps client concurrency against it in closed loop; the confirmation sweep covers client concurrencies 1, 2, 4, 8, 16, 32, 48, 64, 96 and 128, each arm over the concurrencies where it competes (`bench/campaigns/confirm.sh`). Before trusting a launch it checks that decode CUDA graphs were captured for every batch size up to the server’s capacity (the target decode graph, or the target-verify, draft-decode and draft-extend graphs under speculation), that prefill graphs were captured, that the overlap scheduler is running (the scheduler reports it, no fallback is logged, and a stack sample of the scheduler process shows its overlap loop), and that the recurrent-state cache did not reduce the capacity.

Capacity needs care on this model. The attention cache is not the constraint: 32 KiB per token, and the default memory split leaves room for about a million tokens. The recurrent state is. With the prefix cache and the overlap scheduler on, SGLang reserves five FP32 state slots per running request (three for prefix-cache retention and two ping-pong buffers), while decode touches one; the 667 slots the default split gives the state pool therefore cap the server at 133 running requests (code reading, confirmed against the startup log) [N-E31]. Speculative verification adds one intermediate state per draft token, and SGLang resets speculative servers to 48 running requests, and logs it, unless the capacity is given; the capacity each arm would reach at default settings has not been measured. Feasibility probes show what launches: plain decoding serves 256 concurrent requests with the prefix cache and 1,024 without it, MTP speculation serves 256 without the prefix cache, and an MTP server with BF16 state at 512 fails at verify-graph capture on a FlashInfer workspace overflow. These probes establish what runs, not performance [N-E76]. Every arm is therefore launched at a capacity of 128 running requests, the top of the client sweep (64 for one DFlash arm, below), with the state cache sized for it and the remaining static memory given to the attention cache and the verify states.

Every arm streams every fourth token (`stream-interval = 4` in `bench/arms.toml`). The server still sends the first token at once and the last at finish, so the times that Equation (4) uses do not depend on the interval, and the choice takes load

Table 6. Configuration search on the tuning split: output throughput y (Equation (4)) from one launch per configuration with $\max(32, 4c)$ requests per point, and the accepted length at concurrency 32. FlashInfer attention unless Triton is named; MTP chains draft one token per step; RV: replayed GDN verification. “excl.”: excluded because other processes used more than two CPU cores on average, by a sampler that then overcounted, so these may be false positives. [†]Limited by the attention cache, which without a cap had shrunk to 54K tokens as the state buffers grew: at most 123 requests ran, with retractions. [‡]Capacity 64; the last throughput column is concurrency 64. Selection evidence, not results [N-E78].

Arm	Configuration	Prefix cache	y (tokens/s) at concurrency c				Accepted length
			1	8	32	128	
Plain	FlashInfer	on	282	1,982	6,087	13,421	–
	FlashInfer	off	282	2,004	excl.	13,844	–
	Triton	off	286	2,018	6,241	13,846	–
	FlashInfer, replayed decoding	off	244	1,807	5,909	14,981	–
MTP	2 steps	on	385	2,288	5,455	9,153	2.67
	3 steps	on	461	2,482	5,839	9,593	3.28
	4 steps	on	465	2,608	5,761	9,276	3.72
	5 steps	on	468	2,575	5,684	8,719 [†]	4.11
	3 steps	off	466	2,534	6,127	10,106	3.27
	5 steps	off	481	2,699	6,087	9,456	4.14
	7 steps	off	465	2,514	5,690	8,493	4.62
	3 steps, RV	on	446	2,646	6,469	11,941	3.28
	5 steps, RV	on	462	2,666	6,502	11,186	4.11
	3 steps, RV	off	456	2,667	6,925	13,011	3.29
	3 steps, RV, Triton	off	535	3,093	7,103	11,353	3.27
	4 steps, RV, Triton	off	552	3,104	6,958	10,804	3.73
	Adaptive depth, RV	off	474	2,603	5,968	10,010	1.97
DFlash	Block 4	off	498	2,682	6,380	11,477	3.28
	Block 8	off	686	3,443	6,727	10,226	4.75
	Block 8, FA4 draft attention	off	769	excl.	6,983	10,594	4.75
	Block 8, Triton	off	807	3,613	6,307	8,957	4.75
	Block 16 [‡]	off	excl.	3,398	5,386	6,737	5.71
	Block 16, Triton [‡]	off	851	3,639	5,199	6,397	5.72

off the single-threaded streaming path (below). Each point sends $\max(64, 8c)$ measured requests, the first prompts of the confirmation split in order, after one wave of warm-up requests at the same concurrency, and the prefix cache is flushed before every point so that no measured prompt is served from cache. The confirmation sweep runs in three sessions. Each session launches every tuned arm afresh in the same exclusive hold as its matched plain baseline (the FlashInfer plain arm for FlashInfer arms, the Triton one for Triton arms), in reversed order in alternate sessions, so that drift does not align with an arm. Within a session each arm runs once, over its concurrencies in ascending order; the harness alternates the order of concurrencies only between repeats within one launch, which the confirmation sweep does not use. Speedups are ratios within a session; a pair with an invalid point on either side is dropped and counted, never replaced by another run, and points at which the attention cache limited the batch are flagged from the scheduler log. Timed runs hold the GPU exclusively, and because the scheduler, tokenizer manager and client are single-threaded CPU loops, each point also waits until other processes use fewer than two cores and records the foreign load it saw.

The speedup denominator is the strongest existing configuration, not a default. Arm A is ordinary autoregressive decoding. Arm B is native MTP speculation, whose draft steps reuse the target’s tied head. Arm C is the public DFlash drafter (Section 4.1 of the paper). The search ran on the tuning split [N-E78]: 23 configurations, each launched once and measured at client concurrency 1, 8, 32 and 128 (64 instead of 128 for the two block-16 configurations) with $\max(32, 4c)$ requests per point and 512 output tokens. It covered MTP chain depth (two to five steps with the prefix cache, three, five and seven without it), the prefix cache, SGLang’s replayed GDN verification and replayed decoding, its batch-adaptive depth, Triton against FlashInfer attention, DFlash blocks of 4, 8 and 16 tokens and FA4 attention for the DFlash drafter. A one-step chain is not in the search, because it failed a launch check that required a draft-decode graph such a chain does not capture; replayed verification was refused at launch for DFlash and with FlashInfer’s GDN decode kernel. Draft trees (top- k 2 and 4, which run on this hybrid model, with accepted lengths of 3.45 and 3.56 at concurrency 1 in the feasibility probes [N-E76]), the verify attention mode, the prefill chunk size and graph backend were not searched. With one launch per configuration, Table 6 is selection evidence, not a result.

Three regularities decide the arms. First, speculation pays at low concurrency and loses at high. MTP with three steps delivers 1.64 times plain decoding’s y at concurrency 1 (461 against 282 tokens/s); with snapshot verification it falls behind

from concurrency 32 (5,839 against 6,087), with replayed verification and no prefix cache it still leads there (6,925) and trails at 128 (13,011 against 13,844), and at concurrency 128 every speculative configuration trails plain decoding. Second, replayed verification, which removes the per-draft-token state snapshots, recovers 24% at concurrency 128 for three steps (11,941 against 9,593 tokens/s) at a 3% cost at concurrency 1, and depth 3 beat depths 4 and 5 at concurrency 128 in every pair the attention cache did not limit, while trailing them by at most 4% at concurrency 1 and 7% at 8; in the low-concurrency arm’s own configuration (replayed verification, Triton), depth 4 led depth 3 by 3.2% at concurrency 1 and 0.4% at 8 and trailed it by 2% at 32, so one depth serves both MTP arms. Third, Triton attention helps every speculative family at low concurrency and hurts at high concurrency, and plain decoding is indifferent to it. DFlash at block 16 under Triton has the highest throughput of any configuration at concurrency 1 and 8, and at concurrency 1 a per-request rate 3.35 times that of plain decoding under Triton (957 against 286 tokens/s in x); among DFlash configurations block 4 leads at concurrency 128 (11,477 tokens/s, 8% above block 8 with FA4 draft attention).

The chosen arms (`bench/arms.toml`) all run without the prefix cache, which this workload does not reuse and whose state slots deeper drafts and replayed verification need. Arm A is plain decoding under FlashInfer, with a Triton twin as the matched baseline for Triton arms. Arm B is MTP with three steps and replayed verification, under FlashInfer for high concurrency and under Triton for low. Arm C is DFlash at block 8 with FA4 draft attention for every concurrency, at block 16 under Triton for low concurrency, capped at 64 running requests because its 16 verify states per request take about 0.8 GB, and at block 4 for high concurrency. Two of these choices change the target’s arithmetic: replayed verification computes its verify outputs with a chunked transform on TF32 tensor cores (code reading at the pinned commit), and Triton attention for the target replaces the target’s attention kernel. Both are exact up to rounding by the comparison below, as is replayed decoding, so every confirmed arm is stock or exact up to rounding. MTP with SGLang’s snapshot verification under FlashInfer is kept as a stock reference, and plain decoding with replayed decoding, the fastest configuration at concurrency 128 on the tuning split and the slowest at 1, as a variant.

Output equality Each arm that changes the target’s arithmetic was compared with a matched stock reference on the 320 divergence prompts of Appendix D.4 of the paper, decoded greedily for 256 tokens with the top five log-probabilities at concurrency 1 and without the prefix cache, like every arm: plain levers against stock plain decoding, buffered MTP against stock MTP with three steps, and Triton DFlash at block 16 against stock DFlash at block 16, both DFlash runs at a capacity of 4 [N-E80]. The class rule was fixed after the first comparisons had been seen. It rests on the divergence classes of Appendix D.4 of the paper, which predate them: an arm is exact up to rounding if every prompt’s first divergence is at an exact tie, within one BF16 step, or a near tie with both margins within two steps, and no output differs in length; it is lossy otherwise. Every arm passes, with no event above a near tie (Table 7). The table also gives each rate’s ratio to the batch-shape floor of 3.42 per 1,000 (plain decoding at concurrency 1 against 32 on one server with the prefix cache on and at most 16 running requests). Every ratio’s 95% interval includes 1, and the ratios inform the class rather than decide it. The ratios depend on that floor’s cap: with at most 8 running requests and fixed pools the same pair diverges at 0.63 per 1,000 (Appendix D.4 of the paper), while the class rule does not use the floor’s rate. The classes above were measured with servers capped at 16 running requests (4 for DFlash), and none of these arms has been compared at a cap of 8; only stock plain decoding and stock MTP were rerun there, and they too diverge only at rounding-level events, an exact tie, one BF16 step or two steps at most [N-E27]. Replayed plain decoding was first classed lossy against the plain run of Appendix D.4 of the paper, which has the prefix cache on: one prompt’s first divergence was large. At that position the prefix-cache-on run’s own token rests on a logit gap of 0.0625, which stock plain decoding without the prefix cache resolves the other way, and with the prefix cache on a request’s log-probabilities can depend on earlier requests (Appendix D.4 of the paper). Against the reference without the prefix cache, replayed plain decoding first differs from it in that prompt at a tie further on, and its rate is the floor’s. The two stock plain references, with and without the prefix cache, differ in 96 of 320 prompts at 1.69 per 1,000, all rounding-level.

Speculation against plain decoding Stock speculation against the same plain reference diverges at 1.15 times the floor’s rate both for MTP with three steps and for DFlash at block 16 (Table 7). Against the prefix-cache-on run the ratios were 1.29 and 1.28, with intervals above 1, so that apparent excess came at least partly from the reference’s prefix-cache setting, not from speculation: at the pinned commit that setting changes the recurrent prefill of prompts longer than one 64-token chunk, from the first output token, in all 6 such prompts of a 40-prompt check of plain decoding at batch 1 and in 192 of the 193 among the 320 for MTP with three steps at batch 1 with fixed pools (Appendix D.4 of the paper). Turning the overlap scheduler off, which at the pinned commit also changes the prefix cache’s recurrent-state strategy, changes the same first-token log-probabilities for MTP. For plain decoding on the same 320 prompts either switch changes the top-five log-probabilities of the first output token, which come from the prefill alone, in 192 of the 193 prompts longer than 64 tokens and in none of the 127 shorter ones, as for MTP; in tokens, the prefix cache off changes 96 plain outputs and the overlap scheduler off 166 [N-E27]. Against this floor, then, an excess of speculation is not detected, though one of up to about 40% is not excluded. Three limits apply to this check: the floor is a prefix-cache-on pair on one server, and a floor without the prefix cache was not run; the servers’ pools were sized from free memory, not pinned, although for MTP with

Table 7. Greedy outputs against matched stock references on the 320 divergence prompts (256 tokens, top-five log-probabilities, concurrency 1, no prefix cache; servers capped at 16 running requests, 4 for DFlash): divergences per 1,000 compared tokens and the ratio to the batch-shape floor of 3.42 per 1,000, with its 95% interval where computed. Every arm that changes the target’s arithmetic is exact up to rounding by the class rule. Measured [N-E80].

Arm	Reference	Per 1,000	Ratio to the floor
<i>Arms that change the target’s arithmetic</i>			
MTP, three steps, buffered verification	stock MTP, three steps	3.79	1.11
the same with Triton attention	stock MTP, three steps	4.05	1.19
DFlash, block 16, Triton attention	stock DFlash, block 16	3.82	1.12
Plain decoding, Triton attention	stock plain decoding	3.89	1.14
Plain decoding, replayed	stock plain decoding	3.40	0.99
<i>Stock speculation</i>			
MTP, three steps	stock plain decoding	3.95	1.15 (0.93–1.43)
DFlash, block 16	stock plain decoding	3.92	1.15 (0.93–1.42)

three steps at concurrency 1 without the prefix cache the run of this comparison with pools sized at start-up and one with fixed pools at a cap of 8 agree bitwise on all 320 prompts [N-E27]; and the DFlash runs used a capacity of 4 against plain decoding’s 16. Whether speculation shows an excess depends on the floor, however. With fixed pools and at most 8 running requests, both runs with the prefix cache on, plain decoding at concurrency 1 against 32 diverges at 0.63 per 1,000 while MTP with three steps against plain decoding at concurrency 1 diverges at 3.68, about six times that floor (derived), and every MTP configuration diverges from plain decoding at 3.5 to 4.0 per 1,000, about as often as it diverges from itself between concurrency 1 and 32 (3.1 to 3.5) [N-E27]. The speculative rate is about the same in both regimes, and what moves is plain decoding’s own batch-shape divergence.

Quality check Output length and throughput say nothing about whether a change preserved the model. The quality check is the full GSM8K test split (1,319 problems) scored with the GSM8K grader of the `sg1-eval` package, version 0.1.2 (a zero-shot prompt, the last boxed answer, symbolic equality) with thinking on, natural stopping at 16,384 tokens, and Qwen’s recommended thinking-mode sampling for precise tasks (temperature 0.6, top- p 0.95, top- k 20) under a fixed request seed. Two runs are compared problem by problem: the accuracy difference, an exact McNemar test on the discordant problems, and the share of identical final answers. Sampled accuracy tests equality in distribution for a lossless change and prices a lossy one; it cannot show that greedy outputs are unchanged, which the output comparisons of Appendix D.4 of the paper address. No arm differs detectably from plain decoding [N-E81]. Two launches of plain decoding score 89.99% and 90.30%, buffered MTP 89.54%, MTP with snapshot verification 88.93%, DFlash at block 8 89.69% and replayed plain decoding 91.05% (run while it was still classed lossy). Every paired difference against either plain run lies between -1.36 and $+1.06$ points, with exact McNemar $p \geq 0.18$. A non-significant test excludes nothing by itself. The paired 95% intervals (Wald intervals on the discordant problems, derived from the committed counts) have lower bounds between -0.7 and -3.3 points, the lowest for MTP with snapshot verification against the second plain run (-1.36 points, interval -3.27 to $+0.54$), so the data exclude losses larger than about 3.3 points for every arm and cannot exclude smaller ones. Two sampled runs of one arm agree on the final answer for only 77% of problems, so the fixed request seed does not make these runs reproducible, and 17 to 19% of outputs in every arm reach the 16,384-token limit.

Frontiers The baseline frontier is the upper envelope over arms A, B and C, since no arm need be best at every concurrency: speculation saves weight traffic, which matters most at low concurrency, and adds per-request state traffic, which matters most at high concurrency (Table 19 of the paper). Every arm carries an exactness class in `bench/arms.toml`: *stock* when it sets only flags known to leave the target’s arithmetic as in plain decoding with FlashInfer target attention and stock state handling, *exact up to rounding* by the comparison above, *pending* until compared, and *lossy* otherwise. All nine confirmed arms are stock or exact up to rounding, so the envelope over exact arms equals the envelope over all arms. The frontier over three confirmation sessions, with paired ratios against the matched plain baselines and the run-to-run variance, is committed [N-E79]; only points with three valid sessions rank, so replayed plain decoding, with one session, does not rank yet; its two further sessions were not run. The certified head’s before-and-after frontiers, with output-equality checks at the same batch shape, are in the paper’s Appendix E.5 [N-E73].

Measurement hazards Two effects outside the GPU shaped this protocol. At high concurrency plain decoding can be limited by the streaming path rather than the GPU; speculative arms stream several tokens per chunk and are less exposed, so a frontend bottleneck would inflate their measured advantage. A diagnosis with one plain server at client concurrency 256 on a quiet host measured the effect [N-E77]. With one-token chunks the tokenizer-manager process (HTTP and the OpenAI

serving layer) ran at a full core and the client received about 85% of the server’s full-batch decode rate; streaming every four tokens brought that process to 0.35 cores and gave the same throughput as a non-streaming client, about 92% of the full-batch rate, the rest being the prefill of each new wave and the drain, which y includes by definition. More tokenizer and detokenizer workers made it worse, and more client workers changed nothing. Every row of that diagnosis is a single point; concurrency above 256 on a quiet host was not measured. The same diagnosis attributes the low throughput of the earlier feasibility probes at concurrency 256 to 1,024 to CPU contention on the host, not to a frontend limit. That is the second effect: exclusive use of the GPU does not keep CPU-heavy jobs off the host, which is why every point records foreign CPU load and waits for a quiet machine.

5 Drafting and repair hypotheses

How much a verify pass is worth depends on the drafter in front of it: a cycle commits as many tokens as the target accepts plus one, and its cost includes the draft. The native MTP layer of Qwen3.5-4B drafts sequentially, paying one small layer plus the full 1.27 GB head per drafted token, and its acceptance decays with depth. A block drafter proposes a whole block in one forward pass. Every drafter that projects through a head pays for it at every draft step: Medusa and the EAGLE family (Cai et al., 2024; Li et al., 2024a;b; 2025), native multi-token prediction (Gloeckle et al., 2024; DeepSeek-AI, 2024) and DFlash (Chen et al., 2026a). This section describes the public block drafter for this target, how SGLang runs it on a hybrid model and what is measured about it, and then states the proposals on drafting and verification as hypotheses with their kill tests: a selector trained for rate (P6), exact repair of long draft windows (P2, P3), reuse of a cached window after a rejection (P9), and the treatment of the recurrent state in verification (P7b, P10, P11). P6 passes its first screen; P3 is refuted, because its anchored evaluator fails at the operator level; P2’s first test depends on the verify kernel, and with SGLang’s Triton kernel (exactness not classified) P2 turns on drafting long, almost fully accepted blocks; P9’s oracle bound does not reject reuse with top-8 and top-16 candidate sets at concurrency 1; the remaining tests were not run.

5.1 The public block drafter

DFlash has a public drafter for exactly this target, `z-lab/Qwen3.5-4B-DFlash` at revision `9a1996cc`, under Apache-2.0 (Chen et al., 2026a; `z-lab`, 2026); `modal-labs/Qwen3.5-4B-DFlash` carries byte-identical weights. Read from the checkpoint rather than the card, it is a six-layer Qwen3-style transformer of width 2,560 (32 query and 8 key-value heads of dimension 128, MLP width 9,216) with no embedding or head of its own: it embeds with the target’s table and projects through the target’s tied head. It conditions on the target’s hidden states after layers 1, 5, 9, 13, 17, 21, 25 and 29, concatenated per token and projected by one linear map and an RMS norm to a single context vector. Five layers use sliding-window attention (window 4,096) and the last uses full attention. It has 634 million parameters, 1.27 GB in BF16, almost exactly the size of the target’s head (derived).

A cycle in SGLang’s DFlash worker (SGLang contributors, 2026d;c) works as follows (code reading at the pinned commit). The block is the last committed token followed by fifteen copies of a mask token (id 248,077). Each draft layer’s keys and values for the committed context come from the projected target features, which the worker writes into the drafter’s own cache as tokens are committed. One forward pass over the sixteen positions yields sixteen hidden states; positions 1–15 go through the target head, and their argmaxes are the drafted tokens. The target verifies the block in one pass, the worker accepts the longest drafted prefix that matches the target’s argmax, commits it plus the target’s token at the first mismatch, and appends the committed tokens’ features to the drafter’s cache. The sliding layers are causal inside the block and the full layer is not; SGLang infers this from the layer types because the configuration sets `no_is_causal`, which matches the attention pattern of the DFlash training recipe in SGLang’s SpecForge repository (`sgl-project/SpecForge`).

On the hybrid target the verify pass has a cost the drafter’s parameters do not show. SGLang verifies a chain on the GDN layers by keeping the recurrent state after every block position, so that the state after the accepted prefix can be committed without recomputation. Each state is 50.3 MB per request, so a 16-token block writes about 805 MB per request per cycle, and each running request reserves space for sixteen of them (derived). That bounds block-16 capacity on this machine, and it is a property of the verifier, not of the drafter.

The card’s launch line targets a B200 with TRT-LLM target attention, FA4 draft attention and piecewise-compiled prefill graphs. TRT-LLM attention is Blackwell-only, and the piecewise prefill graph fails to capture for this target at the pinned commit. FA4 draft attention does run on this host, and one tuned serving configuration uses it (Section 4). The acceptance runs below serve the drafter with FlashInfer for target and draft attention, the FlashInfer GDN kernels, the default prefill graph and the card’s overlap planning stream, with decode, verify and draft CUDA graphs and the overlap scheduler on; their launch records give the exact command, environment and resolved server settings [N-E54].

5.2 What a cycle costs

Counting weight bytes gives the shape of the trade before any timing. At batch one a DFlash cycle reads the drafter once (1.27 GB), the target head once for the fifteen drafted positions (1.27 GB), and the target once to verify (8.41 GB): drafting costs about 30% of the verify pass’s weight traffic and proposes fifteen tokens. An MTP chain of three steps reads the MTP layer and the full head three times (about 4.5 GB, 54% of the verify pass) and proposes three (derived). Activation traffic, the per-position GDN states, attention over the context and host time are not in these counts; the measured MTP cycle of Appendix E.2 of the paper shows how much the host adds. Appendix E.2 of the paper splits the tuned DFlash cycles by phase at batch 1 to 64 [N-E37]: with 16-token blocks at batch 1 the target’s verify forward takes 66% of the traced cycle and the two head projections 9.5%, and at batch 1 to 64 the GPU is busy 96.6–98.8% of the traced cycle, so this arm has no host gap; the arm with 8-token blocks keeps up to about 0.66 ms of idle per untraced cycle at batch 1 (derived, an overestimate).

5.3 Acceptance by block position

Let h_j be the number of verify cycles in which exactly j drafted tokens were accepted, $N = \sum_j h_j$, and $S(k) = \sum_{j \geq k} h_j / N$ the fraction of cycles that accept at least k . The conditional acceptance of block position k is $\alpha_k = S(k) / S(k-1)$, and a cycle commits $\tau = 1 + \sum_{k \geq 1} S(k)$ tokens on average over cycles. SGLang reports h per request, so α_k and τ are measured rather than modelled. The model card instead averages each request’s tokens per cycle over requests; we call that quantity $\bar{\tau}_{\text{req}}$. It weights short responses more than τ does, and for these runs it is the larger of the two. The card reports mean accepted lengths of 5.9–7.7 at block 16 on a B200 for its five benchmarks, against 3.3–3.5 for MTP with three steps (z-lab, 2026). The same card reports, on one B200 with greedy decoding, 3.40–4.60 $\times$ over autoregressive decoding at concurrency 1 with blocks of 16 and 2.15–2.61 $\times$ at concurrency 32 with blocks of 8, against at most 2.31 $\times$ and 1.80 $\times$ for the MTP layer. These are the authors’ numbers on other hardware.

The measurement uses a panel of 32 MATH-500 problems and the first 16 chat, 16 code and 16 GSM8K prompts of the benchmark’s earlier confirmation split, decoded greedily with thinking on, stopping naturally at up to 2,048 new tokens, at concurrency 1, with blocks of 16 and 8 [N-E55]. The runs used an engine trace hook that synchronizes every cycle on a shared GPU, so acceptance and token counts are valid and timings are not; the panel contains benchmark test problems and measures acceptance only, never quality.

Table 8. Committed tokens per verify cycle for the public DFlash drafter on GH200 (greedy, thinking on, concurrency 1), pooled over cycles (τ) and averaged over requests as the model card does ($\bar{\tau}_{\text{req}}$). Measured in correctness runs [N-E55]; the survival form $1 + \sum_k S(k)$ agrees with the pooled τ to within 0.013 in every column.

Block	Average	All	Chat	Code	GSM8K	MATH-500
16	τ , over cycles	6.18	3.98	7.63	6.72	7.55
16	$\bar{\tau}_{\text{req}}$, over requests (the card’s)	6.98	4.26	7.87	7.16	7.81
8	τ , over cycles	5.04	3.64	5.75	5.31	5.77
8	$\bar{\tau}_{\text{req}}$, over requests	5.34	3.80	5.76	5.45	5.84
Requests (verify cycles at block 16)		80 (21,274)	16	16	16	32

A block of 16 commits 6.2 tokens per verify cycle and a block of 8 commits 5.0 (Table 8). Acceptance depends strongly on the domain: chat commits about four tokens per cycle, GSM8K 6.7, and code and MATH-500 about 7.6. The per-request MATH-500 average, 7.8, sits near the 7.48 the card reports for that benchmark in the same convention (z-lab, 2026), but that is a cross-hardware reference, not a reproduction: the card used a B200 and a 4,096-token cap and does not state how many problems it used, where this panel uses a GH200, a 2,048-token cap and 32 sampled problems. A gate with the card’s own settings on its MT-Bench prompts does reproduce its number within sampling noise [N-E57]: over the 80 MT-Bench first turns at block 16, greedy with thinking on, up to 4,096 new tokens and concurrency 1, the mean accepted length per request is 5.73 (bootstrap 95% interval 5.36–6.13; 5.25 pooled over cycles) against the card’s 5.93 on a B200, with 26 of the 80 responses reaching the token limit. The panel’s low chat acceptance comes from its OASST1 prompts (4.06 per request over 14 prompts) rather than its two MT-Bench prompts (5.70).

The per-position rates of Figure 2 show why a longer block still commits more tokens per cycle; whether it pays depends on the cost of a cycle, and serving decides it: 16-token blocks lead the confirmed frontier at concurrency 1 to 4 and 8-token blocks at 8 to 32 [N-E79]. Over all domains the conditional acceptance dips at the second position (0.80) and then rises toward 0.9, so the share of cycles that reach position 15 is still 13%. The rise is a selection effect: cycles that get that far are in predictable stretches of text, which is not evidence that the drafter is better deep in the block. Chat is lower throughout (0.69 at positions 2 and 3), and the dip at position 2 appears in every domain. Two further observations bound what training could change. Acceptance, pooled over tokens, is lower in the answer after the thinking text than in the thinking itself for chat (3.3 against 4.2 tokens per cycle) and higher for code and mathematics, so no single segment dominates. And of the

18,596 cycles with a rejection, the first rejected draft equals the target’s token at the previous position in 6% (1,082), a repetition-type failure, and is some other token in 94% [N-E56]. On a second panel (the first 16 chat, code and GSM8K-train prompts of the serving workload’s confirmation split, and the same 32 MATH-500 problems), the drafter commits 6.81 tokens per cycle per request (6.11 pooled) at block 16, and the native MTP layer with three draft steps 3.38 (3.34) [N-E58]. MTP accepts more at the first three positions (0.90, 0.86 and 0.86, against 0.86, 0.79 and 0.81), and the block drafter keeps drafting to position 15.

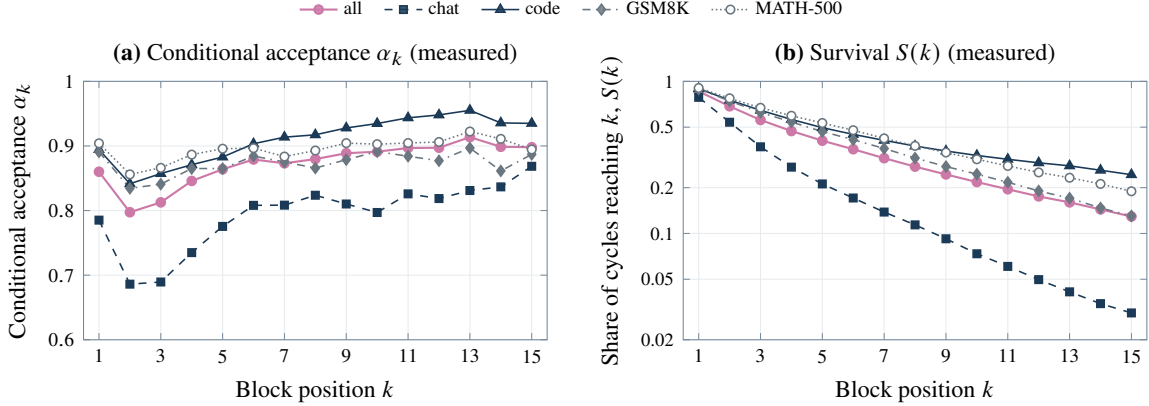


Figure 2. The public DFlash drafter by block position, block size 16, greedy with thinking on, per domain of the panel. (a) Conditional acceptance α_k ; (b) the share of verify cycles that accept at least k drafted tokens, on a logarithmic axis. Measured in correctness runs [N-E55]; the late rise in (a) is a selection effect of the few cycles that get that far.

5.4 Output equality and limits

Greedy speculative verification is exact in real arithmetic, but the verify pass computes the target’s logits at a different batch shape from plain decoding, so the stock noise floor of Appendix D.4 of the paper applies. On the second panel at concurrency 1, 77 of 80 outputs of the block drafter diverge from plain decoding somewhere in up to 2,048 tokens, 3.06 divergences per 1,000 compared tokens, and 77 of 80 MTP outputs, 2.50 per 1,000 [N-E59]. At each divergence the plain run’s two competing tokens were close: an exact BF16 tie at 47 of the 77 DFlash divergences and 40 of the 77 MTP ones, within one BF16 spacing at 28 and 37, and in 2 DFlash cases the speculative run’s token was outside the plain run’s top two. The classes are one-sided, because only the plain run recorded its top two log-probabilities, and the DFlash divergences have not been traced to a kernel; the rates are of the same order as the plain-decoding floor between concurrency 1 and 32 at a cap of 16 running requests (3.42 per 1,000). Two-sided margins and a trace to the first differing kernel were not measured. Four limitations are candidates: acceptance that decays with block position or by domain; drafting cost dominated by the full-vocabulary head for fifteen positions; the verifier’s per-position state writes, which bound block size at high concurrency; and draft states far from the target’s in the head’s metric, which is what defeats transport on DFlash drafts (median relative drift 0.92, Section 5.3 of the paper). The public drafter’s training data are not published, so none of these limitations can be traced to its data. A custom drafter is worth training only against one of them, judged by total serving time: fine-tuning on the target’s own outputs for acceptance, adding DFlash 2’s selector (Inco AI, 2026a;b) for acceptance, or an auxiliary loss on $\|h^t - h^d\|_2^2 / \|h^t\|_2^2$ if the roughly hundredfold drift reduction transport needs proves reachable (Appendix C.8 of the paper). A training set of 18,000 prompts, disjoint from every benchmark and quality set by construction, is built and its manifest committed [N-E64]; no drafter was trained, and the measurements here do not decide which of these limitations a training run should target.

5.5 Training a selector for rate (P6, hypothesis)

A later proposal keeps the DFlash backbone and the verifier frozen and trains only a small predecessor-conditioned selector over the backbone’s $K = 16$ candidates per position, $q_i(b | a) = \text{softmax}_{b \in C_i} [\ell_i(b) + s_{\theta,i}(a, b)]$, so that one coherent path is proposed; DFlash 2’s selector architecture (Inco AI, 2026a), whose training objective the announcement does not state, is the baseline, not a novelty. The objective is emitted tokens per unit time, $\mathbb{E}[\sum R] / \mathbb{E}[\sum C]$ with $R = L + 1$ per cycle, trained as $R - \lambda C$ with λ set to the measured rate. If λ^* is the best achievable ratio in Equation (1), then $\mathbb{E}R - \lambda^* \mathbb{E}C \leq 0$ for every policy, with equality exactly for the policies that attain the ratio, because $\mathbb{E}C > 0$; so maximizing the difference at the right λ maximizes the rate. Two identities make the accepted-length term trainable exactly.

Proposition 5.1 (Prefix value of a sampled draft policy). *Let the draft path Y be sampled position by position, $Y_i \sim q_i(\cdot \mid Y_{<i})$, and let L be the number of accepted drafted tokens. Against a greedy target with continuation g , $\mathbb{E}L = \sum_k \prod_{i \leq k} q_i(g_i \mid g_{<i})$, and $\nabla_\theta \mathbb{E}L = \sum_j (\sum_{k \geq j} S_k) \nabla_\theta \log q_j(g_j \mid g_{<j})$ with $S_k = \prod_{i \leq k} q_i(g_i \mid g_{<i})$. Against a sampled target with the standard rule, with $A_k = \prod_{i \leq k} \min(1, p_i(Y_i)/q_i(Y_i))$,*

$$\mathbb{E}L = \mathbb{E}_Y \left[\sum_k A_k \right], \quad \nabla_\theta \mathbb{E}L = \mathbb{E}_Y \left[\sum_j \mathbf{1}\{q_j(Y_j) < p_j(Y_j)\} \left(\sum_{k \geq j} A_k \right) \nabla_\theta \log q_j(Y_j) \right].$$

Proof. Greedy: $L \geq k$ exactly when $Y_i = g_i$ for $i \leq k$, which has probability S_k , and $\mathbb{E}L = \sum_k \Pr(L \geq k)$; differentiate the product. Sampled: given the path, position k is reached and accepted with probability A_k , so $\mathbb{E}L = \mathbb{E}_Y \sum_k A_k$. Its gradient has a score part and a pathwise part. In the score part, $\mathbb{E}[A_k \nabla \log q_j(Y_j)]$ vanishes for $k < j$, because A_k depends only on $Y_{<j}$ and the score has conditional mean zero, leaving $\sum_j \nabla \log q_j \sum_{k \geq j} A_k$. In the pathwise part, $\nabla \min(1, p_j/q_j)$ is zero where $q_j < p_j$ and $-\min(1, p_j/q_j) \nabla \log q_j$ where $q_j \geq p_j$, which contributes $-\mathbf{1}\{q_j \geq p_j\} \sum_{k \geq j} A_k \nabla \log q_j$. The two parts add to the stated expression. $\square$

Freezing the acceptance probabilities as a fixed reward keeps only the score part and so gives a biased gradient. The proposition is a direct calculation and we claim no novelty for it. Acceptance-aware training already optimizes the greedy expected accepted length directly, with a window total-variation loss for sampling (Xia et al., 2026); BV Loss trains on the exact expected acceptance length under block verification (Kim et al., 2026b); D-PACE, PARD-2 and VAT weight a cross-entropy by position or by the chance of reaching it (Wu et al., 2026; An et al., 2026; Gu et al., 2026); variational speculative decoding optimizes a bound on sequence acceptance (Zou et al., 2026); TIGER uses the realized accepted length as a reinforcement-learning reward (Vo et al., 2026); and DBLAST drafts blocks with explicit dependence between positions (Karimi et al., 2026). The only trained DFlash selector we know of, LiLiCorr’s, uses cross-entropy over candidates with a target-weighted penalty on distractors (Rusanovsky et al., 2026). What remains for P6 is the combination: a DFlash 2-style selector over a frozen backbone, trained for rate rather than for accepted length, with the sampled-target gradient above; whether that pays is the experiment. The verification width must be chosen from committed-prefix features before proposals are drawn, for the reason of Appendix B.5 of the paper; choosing a draft-tree budget from a profiled verify-cost curve at inference, as CaDDTree does for DFlash marginals (Zhang et al., 2026a), and confidence-scheduled width, as in DSPark (Cheng et al., 2026), are the existing ways to set it.

The cheapest test needs no training. From ordinary greedy runs, log the candidate sets C_i and the completed greedy continuation g , and compute $U = \max\{k : g_i \in C_i \text{ for all } i \leq k\}$, an upper bound on the accepted length of any selector over those frozen candidates; if U barely exceeds DFlash’s actual acceptance there is little to gain. Only then is the rate objective compared, at fixed width, with the strongest matched objective (VAT’s weighting ranks first in its own ablation (Gu et al., 2026); D-PACE-style weighting and position-weighted cross-entropy are the other controls) using the same backbone, candidates, selector size, features, data and budget. The pre-registered effect size is 1.25 $\times$ end to end over the strongest matched baseline, rejected if the upper confidence bound falls below it; a gain that vanishes against an identical selector trained with another objective supports the selector, not the objective.

The screen does not kill P6 [N-E60]. On the block-16 trace of the first panel the drafter’s block was recomputed offline at each verify cycle’s anchor; the offline drafter agrees with the engine’s drafted token at 97.4% of positions, and its own acceptance matches the engine’s (6.200 against 6.204 tokens per cycle). Over the top 2, 4, 8 and 16 candidates per position the bound $1 + \mathbb{E}U$ is 7.51, 8.47, 9.33 and 10.14 tokens per cycle, against the engine’s 6.20; for chat it rises from 3.99 to 7.17. It is an oracle bound per cycle along the stock trajectory, not a bound on another selector’s tokens per cycle, since a better selector would reach other states, and it excludes the 207 of 21,274 cycles whose block runs past the end of the output, which accept less than average and so bias it slightly upward. The matched-objective comparison was not run.

5.6 Exact repair of long windows (P2, P3)

A block drafter proposes a whole block in one forward pass and the target verifies it in one pass; the public DFlash card reports a mean accepted length of 7.48 tokens per verify pass at block size 16 on MATH-500 on a B200 (z-lab, 2026). While decoding is limited by weight traffic, a verify pass over a wider block costs little more than one over a narrow block, so the question is whether the target can commit far more than about seven tokens per pass without giving up exactness. Two proposals are tested with oracle experiments designed to kill them cheaply: long-window repair of a DFlash window by the target’s own predictions (P2), and target-anchored residual decoding (P3), which repairs a window by updating the target’s previous computation with fixed low-rank corrections and then audits it exactly. Both keep exactness by the same device: only tokens that an exact audit pass of the target has verified are committed, approximate states never enter the committed cache, and the noise of a position is never resampled.

Prior work Reusing the verifier’s work on rejected positions is an active line. ReTrace conditions the next DFlash block on the rejected trajectory (Lin et al., 2026), DFlow feeds the verifier’s hidden states at rejected positions back to the

drafter (Zhang et al., 2026c), Carryover Drafting reuses those states as extra draft context (Koo et al., 2026), and CURE repairs uncertain positions of block-parallel drafts locally (Liu et al., 2026a); token recycling reuses candidate tokens from earlier steps (Luo et al., 2025), and CARD and H-Spec restructure parallel speculation around cached queries and drafter state (Zhou et al., 2025; Jiang et al., 2026). Jacobi-style iteration has been applied to translation, to feedforward computation, to image generation with probabilistic acceptance and to causal language models trained for it (Santilli et al., 2023; Song et al., 2021; Teng et al., 2025; So et al., 2026; Hu et al., 2026), and speculative speculative decoding overlaps drafting with verification (Kumar et al., 2026). P3’s anchored update is closest to computation reuse across similar inputs, which DeltaCNN, Eventful Transformers and CacheBlend apply to video frames, vision tokens and cached retrieval contexts (Parger et al., 2022; Dutson et al., 2023; Yao et al., 2025). None of these, as far as we found, commits a long window through an exact audit of a cheap residual evaluator of the target itself; that combination is what the kill tests below examine.

Setting and cost model Greedy decoding solves a triangular system: with $F_i(y_{<i})$ the target’s argmax after the committed prefix and $y_1, \dots, y_{i-1}$, the continuation satisfies $y_i = F_i(y_{<i})$. A verify pass evaluates F at every position of a candidate block from the committed prefix and commits the longest prefix with $y_i = F_i(y_{<i})$ plus the target’s token at the first mismatch. The commit is exact for any candidate, because every committed token is the target’s own decision on a verified prefix and the committed recurrent state and attention cache are those the target computed for that prefix. The Jacobi map $y_i^{(r+1)} = F_i(y_{<i}^{(r)})$ fixes at least one more position per application and reaches the solution within the block length; this is Jacobi iteration applied to decoding (Song et al., 2021; Santilli et al., 2023), and lookahead and consistency-trained decoders pursue related fixed-point iterations (Fu et al., 2024; Kou et al., 2024). For sampling, fixed per-position random inputs held across repairs give the target law, as in Theorem B.3 of the paper.

Let DFlash commit A_D tokens per cycle of cost C_D (draft, verification, state commit and control). A repair scheme that commits A_R tokens per attempt at cost $C_R = C_{\text{init}} + C_{\text{anchor}} + k C_{\text{repair}} + C_{\text{audit}} + C_{\text{state}} + C_{\text{control}}$, averaged over all attempts including failures and fallbacks, speeds decoding up by $S_{\text{decode}} = A_R C_D / (A_D C_R)$. Prefill and scheduling take a fraction f of a request’s time that repair does not change, so end to end $S_{\text{e2e}} = 1 / (f + (1 - f) / S_{\text{decode}})$; a $5\times$ end-to-end gain needs $S_{\text{decode}} = 5.44$ at $f = 2\%$ and 9 at $f = 10\%$ (derived). Two oracle bounds use a perfect candidate of width B (the reference continuation, or a forced full acceptance). With $V(B)$ the target verify pass and $C_{\text{state}}(B)$ the work needed to continue from the accepted boundary,

$$S_a(B) = \frac{B C_D}{A_D (V(B) + C_{\text{state}}(B))}, \quad S_b(B) = \frac{B C_D}{A_D (C_{\text{anchor}}(B) + C_{\text{audit}}(B) + C_{\text{state}}(B))}. \quad (5)$$

S_a is an ideal drafter paying one verify pass per block of B tokens; S_b is P3’s two-pass design with free repair, $C_{\text{anchor}}(B)$ a full pass that also writes the anchor cache and $C_{\text{audit}}(B)$ a full pass. If S_b cannot reach the target end to end, the two-pass design is dead whatever the repair does; if even S_a cannot, better drafting alone cannot reach it with this verifier. A concrete repair pays only if $A_R / C_R > A_D / C_D$, and it is rejected outright if a lower bound on its cost gives $C_R^{\text{lower}} / (B + 1) \geq C_D / A_D$, since no attempt commits more than $B + 1$ tokens.

What a repair must achieve Local low rank does not imply useful repair of a whole model. Three things must hold separately: the fixed cheap operators stay accurate after candidate tokens change, not only near the anchor; their errors stay below the decision margins after attention, nonlinearities, the recurrent state and later layers; and each sweep resolves enough positions to pay for itself and for the audit. A sufficient condition links these quantities. Let m_i be the exact top-two logit margin at position i , ε_i a bound on the logit error the approximate evaluator makes at i when its inputs are exact, b_j a bound on the error in the current guess’s effect at an earlier position j ($b_j = 0$ once j is correct), and C_{ij} the influence of position j on the logits at i . If $2(\varepsilon_i + \sum_{j<i} C_{ij} b_j) < m_i$, position i is correct after the next sweep, by the triangle inequality. Two consequences follow. A persistent approximation error η in a recurrence with contraction factor $a < 1$ leaves an error floor of about $\eta / (1 - a)$. And a faithful evaluator can advance only as fast as exact Jacobi iteration, which on a copy chain $F_i(y) = y_{i-1}$ gains one position per sweep even with exact rank-one operators; a non-faithful map can jump ahead, as consistency-trained models do (Kou et al., 2024), but that would be better drafting, not repair. Expected accepted length is best read through hazards: with h_i the conditional failure rate at position i given an accepted prefix, $\mathbb{E}A = \sum_a \prod_{i \leq a} (1 - h_i)$. The decisive control is a small target-conditioned drafter given the same anchor information and matched in compute, memory traffic, training data, verifier width and target calls; if it does as well, the supported claim is better target-conditioned drafting, not cheap repair of a trajectory. This is analysis that fixes the measurements, not a result.

Test 1: perfect continuations at width B $V(B)$ and $C_{\text{state}}(B)$ are measured inside SGLang’s DFlash worker in the DFlash model card’s serving configuration (FlashInfer attention and GDN kernels, CUDA graphs, overlap scheduling, concurrency 1) at block widths $B = 16, 32, 64, 128, 256$, forcing every verify pass to accept the whole block; the committed tokens are then meaningless, but a pass’s cost does not depend on token values. CUDA events around each phase of the cycle give

GPU times without host synchronization. SGLang’s GDN verify kernels write one FP32 state per verified position and the commit copies the accepted one back. At the pinned commit the GDN verify kernel follows the decode backend unless overridden, so this configuration verifies with FlashInfer’s kernel, while SGLang’s default linear-attention backend, which the served arms of Appendix E.3 of the paper keep, verifies with a Triton kernel (code reading). The replay protocol, which keeps per-token inputs and folds the accepted prefix at commit, does not start with DFlash on this GDN model at the pinned commit. The per-position state writes are measured at $B = 16, 64$ and 256 by a verify pass that drops them, and bounded by their bytes at a 3.0 TB/s write rate at the other widths. Feeding the reference continuation instead of forcing acceptance shows how often a verify pass at width B rejects the plain-decoding continuation; such rejections are the stock noise floor of Appendix D.4 of the paper, not drafting errors. The measurement [N-E65] ran eight MATH-500 problems of 2,048 tokens at concurrency 1, with verification forced to accept the whole block through the engine’s simulated-acceptance setting, in the configuration of the drafter’s acceptance runs; foreign CPU load stayed below 0.6 cores. On the same problems DFlash at block 16 takes $C_D = 7.457$ ms per cycle and commits $A_D = 7.675$ tokens, and prefill and scheduling take $f = 2.28\%$ of a request, so a $5\times$ end-to-end gain needs $S_{\text{decode}} = 5.51$. The verify pass takes $V(B) = 4.78, 7.30, 15.62, 28.10$ and 44.82 ms at $B = 16, 32, 64, 128$ and 256 , and the state commit 0.11 – 0.14 ms at every width (medians; the p10 and p90 cycle periods are within 1.3% of the median). An ideal free drafter would reach $S_a = 3.17, 4.19, 3.95, 4.41$ and 5.53 in decode, and $5\times$ end to end only at $B = 256$ (5.01); paying DFlash’s own drafting cost at width B , perfect blocks reach 4.68 end to end there. For P3, a ceiling that charges the anchor pass a single read of the 8.41 GB of weights at the measured 3.83 TB/s plus writing the anchor cache, and the audit its measured $V(B)$ and commit, reaches 5.24 in decode and 4.78 end to end at $B = 256$; an estimate that charges the anchor pass $V(B)$ less its measured per-position state writes, plus writing the anchor cache, gives 2.89 and 2.77. These are derived from the measured times.

With the audit charged the measured verify pass, P3’s two-pass design misses the $5\times$ target at every width even with free repair, so by its pre-registered rule it is refuted for this verifier, and P2 meets its target only at the edge of this test, at $B = 256$ with a drafter that costs nothing and every block accepted. Both verdicts are statements about the FlashInfer verify kernel, and two further runs at $B = 16, 64$ and 256 separate its parts [N-E66]. Dropping the per-position FP32 state writes, with an engine patch used for timing only, shortens the pass by 0.26, 1.26 and 4.10 ms, 5.3, 8.1 and 9.1% of $V(B)$, close to their bytes at 3.0 TB/s. These savings compare a pass from a later session with the Stage A session’s $V(B)$; a control that ran all three variants at $B = 16$ and 256 in one session reproduces every cross-session time to within 1.5% and puts the writes at 0.18 and 3.98 ms, 3.8 and 8.9% of $V(B)$. An audit that dropped those writes and continued from the accepted boundary by an uncharged replay would therefore lift P3’s ceiling to 5.18 end to end at $B = 256$, where the target needs a saving of 2.33 ms; such a verifier would also make the DFlash baseline cheaper, since a block-16 cycle writes 16 per-position states. The larger cost is the FlashInfer kernel itself: selecting SGLang’s Triton GDN verify kernel instead, a stock option, makes the whole pass $2.06\times$ faster at $B = 64$ and $2.34\times$ at $B = 256$ at one request ($V(B) = 4.55, 7.58$ and 19.18 ms).

The Triton kernel is a different numerical path. DFlash at block 16 with it commits 7.603 tokens per cycle against 7.675, so its greedy outputs differ from the FlashInfer configuration’s, and no figure that uses it has an exactness class against plain decoding. With the DFlash baseline on the same kernel, 7.302 ms per cycle and $f = 2.39\%$, so that $5\times$ end to end needs 5.54 in decode, the derived speedups change both verdicts (Table 9). Perfect blocks reach $5\times$ end to end from $B = 64$, 5.21 paying DFlash’s drafting cost and 6.84 with a free drafter, and P3’s two-pass estimate passes at $B = 256$ (6.21), its ceiling already at $B = 64$. The baselines on the two kernels are single runs from two sessions, so the baseline’s own gain from the switch is not established; the wide blocks gain about two times. The Stage A verdicts therefore depend on the verify kernel. With the Triton kernel P2 turns on drafting, since a $5\times$ gain needs blocks of 64 or more tokens accepted almost entirely and the repair mechanisms tested here do not produce them (Tests 2 and 3); P3 stays refuted by Test 3 whatever the verifier. In per-position terms, a cycle drafted by DFlash at width B would have to commit 61.1 of 64 tokens at $B = 64$ or 132.2 of 256 at $B = 256$, which for a drafter with the same conditional acceptance at every position means at least 0.9985 or 0.9941, against 0.80 to 0.91 for the public drafter at block 16; with the FlashInfer kernel none of the measured widths, 16 to 256, suffices even with every block accepted (derived) [N-E67]. The drafter’s top-16 candidate sets still contain the target’s token at a conditional rate of 0.91 to 0.92 from the fourth position, but that rate is measured only at the cycle starts of the stock block-16 trajectory, so it bounds neither a selector, which would start its cycles elsewhere, nor a wide-block drafter. Past $B = 16$ the FlashInfer pass grows by about 0.167 ms per extra token, roughly five times what that token’s GEMM work and its 48 MiB per-position state write account for. A kernel trace at $B = 256$ (Nsight Systems, two requests, 19 complete cycles) finds most of the pass in FlashInfer’s GDN verify kernel [N-E66]. It takes 1.36 ms per GDN layer, 5.3 μ s per position and layer, which fits a walk over the positions one after another; that is 32.68 ms per cycle and 72% of the verify phase under the profiler. The causal-convolution update follows with 6.48 ms and the GEMMs of target and drafter take 5.87 ms. The Triton kernel replaces only the verify kernel; if the other kernels are unchanged, the Triton GDN kernel takes about 6.8 ms of the 19.19 ms Triton pass and the convolution update about a third of that pass (derived, not traced). The Triton kernel’s exactness class, a trace of its pass and the acceptance of the reference continuation per width were not measured.

Test 2: Jacobi repair from DFlash windows The second test asks how close exact repair from real DFlash windows gets, with no reference tokens. Every DFlash block $y^{(0)}$ is verified, and then k further full target passes run from the same

Table 9. Stage A with SGLang’s Triton GDN verify kernel, over DFlash at block 16 on the same kernel: speedup in decode / end to end. Derived from measured verify-pass times; the S_b estimate bounds this kernel’s state writes by their bytes. The kernel switch has no exactness class [N-E66].

B	S_a , free drafter	Perfect blocks, DFlash drafting	S_b estimate	S_b ceiling
16	3.30 / 3.12	2.10 / 2.05	1.71 / 1.69	2.23 / 2.17
64	7.98 / 6.84	5.81 / 5.21	4.30 / 3.99	6.17 / 5.49
256	12.72 / 9.94	10.73 / 8.71	7.11 / 6.21	11.28 / 9.06

committed prefix: *recycle* sweeps apply the Jacobi map, and *keep* sweeps replace only the first mismatching token by the target’s token and keep the rest of the draft. The pass on the original draft is then re-run and committed, so the trajectory is plain DFlash and each sweep’s accepted length is measured on real windows. Re-running a verify pass on the same prefix needs the committed GDN convolution window restored first, because the verify kernel shifts it in place, and the probe checks that the re-run reproduces the first pass’s argmax exactly. A sliding variant drafts each new block from the previous pass’s suffix predictions and fills the rest with DFlash.

Its one-step form has been measured offline on the drafter’s block-16 trace [N-E68]. At the 18,525 cycle boundaries after a rejection, a next block drafted from the previous pass’s target predictions accepts 0.52 drafts on average, against 4.44 for the fresh DFlash draft the engine used; keeping the previous draft’s tail gives 1.11, and choosing the best of the three with hindsight 4.55, 2.5% more than fresh drafting. At block 8 the figures are 0.45 and 1.15 against 3.21 (3.30). The target’s prediction after a wrong draft token matches the continuation 14% of the time once that token is corrected, against 84% for the drafter’s first drafted token. Recycling the target’s suffix predictions therefore does not improve on fresh drafting. Full recycle sweeps advance at the rate of the copy chain above: on the 80 held-out blocks of Test 3, exact Jacobi from the DFlash draft accepts 2.98 drafts before any sweep and 4.33, 5.35, 6.38, 7.34 and 8.30 after one to five full target passes, about one token per pass [N-E69]. In the engine [N-E70], on 67 decode checkpoints of the drafter’s shared panel (384 new tokens each, greedy, one request at a time, FlashInfer GDN kernels, four sweeps of each kind per block), each exact sweep adds 0.84–1.58 accepted drafts at $B = 16$ and 1.07–1.90 at $B = 32$, fewer with every sweep. The committed tokens per exact target pass, $(a_r + 1)/(r + 1)$ with a_r the accepted drafts after r sweeps, therefore fall from DFlash’s 6.49 at $B = 16$ to 2.07 (recycle) and 2.19 (keep) after four sweeps, and from 6.69 to 2.30 and 2.45 at $B = 32$. Even P3’s ceiling, which charges the sweeps nothing and pays only an anchor and an audit pass, commits 5.18 (recycle) or 5.46 (keep) tokens per pass after four sweeps at $B = 16$, below DFlash’s 6.49, and 5.74 or 6.13 at $B = 32$ against 6.69. After four keep sweeps 72% of $B = 16$ blocks and 99% of $B = 32$ blocks are still not fully accepted. After one corrected token the target’s prediction right after it changes in 87% of cases (86% at $B = 32$), the next one in 49% and about a quarter four positions on, so a correction invalidates many of the downstream guesses that a sweep would have to confirm. Restoring the GDN states and re-running the original draft reproduced the first pass’s argmax in all 8,053 cycles. With Stage A’s FlashInfer timings at concurrency 1, a sweep costs one verify pass (4.78 ms at $B = 16$), while fresh DFlash commits about 0.87 tokens per ms (this panel’s 6.49 tokens per Stage A’s 7.46 ms cycle); the best sweep, the first keep sweep, adds 0.33 tokens per ms of its own cost, so every sweep lowers throughput (derived). P2 is therefore rejected as a route to 5 $\times$: repair from a DFlash window gains about one token per exact target pass, while Stage A needs blocks of 64 or more tokens accepted almost entirely.

Test 3: anchored residual evaluation For every linear operator $z = Wx$, P3 keeps the anchor $(\bar{x}, \bar{z} = W\bar{x})$ from a full pass over a DFlash block and repairs a changed block with $\tilde{z} = \bar{z} + (WU)U^T(x - \bar{x})$, where the orthonormal basis U is fixed per operator input and WU is precomputed, so a repair sweep streams WU instead of W ; a basis computed from the current correction would stream W again. Nonlinearities, normalizations, the causal convolution, the GDN recurrence from the exact committed state and attention over the exact prefix run exactly on the repaired values. The measurement [N-E69] ran the Hugging Face implementation of the target (BF16, pinned revision) block by block from an exact prefix cache; its argmax agrees with the engine’s verify argmax at 99.1% of the development blocks’ positions. Blocks are real block-16 DFlash drafts with a rejection, from the drafter’s shared trace: 120 development blocks from even-indexed requests, on which each operator’s basis is fitted as the leading principal components of its exact input changes, and 80 held-out blocks from odd-indexed requests. Every linear operator is repaired (the GDN input and output projections, the attention projections, the MLP and the tied head) at ranks 16 to 512; rank 0, which reuses the anchor outputs unchanged, is the initializer without repair. Each held-out block is replayed twice against the original anchor, $y^{(1)} = F(y^{(0)})$ and $y^{(2)} = F(y^{(1)})$, and at every position the test records whether $\arg \max \tilde{z}_i = \arg \max z_i$ and the ratio $R_i = 2\|\tilde{z}_i - z_i\|_\infty / m_i$ ($R_i < 1$ guarantees the same decision). A free-running repair from $y^{(1)}$ is audited exactly after every sweep and compared with exact Jacobi iteration from the same anchor.

The first of the three conditions fails, and the other two fail with it. On held-out exact changes the median relative error $\|W(I - UU^T)\Delta x\| / \|W\Delta x\|$ of the first projection that reads each operator input is 0.76–0.90 at rank 16, 0.64–0.81 at rank 128 and 0.51–0.73 at rank 512, by operator class, and even on the development blocks rank 128 captures a median of only

51–64% of the change energy: the changes that token corrections cause are not confined to a small fixed subspace. On the first replay the repaired argmax equals the exact one at 35% of the changed positions at rank 0, 37% at rank 128 and 44% at rank 512, and at the corrected position itself at 11–14%; on the second replay at 24–35%. $R_i < 1$ at 43 of the 921 positions of the first replay where it is defined (4.7%; the other 41 are exact ties of the exact logits, with margin 0), the same positions at every rank including rank 0; these have large exact margins (median 12 logits, against 1.3 over all replayed positions), where even the anchor’s logits stay within half the margin, so the repair earns no certificates, and the median R_i is 13–16. Free-running repair accepts 4.33 drafts before its first sweep and 4.33–4.44 after four sweeps at every rank, at most 0.11 more than no repair, while exact Jacobi from the same anchor gains about one token per sweep (Test 2), which also bounds any faithful cheap evaluator.

The economic gate compares committed tokens per unit cost with DFlash at block 16, charging repair sweeps a lower bound that counts only their bytes and the audit its measured $V(16)$ and commit (derived). The anchored evaluator reaches at most 0.28 of DFlash’s rate at any rank and sweep count. Exact Jacobi with each sweep charged only the bytes of reading the anchor cache and the GDN state, a faithful evaluator at that cost, reaches 0.27, 0.47, 0.66, 0.85 and 1.03 after zero to four sweeps, so even it would at best match DFlash after four sweeps. The bound of the cost model, the audit’s cost over the most tokens an attempt can add, does not reject at $B = 16$; the measured progress does. The pre-registered control against a compact drafter with the same storage and compute was not run: the evaluator gains 0.11 accepted drafts over no repair, which leaves nothing for that control to explain. P3 is refuted at the operator level, whatever the verifier.

5.7 Reusing a cached window after a rejection (P9)

P9 proposes, after a rejection, to condition a program compiled from the first cycle’s cached candidate sets on the corrected prefix and to propose the rest of the old window from it, at most once, instead of drafting afresh. Its pre-registered rule compares the two over two cycles,

$$\Delta = \mathbb{E}[G_2^R - G_2^F] - r_F \mathbb{E}[A + T_2^R - T_2^F], \quad r_F = \frac{\mathbb{E}[G_1 + G_2^F]}{\mathbb{E}[T_1 + T_2^F]}, \quad (6)$$

with G the tokens a cycle commits, T its time, R and F reuse and fresh drafting in the second cycle and A the extra cost reuse adds to the first, and rejects the reuse at the tested configuration if the upper end of Δ ’s 95% interval is at most 0.

Its first test is a support oracle at concurrency 1 [N-E71]. On the drafter’s block-16 trace (21,067 cycles of 80 requests), block 16 being the fastest DFlash block at concurrency 1 on the serving workload (Table 6), there are 18,274 boundaries after a rejection. P9’s program conditions on the whole corrected prefix through the frozen candidate sets of the old block, so reuse is possible when those top- K sets contain every token of the corrected prefix and some of the old window remains, which is known at decision time; the oracle then proposes the true token wherever the old candidate set contains it, charges a reused cycle the measured fresh cycle less its 2.33 ms draft phase, that is a full block-16 verify padded beyond the remaining positions, and takes A , the cost of conditioning and the draft-cache catch-up as zero, so it bounds from above P9’s program over the same candidate sets when that program verifies a padded block-16 and reuses at every boundary where reuse is possible. With the top-16 sets the corrected prefix is supported at 88.8% of the boundaries, the oracle commits 5.08 tokens in the second cycle against 5.55 for fresh drafting on those boundaries, and $\Delta = +0.99$ tokens per boundary (95% interval 0.89 to 1.07). With the top-8 sets $\Delta = +0.22$ (0.12 to 0.32), and with the top-1, top-2 and top-4 sets the rule rejects reuse. Keeping the old draft’s tail without conditioning, the unchanged cached control, gives $\Delta = -1.93$ and is rejected. These verdicts, the rejections included, hold for P9’s program over the recomputed candidate sets, with a padded block-16 verify and reuse at every boundary where reuse is possible. A program whose support starts at the correction, conditioning on the suffix alone, is a different class and is not covered; with recomputed sets it could also reuse at boundaries where the recomputation drops an already accepted token. The engine’s own sets, whose top-1 token differs from the recomputation’s at 2.6% of positions, were not captured, and a program over them is not covered. At concurrency 1 the oracle bound therefore does not reject reuse with top-8 and top-16 sets. It is an upper bound, not a served speed-up: the candidate sets are an offline recomputation whose top-1 token matches the engine’s draft at 97.4% of positions, the costs are one run’s phases at concurrency 1, and a real program must select the token, which the old draft’s own choices fail to do (the cached control), while its compile, conditioning, retention and draft-cache catch-up costs come out of the same margin.

Three further bounds over the same boundaries and bootstrap resamples widen the class of programs, and none of them is a result about a real program [N-E71]. A program that verifies only the remaining positions at a smaller width could pay less for its second cycle than the padded 4.73 ms verify the oracle charges. Charging the reused verify nothing, its lower bound, and every other phase at its block-16 cost gives $\Delta = +0.01$ (0.00 to 0.02), +0.89 (0.81 to 0.98), +1.88 (1.75 to 2.05), +2.89 (2.71 to 3.10) and +3.86 (3.64 to 4.11) with the top-1, top-2, top-4, top-8 and top-16 sets. This bounds from above P9’s program reusing at every boundary where reuse is possible, at any verify width whose other phases cost at least their block-16 values in the measured fresh block-16 run (0.395 ms per cycle beyond draft and verify), and it rejects none of the tested sets, so this bound alone does not extend the rejections of the top-1, top-2 and top-4 sets to a narrower verify; the verify measured at each remaining width, below, does. A narrower cycle can also cut those phases, to 0.384 ms at block 8,

so it is not a bound for every implementation. On the point estimates the Δ of the top-1, top-2 and top-4 sets stays negative only while a reused verify takes more than 0.89, 2.71 and 3.87 ms on average; the remainder verify measured below settles this for SGLang’s verify pass with FlashInfer’s kernel. A program that chooses between reuse and fresh drafting at each boundary could avoid the boundaries where reuse loses. An oracle gate that knows fresh drafting’s outcome and takes the better of the two at each boundary where reuse is possible has a Δ of at least 0, and at least the Δ of always-reuse, which reuses at every such boundary, by construction, so it rejects nothing: +0.01 (0.01 to 0.01), +0.45 (0.42 to 0.49), +0.91 (0.85 to 0.98), +1.44 (1.35 to 1.53) and +2.05 (1.93 to 2.18) with the same five sets. It bounds from above P9’s program with any gate over the same sets with a padded verify, and its Δ is the bound to quote for such programs. Its excess over always-reuse, 1.06 to 1.33 tokens per boundary with the top-2, top-4, top-8 and top-16 sets, is only the gain from gating the oracle’s reuse arm: a real program’s reuse arm is no better than the oracle’s, so gating it can gain more, and the excess does not bound that gain. The same gate with the reused verify free bounds P9’s program with any gate at any verify width, under the same condition on the other phases: +0.04 (0.03 to 0.05), +1.47 (1.35 to 1.61), +2.54 (2.35 to 2.76), +3.47 (3.24 to 3.73) and +4.35 (4.09 to 4.64). A real gate decides before the second cycle without seeing fresh drafting’s outcome, so it pays off only if what is known at decision time (the correction position, the number of old positions left and the old sets’ probabilities at them) predicts the boundaries where its own reuse arm loses to fresh drafting; the length of the supported suffix is not known then, because it depends on the greedy tokens after the correction.

Three measurements from one later session put costs into these bounds [N-E71], all with FlashInfer’s GDN verify kernel, as in the oracle’s block-16 baseline. After a correction at block position J , a program could verify the $m + 1 = 16 - J$ remaining positions instead of a padded block. Forced full acceptance at widths 2 to 16 gives a verify of 3.83 ms at width 2, 80% of the 4.78 ms at width 16, and about 68 μ s for each further position. Charging every reused cycle the measured verify of its own width, with every other phase at its block-16 value, saves about 0.3 ms of verify on the average reused cycle, because most corrections come early and leave a wide remainder. Then $\Delta = -0.04, -0.57, -0.27, +0.39$ (0.31 to 0.47) and +1.17 (1.09 to 1.26) with the top-1, top-2, top-4, top-8 and top-16 sets: always-reuse with SGLang’s verify pass stays rejected with the first three, and the oracle gate at these widths bounds a gated program at +2.16 (2.03 to 2.30) with top-16. Lifting top-4’s upper end to zero would need the other phases about 0.35 ms below their block-16 values, against the 0.011 ms a block-8 cycle saves. The Triton kernel’s widths were not swept.

The program itself was timed on random values: a fixed-shape program over top-16 sets (the backward messages, the forward message over the corrected prefix and the greedy walk) captured as one CUDA graph takes 133 to 315 μ s per reuse, longest when the correction comes first, and changes little with the rank (2, 4 or 8) or the batch (1, 8 or 16 requests at once); run eagerly it takes 3.2 to 5.8 ms, longer than the draft it would replace. Building its matrices from the drafter’s state is not included. At concurrency 1 the graph costs 6 to 14% of the 2.33 ms draft, and charging $A = 0.3$ ms on every reuse lowers Δ by r_F times the supported rate times A , about 0.2 tokens with top-16.

At concurrency 8 and 16, fresh DFlash at block 16 spends 25% and 20% of the cycle drafting (31% at concurrency 1), and the batched draft grows by 73 and 55 μ s per added request. A reusing request does not skip the batch’s draft there: the batch drafts one row fewer, and the shorter cycle serves every request. The oracle at these concurrencies charges the padded verify, takes acceptance from the concurrency-1 trace, without remeasuring it, and takes the phases from the run at that concurrency. Crediting a reuse with the request’s even share of the draft, an upper bound if the draft’s cost is concave in its rows, as the measured growth suggests, keeps top-16 positive (+0.70 at concurrency 8 and +0.48 at 16) and leaves top-8 not rejected at 8 only. Crediting it with the growth per added request, which under the same condition bounds what one request reusing alone saves, rejects always-reuse with every set, top-16 included (−0.19 and −0.18), while the oracle gate still bounds a gated program at +1.27 with top-16. A real batch saves less still: SGLang replays the draft from graphs captured at fixed batch sizes, so a batch that drafts one row fewer may pay for the full graph, and one step of the program costs more GPU time than the 55 to 73 μ s draft row it removes. Beyond concurrency 1, reuse would have to remove whole draft passes, with many requests of a batch reusing in the same cycle, which this oracle does not measure. A conditioned program and a matched autoregressive refiner over the same cached information, each against Equation (6) with real progress and all costs charged, were not built.

5.8 Three external drafting proposals (P-A, P-B, P-C)

Three external proposals aim at the same $5\times$ end-to-end gain over optimized DFlash with blocks of 64 to 256 tokens, without near-perfect drafting at every position: innovation-clock drafting (P-A) overrides a cheap interpreter only where it is wrong; prefix-isolated sparse planning (P-B) drafts a few anchor tokens and expands between them; causal defect drafting (P-C) repairs a rejected window by sweeping from the audit’s target scores. Their first decisive tests ran on the CPU over the block-16 DFlash trace, against rules committed before any of the oracles ran, and each result is an oracle upper bound: perfect controllers, free drafting where stated, and the measured cycle with SGLang’s Triton GDN verify kernel, whose exactness class was not established [N-E72]. All three are no-go. For P-A, about half the output positions are ones the interpreter gets wrong (0.47 to 0.54, by interpreter); a perfect-event oracle is rejected for every budget up to 16 free perfect overrides per cycle and passes only at 32, and controllers that draw their tokens from DFlash’s top-16 candidates or the interpreter have a mean conditional support of 0.924 per position, against the 0.9892 ($B = 64$) and 0.9925 ($B = 256$) needed;

that support is measured over block positions 5 to 15 at the anchors the stock block-16 trajectory visited, and a program drafting 64 or 256 tokens would visit other anchors and later positions. For P-B, with a perfect controller and an expander that drafts the first gap no better than DFlash at block 16, every anchor count the proposal considers, 2 to 16, is rejected: at most 14.4 to 37.0 tokens per cycle at $B = 64$ against 44.4 needed, and 43.4 at $B = 256$ against 111.6, the most lenient accounting. A correction made after the first run, with the rule unchanged, to the bound’s extension past slot 15 raised the $B = 256$ figure from 39.7 and the $B = 64$ figure for 2 or 4 anchors from 13.7; no verdict changed. For P-C, at 18,022 boundaries after a rejection, a sweep with point-predictor interpreters accepts at most 1.60 or 2.00 drafts per boundary against 3.56 for fresh DFlash drafting over the same horizon. The common cause is the requirement of Test 1: each proposal still needs a drafter far more accurate per token than DFlash at block 16, and none of the three mechanisms supplies it. The proposals’ Lean statements, in formal/drafting/, elaborate with no sorry [N-E17]; they establish the encodings, the clipping of anchored plans and audit soundness, not the speed-ups, and the defect proposal’s convergence bound requires exact locality, so for a real target it gives one sweep per position, the bound of plain Jacobi iteration.

5.9 Verifying the recurrent state (P7b, P10, P11)

Three proposals concern how a verify pass treats the recurrent state. P7 has two parts. P7a, the five exactness contracts and the witnesses that separate them, frames every exactness claim in the paper (its Appendix D.1). P7b would verify a draft block’s GDN outputs in parallel, through the triangular form of the chunked update instead of the sequential recurrence, emit an output where its sound enclosure lies in one BF16 rounding cell and recompute it otherwise, and commit the accepted state by the rounding-preserving replay of Section 3.5. Its first rejection test is negative for SGLang’s chunked kernel, which is not a usable fast path: it disagrees with the recurrent outputs in most BF16 words and is slower at every width (Section 3.4; synthetic inputs, one request). The certified path itself, against strict replay on a captured trace, was not tested.

P11 would predict after the first target layers how many draft positions each request will accept, run the remaining layers only on that prefix, and resume the same block when the prediction was short; the predictor would control work, never tokens. It was dropped by its author in favour of P10, because verify kernels keep the recurrent state in registers, so shortening the verified suffix does not remove per-token full-state reads (reasoning). P10 instead replays the previous cycle’s accepted tail inside the next verify kernel and checkpoints the state after the anchor token. Its first test rejects it without building: P10 was to be built only if SGLang’s exact GDN fold took at least 9.09% of the speculative cycle at concurrency 8 and 16, and it takes 4.9% and 6.6% (below), so removing it outright would make the cycle at most 1.05 $\times$ and 1.07 $\times$ faster [N-E61].

Exact verification without per-position snapshots SGLang’s fold-every-commit protocol verifies with the stock recurrent kernel, which also writes the raw window to a ring, and replays the accepted prefix into the checkpoint at commit with a bitwise clone of the recurrent update, so it never writes the per-position FP32 states (805 MB per request per cycle at block 16). At the pinned commit it refused DFlash on GDN models; engine patches enable it for DFlash and turn it on for GDN pools under MTP (engine/sglang/patches/drafter/). On random inputs at the GDN layer’s shape, its verify output and committed state are bitwise equal to the stock verify’s in all nine cases tested, while SGLang’s circular replay differs in 19 to 20% of BF16 words [N-E61]. In served greedy decoding the fold equals stock verification in every token and every top-five log-probability: for MTP with three steps on all 80 panel sequences at concurrency 1, with the same running limit and recurrent slots but KV pools of 362,723 (stock) and 416,669 (fold) tokens; and with all pools matched, for DFlash at block 16 on the 8 requests rerun at concurrency 1 with the prefix cache on, and for both on all 80 in batches of 4 (DFlash) and 8 (MTP) with fixed batch composition and the prefix cache off. With a closed-loop client at concurrency 8, stock MTP matched a stock rerun on only 10 of the 80 sequences, so no bitwise claim is made for closed-loop serving above concurrency 1. A first check with unmatched pools had differed, and the matched rerun traced those differences to the fold server’s own pools. In held batches the cycle is 8.6% shorter at batch 8 and 11.9% at batch 16: the verify loses the per-position state writes (1.10 and 2.16 ms) and the commit gains one state read and write per request (0.22 and 0.38 ms). Served on the two tuned DFlash arms in one session (order stock, fold, fold, stock), the fold is 5.8 to 6.1% faster at concurrency 8, 8.2 to 9.4% at 16 and 11.8 to 12.0% at 32 with unchanged tokens per cycle, but 3.2% slower at concurrency 1 on block 16, the best configuration there, and 1.4% on block 8. The loss at small batches followed the kernel’s launch shape. At the pinned commit the ring-writing verify keeps value tiles of 32, where the snapshot-writing verify uses tiles of 4 for up to 64 sequences. In isolation, for one sequence at block 16, the ring-writing verify takes 29.0 μ s per layer with tiles of 32 and 15.7 μ s with tiles of 4, against 16.7 μ s for the snapshot-writing verify [N-E63]. An engine patch that gives the ring-writing verify the narrow tiles keeps the fold bitwise equal to stock in the kernel check and in the matched-pool served checks. In a second session at concurrency 1 to 8, the fold is then faster than stock at every point: 1.020 times stock’s throughput at concurrency 1 on block 16 (887 against 870 tokens/s; all four run pairs 1.019–1.020) and 1.012 on block 8. The narrow tiles cost the fold at eight sequences, though. There it is only 1.018 times stock on block 16 and 1.032 on block 8, against 1.061 and 1.058 without the patch. Between the two sessions stock’s throughput moved by at most 0.8%, while the fold’s fell 3.7% at concurrency 8 on block 16 (an unpaired comparison). The patch also changes the tiles for up to 64 sequences,

where the served sessions did not time it [N-E62]. A kernel microbenchmark of the ring-writing verify alone (random inputs at the GDN layer’s shape, 24 layers in one CUDA graph, four repeats, each tile bitwise equal to tiles of 32) locates the crossover. Tiles of 4 take 0.54 to 0.70 of the time of tiles of 32 at one and two sequences, and 1.03 to 1.40 times as long from three to 64, so by a rule declared before the run the narrow tiles pay only up to two sequences. Scaled to the served cycle (derived: 24 layers times the per-layer difference, over the first session’s fold time per cycle, assuming every cycle runs at the client’s concurrency), the per-layer differences predict the fold’s change between the two unpaired sessions to within 0.7 points at concurrency 1, 2 and 8 on both blocks. That is the size of stock’s drift between the sessions, so the kernel times are consistent with attributing the change to the tiles. At concurrency 4 on block 16 the prediction is a 0.7% loss against a measured 1.3% gain, which is not explained. The same estimate puts the patch’s cost at concurrency 16 and 32 at 3.4 to 3.5% on block 16 and 2.2% on block 8 (derived). It is therefore a trade-off by batch size, and the gains above at concurrency 8 to 32 are for the fold without it. By the same estimate, narrow tiles for up to two sequences only would keep both gains, but that rule was not built [N-E63].

5.10 Selector and depth analysis not tested here

Three pieces of analysis bear on the proposals above; none is under test.

Executing the DFlash 2 selector At the inspected implementation adjacent candidate scores have the form

$$S_t(a, b) = U_t(b) + \sum_{r=1}^{d_s} A(a)_r H(h_t)_r B(b)_r, \quad (7)$$

with a gate that depends on the draft hidden state (Inco AI, 2026a; SGLang contributors, 2026c). A greedy walk needs only the realized row $S_t(x_{t-1}, \cdot)$, which reduces score work from $O(BLK^2 d_s)$ to $O(BLK d_s)$ at the price of a dependent sequence of small contractions. Alternatively the lattice can emit, for every predecessor, its winning successor $f_t(a) = \arg \max_b S_t(a, b)$, after which the path is $x_t = (f_t \circ \dots \circ f_1)(x_0)$.

Proposition 5.2 (Finite-map execution). *Given the same row winners, serial map walking and prefix composition produce the same path. A work-efficient scan of length L over K -entry maps uses $O(LK)$ map work and $O(\log L)$ composition stages, excluding the score producer.*

Proof. The t -th prefix composition applies exactly the first t transitions to the anchor, and function composition is associative. A work-efficient tree scan uses $O(L)$ compositions of $O(K)$ indexed evaluations each, with logarithmic depth. $\square$

Function scanning is standard; the open question would be where it pays. The exact rational reference implements the simpler $O(LK \log L)$ Hillis–Steele scan and checks path equivalence only (Table 11).

Prefix value on a fixed lattice With a substochastic surrogate $r_t(a, b)$ for the probability of matching the next target token, $J_r(x) = \sum_{j=1}^L \prod_{t \leq j} r_t(x_{t-1}, x_t)$ is maximized by the recursion $V_t(a) = \max_b r_t(a, b)(1 + V_{t+1}(b))$ with $V_{L+1} = 0$. If the estimate $\hat{r}$ also lies in $[0, 1]$ and edge errors are uniformly bounded by ϵ_t , telescoping gives $|J_r - J_{\hat{r}}| \leq \delta = \sum_t (L - t + 1) \epsilon_t$ and optimizer regret at most 2δ . LiLiCorr’s finding that an exact best-path search over a different score decodes worse than the greedy rule (Rusanovsky et al., 2026) is the relevant warning.

Choosing depth under a cost model If predicted progress $F_i(d)$ is known and cost depends only on the total optional depth m , the depth assignment for a batch is solved by

$$G_i(m) = \max_{0 \leq d \leq L_i} \{G_{i-1}(m - d) + F_i(d)\}, \quad \max_m \frac{G_B(m)}{C(m; z)}, \quad (8)$$

which is exact only under that cost hypothesis. Published policies choose depth in several ways: OPT-Tree searches for the tree that maximizes expected acceptance (Wang et al., 2025), Sequoia sizes trees for the hardware (Chen et al., 2024), SpecDec++ stops drafting with a learned acceptance head (Huang et al., 2025), DSpark schedules verification from confidences and a profiled throughput curve (Cheng et al., 2026), D-cut prunes draft blocks against a profiled cost table aligned with the engine’s CUDA-graph shapes (Liu et al., 2026b), and an open SGLang pull request adds confidence-ranked verification with a profiled cost table to DFlash 2, following DSpark’s pattern, with greedy benchmarks (SGLang contributors, 2026b). CUDA-graph tiers make the cost discontinuous: with rewards (1, 1.9, 2.7) and costs (1, 2, 2) at depths (0, 1, 2), depth one looks worse than depth zero while depth two is best, so stopping at the first local decline fails. Autotuning a kernel that mutates state must restore that state between trials (Triton contributors, 2026). Any such policy must choose depth from committed-prefix information when sampling (Appendix B.5 of the paper).

6 The stock engine under speculation

Appendix D.4 of the paper locates where stock configurations that ought to agree first produce different bits. This section holds the parts of that record the paper summarizes, and how SGLang keeps the recurrent state under speculation with the tests of that path (H5); no claim of the paper depends on them.

6.1 The divergence record in detail

Appendix D.4 of the paper gives where stock configurations that ought to agree first produce different bits, and how often they diverge. The paragraphs below hold the parts of that record it summarizes in a sentence: the pools of the tapped servers, the five prompts whose outputs changed under the tap, the configuration switches of MTP with fixed pools, how a difference reaches the token, the cap-8 runs in detail, history dependence through the prefix cache and the recurrent prefill, and the open questions. They are moved from the paper unchanged.

Pools of the tapped servers With the tap version behind Figure 6 of the paper, tapped runs match untapped ones in tokens and log-probabilities for 162 of 167 prompts [N-E22], and with the cache-hashing version for 35 of 40, the other five being the same five prompts [N-E25]. The tapped and untapped servers allocated different pools, 159,322 (159,324 for the cache-hashing version) against 97,672 attention-cache tokens and 199 against 122 recurrent-state slots, so the five exceptions are not attributed to the tap itself [N-E26]; they are discussed below. For plain decoding against MTP with three steps at batch 1: the two tapped servers of the 167-prompt analysis had different pools (159,322 against 166,522 attention-cache tokens and 199 against 97 recurrent-state slots; 159,324 against 166,481 tokens and 199 against 97 slots for the cache-hashing rerun), and MTP’s draft layer attends over its own cached prefix, so a different copy of a shared prefix could change the proposals, the accepted lengths and the later verify forwards without any target attention output differing first. For plain decoding at concurrency 1 against 32: these two tapped servers also had different pools (159,322 against 89,652 attention-cache tokens and 199 against 111 recurrent-state slots). For the fresh-server repeat at concurrency 32 of the first matrix: the two sessions differed in request arrival, and with it batch composition, and also in their pools: 97,672 against 133,885 attention-cache tokens and 122 against 167 recurrent-state slots [N-E26].

The five prompts that changed under the tap Five prompts differ between the tapped and untapped runs, whose servers had different pools [N-E24,N-E26]. In all five the tapped run first differs from the untapped one in log-probabilities at output index 2, and three of them change tokens, at near ties: at the first changed token the top-two gap is 0 or 0.125 in both runs (tap_check.json). Untapped runs record no module outputs, so the module-level signature comes from two tapped sessions with different tap versions, which served the same 167 prompts in the same order at batch 1 and also differ for four of the five, in plain decoding and with MTP at three steps (tap_signature.json). Their pools differ as well, 93,742 against 159,322 attention-cache tokens and 118 against 199 recurrent-state slots for plain decoding, and 84,003 against 166,522 tokens and 55 against 97 slots for MTP; the attention cache never filled in either plain session. In all eight comparisons the first differing module output is layer 3’s attention, at output index 2 for plain decoding and at a verify step at output index 3 or 5 for MTP, while its QKV projection and every earlier module output that both tap versions hash agree, which at the same batch shape points to different cached KV. The comparison leaves out the GDN convolution, recurrence and attention outputs, the gated norm and the logits, which the two versions do not hash comparably, so one of those could differ first. In the paper’s history test, which serves each of 12 prompts on an empty cache and right after the earlier prompt that shares the longest prefix with it, two prompts change from output index 2 and the cache-hashing tap confirms the repoint for both (Appendix D.4 of the paper). The other four tap-changed prompts do not change in this test, under plain decoding or MTP. For them attention over a different copy of the cached KV is still the likely mechanism, because of the signature at the same batch shape, but what decides which copy a decode step reads in the two tapped sessions is unknown: the sessions differ in tap version, which changes the host time per forward, in how many tokens the earlier requests generated, and in their pools. One untested candidate is ordering: the repoint is a write issued from the host while the next decode step is already queued, so the step from which the switch shows could depend on timing. The cache-level check of these four prompts was not run.

Configuration switches with fixed pools MTP with three steps also ran with the prefix cache off, the overlap scheduler off, deterministic inference or the FP32 head, each at concurrency 1 and 32 on one server with the fixed pools [N-E27]. Against the default configuration at concurrency 1, turning the prefix cache off changes the tokens of 98 of the 320 prompts (1.73 divergences per 1,000) and turning the overlap scheduler off those of 161 (3.40); why both change the prefill is in Appendix D.4 of the paper. With deterministic inference MTP still diverges between concurrency 1 and 32, in 168 prompts (3.48 per 1,000, against 3.25 without it), all at exact ties or within one BF16 spacing, so on all 320 prompts deterministic mode does not make MTP batch-invariant, as the paper’s 8-prompt check found. With the FP32 head it diverges there in 130 prompts (2.44 per 1,000); the larger of the two margins is at most 0.19 nats, so the remaining divergences are order flips between near-equal FP32 logits, and the FP32 head removes BF16 ties, not divergences. In the BF16 pairs every

divergence is an exact tie, within one spacing, or one of eight near ties on three prompts, where one run’s margin is at most 0.375 nats and the other’s at most 0.125; no run committed a token that was not its own top-1. The same switches for plain decoding change it as often as MTP: at concurrency 1 the prefix cache off changes 96 of 320 plain outputs (98 for MTP) and the overlap scheduler off 166 (161), so neither effect is specific to speculation [N-E27]. Without the prefix cache plain decoding still diverges between concurrency 1 and 32 in 33 of 320 prompts (0.50 per 1,000, against 0.63 with it), so its batch dependence does not need the prefix cache, and MTP against plain decoding without it diverges in 177 and 178 prompts (3.95 and 3.90 per 1,000), as with it (171, 181). Deterministic inference makes plain decoding batch-invariant on all 320 prompts (concurrency 1 and 32 bitwise identical in tokens and top-five log-probabilities), while deterministic MTP departs from deterministic plain decoding in 175 and 168 prompts, first at the first verify forward in 315; the batch-invariant kernels remove plain decoding’s batch dependence, but the decode and verify recurrent kernels still differ. With the FP32 head plain decoding diverges between concurrency 1 and 32 in 24 prompts (0.36 per 1,000, margins at most 0.17 nats), and MTP against plain decoding in 136 and 138. Overlap off and prefix cache off agree on the first output token for all 320 prompts and then part at output index 1 in 176 plain-decoding prompts, at its first decode step, so that later difference does not need the speculative path. A second MTP session on a fresh server reproduces the first bitwise at concurrency 1 and 32. No divergence in any of these pairs is large, and no run committed a token that was not its own top-1.

How the difference reaches the token At the position where the runs first choose differently, with competing tokens a and b , the analysis compares in each run the order of the two tokens’ FP32 accumulator values in the head GEMM, before BF16 rounding. Where a run’s BF16 logits differ, rounding is monotone and gives that order. Where they are tied, the float64 dot product of that run’s head input with the two tokens’ weight rows stands in for the accumulator, but the two can differ by up to $\gamma(A_a + A_b)$, with A_t the sum of absolute products of token t ’s dot product. A *rounding flip* keeps the accumulator order in both runs, but in one run BF16 rounding merges the two into an exact tie that the index rule resolves against it; an *order flip* reverses the accumulator order between the runs; an *ambiguous* case has a tied run whose float64 gap lies inside the accumulation bound, so its accumulator order cannot be read from the data. The split therefore depends on the error model assumed for the stock accumulator (Table 10). Where both the BF16 order and a float64 gap outside the bound are available, they never disagree under either model. Under the conservative model most divergences are ambiguous, 93 of 129 and 22 of 40; the stock kernel’s possible accumulation error is as large as the gaps that decide these tokens, which is the same fact that makes the gap condition of Theorem 2.3 of the paper fail at near ties. Under the Hopper model most plain-versus-MTP divergences are rounding flips (89 of 129), and the concurrency-1-versus-32 divergences divide between order flips (17) and rounding flips (14). Whatever the model, a difference changes the chosen token only where the two leading logits are close: in the concurrency comparison, 151 of its 167 divergences are at an exact BF16 tie, 14 within one BF16 step, and none has a margin above 0.375 nats [N-E23].

Table 10. How the first token divergence arises, under the two named models of the stock kernel’s FP32 accumulation error: the conservative model ($\gamma = 6.11 \times 10^{-4}$), which covers any reduction order, and the Hopper model ($\gamma = 1.193 \times 10^{-4}$), Khattak and Mikaitis’s blocked model of Hopper’s BF16 tensor cores, published from measurements of the warp-level `mma` path, with a split- K allowance (Khattak & Mikaitis, 2026); on this GPU, crafted-input probes of the stock head kernels, of which the two inspected issue `wgmma`, show the same accumulation, a measurement on one GPU rather than a vendor contract [N-E7]. The tie-rule and head-GEMM columns count divergences caused by the `argmax`’s tie rule on bitwise-equal logits and by the head GEMM on bitwise-equal inputs; they do not depend on the model. The plain-versus-MTP prompts are the 167 that diverged in the concurrency comparison, generated to two tokens past their known divergence; the 40 concurrency prompts are the 16 whose divergence was not an exact tie and 24 of the 151 that were, drawn at random. Measured [N-E22].

Pair	Diverged	Caused by		Error model	Class at the divergence		
		tie rule	head GEMM		rounding	order	ambiguous
Plain vs MTP, batch 1	129/167	0	0	conservative model	29	7	93
				Hopper model	89	16	24
Plain, concurrency 1 vs 32	40/40	0	0	conservative model	3	15	22
				Hopper model	14	17	9

The cap-8 runs in detail The fresh-server repeat at concurrency 32 drops from 24 diverging prompts to 4 (it reproduces 316 of 320, and all 320 at concurrency 1); the drop comes with both identical pools and the smaller cap, which also narrows the batch compositions, so it is not attributed to the pools alone. With decode batches of at most 8 instead of 16, the floor of 0.63 per 1,000 is not comparable with the 3.42 of the cap-16 runs. A same-server repeat no longer reproduces every token, where the cap-16 runs’ larger pool did. No prefill in any pass reused a cached prefix, and one pass computes 126,408 tokens of prompt and output, more than the 49,152-token pool, so the prefix cache evicts during the first pass. The working

explanation, not checked at the level of the cache, is the history dependence below: the second pass’s requests read different earlier copies of shared prefixes, or run a recurrent prefill that depends on what the prefix cache holds, as one prompt’s same-server repeats do. With the prefix cache on, a same-server repeat is therefore not an equality reference. The pool regime alone leaves batch-1 tokens unchanged: the cap-8 and cap-16 plain runs at concurrency 1 agree on all 320 outputs, and one prompt’s log-probabilities differ, from output index 2 and by at most 0.24 nats.

History dependence through the prefix cache At the pinned commit, after a request’s prefill, SGLang inserts its prefix into the prefix cache, matches it again and repoints the request’s KV slots to the tree’s copies, freeing its own. For tokens the tree already held, such as a chat-template prefix shared with an earlier request, the request then attends over the earlier request’s copy of that KV, valid values for the same tokens at the same positions but computed in a prefill of another shape and so not necessarily bitwise equal; under the overlap scheduler decode step 1 is already in flight with the request’s own KV, so the switch shows from output index 2 (code reading at the pinned commit). No prefill in the tapped sessions or in the test below reused a cached prefix, and by the code this hybrid cache reuses a prefix only at a stored recurrent state, which these short shared prefixes did not have, so the repoint is the only way one request’s KV reaches another.

A deterministic test on stock SGLang serves each of 12 prompts on an empty cache and again right after the earlier prompt that shares the longest prefix with it [N-E28]; the 12 include the five prompts whose outputs changed between tapped and untapped runs (research notes). With the prefix cache on, plain decoding and MTP with three steps keep every token but change log-probabilities for 2 of the 12 prompts, `mt_bench-0056` and `humaneval-0008`, from output index 2 (and 5 for one MTP prompt); with the prefix cache off all 12 are identical in tokens and log-probabilities, for plain decoding and for MTP with three steps. So with the prefix cache on, a request’s output at a fixed configuration and batch shape can depend on which earlier request computed its shared prefix; that is shown for these two prompts. The cache-hashing tap confirms the repoint for both at the level of the cache [N-E25]. Each prompt was served alone and right after its predecessor, on two fresh servers whose pools differ by at most 30 attention-cache tokens. In both, the first differing module output is layer 3’s attention at output index 2; the caches entering decode step 1 are identical, and the keys and values of all 8 attention layers entering step 2 differ at cached positions 3 to 5 for `mt_bench-0056`, whose shared prefix is 6 tokens, and 3 to 21 for `humaneval-0008`, with 22 shared tokens, while positions 0 to 2 are bitwise equal in both copies. The values involved are valid by the code path, no token changed in the history test, and where the tap changed tokens (research notes) it changed only near-tie decisions; we do not consider it state corruption.

History dependence through the recurrent prefill The cache-hashing tap also found a second route, separate from the repoint [N-E25]. `humaneval-0044`, the 128-token prompt whose log-probabilities differed between same-server repeats in the cap-16 runs, was served five times on one server. With the prefix cache flushed between requests, all 4 repeats are bitwise identical to the first. Without a flush, the second request again prefills all 128 tokens, and its module outputs differ from prompt position 1 on, starting at layer 0’s GDN recurrence, with different log-probabilities but the same tokens; the third to fifth reuse a 64-token cached prefix, prefill the other 64, differ from position 64 on and change tokens. That prompt’s recurrent prefill therefore depends on what the prefix cache already holds, and the difference shows from the first output token. A related comparison, plain decoding at batch 1 with the prefix cache on and off on 40 prompts, on two servers with different pools (159,324 against 447,027 attention-cache tokens and 199 against 16 recurrent-state slots) [N-E26], is identical throughout for the 34 prompts of at most 63 tokens. The 6 prompts longer than one 64-token chunk of the recurrent prefill differ from prompt position 1, at layer 0’s GDN recurrence, before any cache is read, and their log-probabilities differ from output index 0; two of them change a token, each at an exact BF16 tie in one run. This is not a question of which copy of the KV a request reads, but it means that a prefix-cache-off control can differ from the prefix-cache-on run from the first output token, as all 6 such prompts tested did. With fixed pools, MTP with three steps at concurrency 1 shows the same on all 320 prompts [N-E27]. Turning the prefix cache off changes the top five log-probabilities of the first output token, which come from the prefill alone, in 192 of the 193 prompts longer than 64 tokens and in none of the 127 shorter ones; of the 198 prompts whose outputs differ at all, 192 differ from that token. At the pinned commit, turning the overlap scheduler off also switches the recurrent-state strategy of the prefix cache from `extra_buffer` to `no_buffer`, so it changes two things at once, and it changes the first output token of the same prompts: the overlap-off and prefix-cache-off runs agree at output index 0 on all 320. So the prefill of a prompt longer than one 64-token chunk depends on that strategy, and `extra_buffer` computes it differently from `no_buffer` and from the path without the prefix cache; that the cause is `extra_buffer`’s checkpoint tracking in the prefill kernel is code reading, not verified. The overlap-off and prefix-cache-off runs part later, in 177 prompts at output index 1, the first verify forward, and in 7 further on; the overlap-off server runs SGLang’s synchronous speculative path, and which kernel differs there has not been located.

Not run The log-probabilities-off control, retraction at concurrency 32 and the FlashInfer and ReplaySSM decode paths of the recurrent state were not tested.

6.2 Recurrent state under speculation

Qwen’s hybrid architecture makes state correctness part of verification. Reusable state includes attention KV, recurrent tensors, convolution history, positions and backend metadata. With state orientation $S \in \mathbb{R}^{d_v \times d_k}$, the gated-delta update can be written

$$S_t = \alpha_t S_{t-1} + u_t k_t^\top, \quad u_t = \beta_t (v_t - \alpha_t S_{t-1} k_t), \quad (9)$$

which is the Gated DeltaNet rule $S_t = S_{t-1} \alpha_t (I - \beta_t k_t k_t^\top) + \beta_t v_t k_t^\top$ (Yang et al., 2025; 2024) with the update vector made explicit. The update depends on the preceding state.

Proposition 6.1 (Replay of valid derived records). *If (α_t, u_t, k_t) were recorded along a causal trajectory from S_0 , applying the first a records to that same S_0 reproduces S_a in exact arithmetic:*

$$S_a = \left(\prod_{j=1}^a \alpha_j \right) S_0 + \sum_{t=1}^a \left(\prod_{j=t+1}^a \alpha_j \right) u_t k_t^\top.$$

Proof. Substitute the expansion at $a - 1$ into (9); the new gate multiplies every preceding term and the new outer product adds the last summand. Induction completes the proof. $\square$

The qualification about S_0 matters: replaying a derived u_t from a different state gives the wrong result even when the token IDs agree. The exact rational reference contains a one-dimensional witness, a record made at state zero and replayed from state two, which gives three where the genuine update gives one [N-E18]. Replying raw inputs with the true transition is a different operation. Regrouping the recurrence also does not preserve floating-point results (Appendix D.1 of the paper); SGLang’s replay fold constrains its operation order and warp count so that it is a bitwise clone of the verify kernel’s state update (SGLang contributors, 2026a). Its `accept_lens` include the bonus token in that backend’s indexing (code reading of `python/sglang/kernels/ops/attention/fla/gdn_replayssm_spec_fold.py` at the pinned commit) (SGLang contributors, 2026a).

A cycle may emit a correction or bonus token before that token has been consumed into every cache, so the runtime must distinguish the *emitted* prefix from the *materialized* state prefix; treating both as one accepted length invites off-by-one state errors. Publication of speculative state should check that the request’s slot and epoch are still current, and memory must not be reused while any stream or captured graph can still write it. An epoch check does not by itself prevent a stale kernel from writing reused memory; ownership and delayed reuse are separate obligations.

What SGLang does, and how it is tested (H5) During verification SGLang computes the recurrent state and the convolution window after every draft position into per-position buffers and leaves the live state untouched; after acceptance a fused scatter copies the buffers of the last accepted position (for a tree, the last accepted node) into the request’s slot (code reading at the pinned commit). The emitted prefix includes the bonus token while the materialized state does not; the bonus token is the first input of the next cycle. Prefix reuse checkpoints the recurrent state every 256 tokens of sequence length into a ping-pong pair of slots, including checkpoints that fall inside a verify cycle, and a freshly allocated slot is cleared by the next forward on the same stream, after any in-flight verification by its previous owner. The MTP draft layer is a full-attention layer, and under greedy verification every committed token is the target’s argmax, so an error in draft-side state could lower acceptance but not change the output.

Each point where a speculative cycle touches this state is tested against a reference that isolates it. Truncation by `max_new_tokens` and stop tokens at every index inside a verify cycle must reproduce the untruncated run’s tokens on the same server. Checkpoints taken during speculative decoding, including in the cycle that finished the request, are restored for a continuation and compared with a cold prefill of the same prefix. Requests aborted mid-stream with a recurrent pool of exactly four slots force the next request onto the freed slot while the aborted request’s last verification may still be in flight. Chunked prefill at 200 and 256 tokens is compared with an unchunked prefill. And the divergences of every MTP run from plain decoding are bucketed by the previous cycle’s commit length, which fixes the draft position that cycle rejected. The first two tests are complete [N-E28], for MTP with three and five draft steps, top- k 1, the prefix cache, overlap scheduler and CUDA graphs on, one request in flight and the first 40 prompts. Both tests compare output token IDs with the reference prefix; log-probabilities and state are not compared. Truncation leaves every truncated run token-identical to the untruncated run’s prefix: 157 of 157 with three steps, keeping 1 to 4 tokens of the final cycle, and 240 of 240 with five steps, keeping 1 to 6. A stop token at every index of a verify cycle also leaves the output token-identical to the untruncated prefix, in 160 of 160 and 240 of 240 cases. In 120 and 200 of them the stop falls inside the draft block, so drafts after it were accepted and folded into the recurrent state before the stop was detected, and none of it changed an output token. Extending each stopped conversation with a new user turn, served warm from the restored checkpoint and cold after a flush, gives token-identical continuations in 139 of 160 and 210 of 240 cases; the rest diverge at exact ties (19 and 30) or within one BF16 step (2, both with three steps), consistent with the warm path’s different prefill computation and with the history dependence above.

The same two tests for the tree, with three steps and top- k 2, and for plain decoding also compare the top five log-probabilities [N-E28]. Truncation leaves 159 of 159 tree runs and 39 of 39 plain runs identical in tokens and log-probabilities to the untruncated prefix, and a stop token at every index of a cycle leaves 160 of 160 and 40 of 40 identical up to the stop, 120 of the tree cases with the stop inside the draft block. Warm against cold continuation gives 142 of 160 identical for the tree (17 exact ties, 1 within one spacing) and 33 of 40 for plain decoding (7 exact ties); plain decoding has no speculative state, so such flips occur without it, consistent with the warm path’s prefill and the prefix cache’s history.

The bucketing by rejection position uses the pinned matrix at concurrency 1 against plain decoding [N-E27]. At fragile positions, where plain decoding’s top-two gap is at most 0.25 nats, the divergences per fragile position after each commit length of the previous cycle, full acceptance included, lie between 0.061 and 0.098 for MTP with one, three and five steps and the tree, and no length stands out. The rates are compatible with one common rate, with chi-square $p = 0.75, 0.72, 0.54$ and 0.38 ; that is a failure to reject, not a proof of equal rates, and the test treats positions as independent although they cluster by prompt and a prompt’s positions stop at its first divergence. No prompt-clustered analysis or bound on a per-length excess rate has been done. No divergence in any of the four MTP configurations falls at a non-fragile position, where a state error would show most plainly. The first verify cycle after the prefill follows no earlier cycle and is counted apart. Its rate is higher for five steps and the tree, 7 of 40 and 6 of 33 fragile positions against 0.077 and 0.081 per fragile position later, and a Fisher exact test against all later cycles gives $p = 0.033$ and 0.049 (0.71 and 0.75 for one and three steps). That test is exploratory: it was chosen after looking at the data, it is not corrected for the four configurations and several cycle buckets examined, and the groups differ in more than state, since the first cycle follows the prefill chunk and every prompt contributes it, while later cycles count only prompts that have not yet diverged. It is a lead, not a finding. A test of it is declared before any of its data exist (experiments/state_safety/README.md, “Declared follow-up”): on 960 fresh prompts disjoint from the 320, with frozen token IDs, it compares the first cycle with later ones for five steps and the tree against plain decoding at concurrency 1, and uses the same configurations at concurrency 1 against 32, which share the early exit of high-hazard prompts but do not compare verification with plain decoding, as a selection control. A first-cycle excess specific to speculation counts as supported only if the one-sided 95% bootstrap lower bounds of both the pooled primary log odds ratio and its excess over the control’s are above 0; otherwise the result is reported as inconclusive, not as evidence of no effect. Runs that fail the declared provenance, completeness or configuration checks are void, as is the analysis if every prompt is excluded, and a void result reports no statistic. The declared runs passed every void check, with both holds attested clean at the declaration commit and all 960 prompts included, and the test is inconclusive [N-E30]. In the pooled primary pairs the first verify cycle diverged at 8 of 162 fragile positions (4.9%) and later cycles at 842 of 11,252 (7.5%), an odds ratio of 0.68 (+0.5 per cell); the one-sided 95% bootstrap lower bounds are -1.437 for the primary log odds ratio and -0.865 for its excess over the control, so neither criterion holds, and the one-sided Fisher exact test gives $p = 0.92$. Under the declared rule this is not evidence of no effect, and an effect that also differs between concurrency 1 and 32 would cancel in the control. Descriptively, and post hoc (an amendment written after the result was seen, which leaves the declared statistics unchanged), the data are inconsistent with a pooled primary odds ratio above 1.28 (one-sided 95%, the 95th percentile of the same bootstrap replicates), below the exploratory odds ratio of 2.36 that the declaration’s power calculation assumed. The runs gave 162 first-cycle fragile positions where the declaration expected about 219, so the test had somewhat less power than planned at that effect size. The exploratory excess, 13 of 73 fragile positions for these two configurations in the cap-8 runs, did not recur. The check of the recurrent state handed from the prefill to the first verify forward, which was to follow a supported result, is not run. For three steps at batch 1 the cache-hashing tap already finds the caches entering the first verify forward, recurrent state included, identical to those entering plain decoding’s first decode step, on 40 prompts (Appendix D.4 of the paper); five steps and the tree were not checked. Checkpoint reuse was tested with the prefix cache on [N-E28]. Twelve thinking-mode prompts for MTP with three steps and 8 each for the tree and plain decoding generate 700 tokens, crossing the 256-token checkpoint interval during decode, and prefixes that end at a boundary or 1, 3 or 37 tokens past it are served for 48 tokens warm, after a flush and a regeneration of the original request, and cold, after a flush; tokens are compared, not log-probabilities. A finished request kept only its latest checkpoint, so a warm request restored a decode checkpoint only when its prefix extended past the last boundary the original crossed: in 9 of 100 cases for MTP, 6 of 68 for the tree and 6 of 68 for plain decoding. Where it did, warm and cold gave the same tokens in 9 of 9, 6 of 6 and 4 of 6 cases, the other two at exact ties. A checkpoint taken in the verify cycle that finished the request was restored in every case, and warm and cold agreed in 45 of 53 cases for MTP and 19 of 21 for the tree, the rest at exact ties. Warm and cold reach the boundary state by different computations, decode or verify forwards against a chunked prefill, so bitwise equality is not expected; every difference is an exact tie or within one BF16 step, and plain decoding, which has no speculative state, shows them too, in 9 of 68 cases against 21 of 153 for MTP and 8 of 89 for the tree. No difference points to a wrong restored state, but few decode checkpoints were restored. Aborts were tested on a server whose batch and recurrent-state pool hold four requests: four lanes each abort a long request mid-stream and then serve a probe, compared in tokens with the same probe served alone [N-E29]. Deterministic plain decoding keeps 40 of 40 probes token-identical; MTP with three steps keeps 28 of 40 with the prefix cache off and on, the rest at exact ties or one BF16 step, which the batch shape (and, with the prefix cache, the request history) can give alone. The lanes are not synchronized and slot assignments are not recorded, so the test does not show that a probe reused an aborted request’s slot. Run-to-run repeats are bitwise identical in tokens and top-five log-probabilities for 176 of 176 repeat pairs, for plain decoding and MTP alike (44 prompts,

five times each at batch 1 with the cache flushed). Chunked prefill at 256 and 200 tokens changes generation only at ties or one step (20 and 21 of 40 prompts diverge, margins at most 0.25 nats), but a few prompt positions' log-probabilities move by up to 3.53 and 5.22 nats, 94 and 90 of 33,853 by more than 1 nat, 13 and 1 of them inside the first chunk, where no state passes between chunks. An exploratory check, added after these drifts were seen, finds no alignment artefact. Their cause is not known, and they are not read as a state error.

7 Further analysis

These results were developed alongside the certified head and transport, and the paper's claims do not rely on them. The first two subsections take the decision beyond the head (P1, rejected); the third compiles the head through the last FFN for a static screen (P12, negative at its screen); the next four concern sampled decisions (the analysis of sampling that the paper uses is in its Appendix B); the last two concern transport's summaries and the conversion of skipped work into time.

7.1 The directional margin

When the information is about the hidden state rather than the logits, a box of logit bounds is loose; the certificate below uses the direction of each competitor instead.

Proposition 7.1 (Directional margin). *Let the logits be $z = Wh$ in real arithmetic and suppose $\|h - \tilde{h}\|_2 \leq \varepsilon_h$ for a known $\tilde{h}$. Then c is the strict argmax of Wh for every such h if and only if*

$$(w_c - w_j)^\top \tilde{h} > \|w_c - w_j\|_2 \varepsilon_h \quad \text{for all } j \neq c.$$

This condition is implied by, and can be much weaker than, the box condition obtained from the row bounds $|z_j - w_j^\top \tilde{h}| \leq \|w_j\|_2 \varepsilon_h$.

Proof. For fixed j , $\min_{\|e\|_2 \leq \varepsilon_h} (w_c - w_j)^\top (\tilde{h} + e) = (w_c - w_j)^\top \tilde{h} - \|w_c - w_j\|_2 \varepsilon_h$ by Cauchy–Schwarz, with equality at $e = -\varepsilon_h (w_c - w_j) / \|w_c - w_j\|_2$. So $z_c > z_j$ throughout the ball exactly when the stated inequality holds. The box condition reads $(w_c - w_j)^\top \tilde{h} > (\|w_c\|_2 + \|w_j\|_2) \varepsilon_h$, and $\|w_c - w_j\|_2 \leq \|w_c\|_2 + \|w_j\|_2$. $\square$

The directional certificate is not free: it needs $\|w_c - w_j\|_2$ for every competitor, which for all j costs as much as a head pass. It is useful after a box screen has reduced the competitors to a few rows, and for the stock contract the gap condition of Theorem 2.3 of the paper applies on top of it, as it does for the ladder. It is also a statement in real arithmetic; the floating-point terms of Appendix A.3 of the paper turn it into an enclosure of computed values.

7.2 Beyond the head: a certified decoder tail (P1, rejected)

Skipping a computation removes its cost only if nothing downstream needs its result, so every skipped result falls into one of three classes. It is *dead* when it has no remaining consumer. It is *deferred* when a later consumer needs it and a reconstruction path exists; the cost is moved, not removed, and must be charged where it lands. It is *soundly enclosed* when a certificate bounds it well enough for every consumer that remains. A claimed saving should name its class, because early exits and approximations can borrow time from future tokens.

The head is dead work in this sense: once a token is committed, no future computation reads the logits. In target-only decoding of Qwen3.5-4B the same is true of more. The last decoder layer is a full-attention layer; its attention writes the key–value cache before its MLP runs, and the final MLP, the final norm and the head have no consumer in future state. A certificate that decided the token before that tail could therefore remove about 1.41 GB of the 8.41 GB streamed per step (the head and the last layer's MLP, Table 19 of the paper), permanently. The cut does not transfer to speculation without re-analysis: the MTP layer and the DFlash drafter consume target hidden states, so work that is dead in plain decoding is deferred there.

The hypothesis (P1) was that a cheap surrogate of that tail with a sound enclosure of the head input could let the commit rule decide the token before the tail runs. A one-shot surrogate that costs C (surrogate plus certificate) and certifies a fraction p of positions pays only if $C < p T_{\text{tail}}$, and the ceiling for a tail that is a fraction f of the step is $1/(1 - f)$. The directional margin is the natural certificate when the surrogate predicts the head input with a radius. The kill test was deliberately optimistic: record the residual stream at the cut, the reference head input, the logits and the tokens; freeze an int8 surrogate of the tail; take the oracle radius $\|h - \tilde{h}\|_2$; and apply the directional certificate with no overhead. If the oracle certification rate satisfied $p_{\text{oracle}} T_{\text{tail}} \leq C_{\text{surrogate}}$, the design was to be dropped; otherwise real bounds would be compared with the dense tail and with the certified head alone.

The kill test rejects the design [N-E4]. It used 2,064 positions from 16 held-out prompts (four per domain), with the head input taken from a BF16 forward pass of a reference implementation on the CPU, teacher-forced on the engine’s greedy outputs; that head input differs from the engine’s captured one by a median 1.8% in relative ℓ_2 norm, and its argmax equals the engine’s token at 99.5% of positions. An int8 per-row copy of the final MLP moves the head input by a median 0.94%. With that realized error as an oracle radius and no overhead, the directional certificate decides 85.4% of positions when the head is exact and 71.8% when the head is int8 as well, while the int8 head alone, with the exact MLP, decides 79.3% without any re-scoring. The MLP is 0.142 GB of the tail’s 1.413 GB, so an int8 surrogate can save at most 0.071 GB per token, and it lowers the rate at which the certified head decides on its own. Certifying the head alone is the part worth building.

7.3 Compiling the head through the last FFN (P12, negative at its screen)

P12 moves the head screen one layer earlier. Let x be the residual stream after the last decoder layer’s attention, $a = \text{SiLU}(W_{gu}) \odot (W_{uu})$ that layer’s FFN activation on $u = \text{RMSNorm}(x)$, and $y = x + W_d a$ its output. The tied head behind the final RMSNorm scores $z_v = w_v^\top \Gamma y / r(y)$, with $\Gamma = \text{diag}(1 + g)$ for the final norm’s weight g and $r(y) > 0$ common to every v , so the greedy winner is the argmax of $t_v^\top q$ over the compiled rows $t_v = [\Gamma w_v; W_d^\top \Gamma w_v]$ with the query $q = [x; a]$. A centre-plus-radius tile bound, $\mu_C^\top q + R_C \|q\|_2$ with μ_C the mean row of tile C and R_C the largest distance of a row from it, lets a tile be skipped when the bound falls below the winner’s score. The test measures the most favourable version of that screen: the winner’s exact score is taken as known, so its skip rate bounds from above what any screen with this bound can skip [N-E5]. It ran the Hugging Face target (pinned revision, BF16 weights, FP32 arithmetic for the dictionary and the scores) on 1,000 queries, 25 positions from each of 40 outputs of the drafter’s block-16 panel, with 64-row tiles in token-id order and again after sorting the rows by a random projection, a cheap clustering control rather than k -means.

The dictionary has 248,320 rows of 11,776 coefficients, 4.6 times the head’s 2,560, and its argmax equals the model’s at 99.7% of the queries. Even with the winner’s score known, the screen skips the same tiles for every query: 0.077% of the rows (192) in token-id order and 0.052% (128) after the random-projection sort. That is the share the static screen skips on the head alone with contiguous 64-row tiles (0.08%, Table 11 of the paper), while every remaining row costs 4.6 times a head row, so the screen does not prune the compiled dictionary. The obstruction is the bound, since the compiled argmax is right: a tile is skipped only when its radius times $\|q\|_2$ is below the winner’s lead over the tile centre, and the winner’s lead over the runner-up is a median $0.125\|q\|_2$. Tighter tilings (k -means, smaller tiles) and per-row bounds were not tested on the dictionary.

7.4 Sampling before the scores are known

A race needs the winner’s interval to separate from every other. A different construction can emit a token with some probability before any candidate is separated, and finish the rest of the law after more computation. Let $\mathcal{P}(I)$ be the set of target laws compatible with the information I held so far.

Proposition 7.2 (Common mass). *Consider procedures that, before acquiring more information, emit token i with probability r_i (the same for every $p \in \mathcal{P}(I)$) and otherwise continue until p is known. Such a procedure outputs exactly p for every $p \in \mathcal{P}(I)$ if and only if $r_i \leq \inf_{p \in \mathcal{P}(I)} p_i$ for every i , and then it may finish by sampling the residual $(p - r)/(1 - \alpha)$ with $\alpha = \sum_i r_i$. The largest immediate probability is*

$$\alpha^*(I) = \sum_i \inf_{p \in \mathcal{P}(I)} p_i.$$

Proof. If the procedure is exact, the final probability of i is at least the probability r_i of emitting it immediately, for every compatible p , so $r_i \leq \inf_p p_i$. Conversely, if $r_i \leq \inf_p p_i$ then $p - r \geq 0$ and sums to $1 - \alpha$ for every compatible p , so the residual is a law, and the output law is $r + (1 - \alpha) \cdot (p - r)/(1 - \alpha) = p$. $\square$

For two candidate laws $\mathcal{P} = \{p, q\}$, $\alpha^* = \sum_i \min(p_i, q_i) = 1 - \text{TV}(p, q)$, the overlap of a maximal coupling, which is exactly the acceptance probability of speculative sampling (Leviathan et al., 2023; Chen et al., 2023). The proposition says that the same idea applies to any set of laws the evidence leaves open, and that nothing can do better without more information.

A two-token example shows that exact sampling can finish where greedy selection cannot. Suppose the evidence says only that the probability of token 1 lies in $[0.49, 0.51]$. Then $\alpha^* = 0.49 + 0.49 = 0.98$. Draw U uniform on $[0, 1]$: emit token 1 if $U < 0.49$, token 0 if $U \geq 0.51$, and otherwise compute p and emit token 1 exactly when $U < p$, reusing the same U . The output has law p , and 98% of draws never need the scores, whereas the greedy decision is undecided because the interval contains $1/2$. For two tokens this procedure is inverse-CDF sampling with bounds on the CDF. In general it gives equality in distribution, not seeded equality with a dense race.

Proposition 7.3 (Bounded logit error). *Let $p = \text{softmax}(z/T)$ and $q = \text{softmax}(\tilde{z}/T)$, and suppose $\max_i(z_i - \tilde{z}_i) - \min_i(z_i - \tilde{z}_i) \leq \Delta$. Then $p_i \geq e^{-\Delta/T} q_i$ for every i . Consequently, with $\alpha = e^{-\Delta/T}$:*

1. *sampling q with probability α , and otherwise computing p and sampling $(p - \alpha q)/(1 - \alpha)$, samples p exactly;*
2. *always sampling q gives $\text{TV}(p, q) \leq 1 - \alpha$.*

Proof. Write $d_i = z_i - \tilde{z}_i$ with $m \leq d_i \leq M$ and $M - m \leq \Delta$. Then $p_i/q_i = e^{d_i/T} / \sum_j q_j e^{d_j/T} \geq e^{m/T} / e^{M/T} \geq e^{-\Delta/T}$. Part 1 is Proposition 7.2 with $r = \alpha q$. For part 2, $\text{TV}(p, q) = \sum_i (q_i - p_i)_+ \leq \sum_i (1 - \alpha) q_i = 1 - \alpha$. $\square$

With self-evidence the range is at most twice the largest envelope half-width, so $\alpha \geq e^{-2 \max_i \beta_i / T}$. The trade-off against the race differs: this sampler needs the cheap pass’s full softmax and, with probability $1 - \alpha$, the whole exact head, whereas the race re-scores a candidate set whose expected size grows like $e^{4\beta/T}$ (Proposition A.7 of the paper) and returns the seeded token.

7.5 An approximate mode, and why it cannot be certified harmless

Proposition 7.4 (Approximate mode: KL and sequence total variation). *Suppose that $\min_c \|z - \tilde{z} - c\mathbf{1}\|_\infty \leq \varepsilon$ at temperature one; at temperature T , replace ε by ε/T . Then $\text{KL}(p \parallel q) \leq \varepsilon^2/2$ and $\text{KL}(q \parallel p) \leq \varepsilon^2/2$, so $\text{TV}(p, q) \leq \varepsilon/2$. If an approximate decoder samples q_t in place of p_t at every position and the bound holds with ε_t at every reachable prefix, the laws P and Q of the first N tokens satisfy $\text{KL}(P \parallel Q) \leq \frac{1}{2} \sum_t \varepsilon_t^2$ and*

$$\text{TV}(P, Q) \leq \frac{1}{2} \left(\sum_{t=1}^N \varepsilon_t^2 \right)^{1/2}.$$

Proof. Let $d = z - \tilde{z} - c\mathbf{1}$ with $\|d\|_\infty \leq \varepsilon$; the shift c changes neither law. Then p is q tilted by d : $p_i = q_i e^{d_i} / \mathbb{E}_q[e^d]$. Let $\psi(\lambda) = \log \mathbb{E}_q[e^{\lambda d}]$, so $\psi(0) = 0$ and $\text{KL}(p \parallel q) = \mathbb{E}_p[d] - \psi(1) = \psi'(1) - \psi(1) = \int_0^1 \lambda \psi''(\lambda) d\lambda$. Here $\psi''(\lambda)$ is the variance of d under the tilted law $q^{(\lambda)}$, at most $(2\varepsilon)^2/4 = \varepsilon^2$ because d takes values in an interval of length 2ε . Hence $\text{KL}(p \parallel q) \leq \varepsilon^2/2$; the reverse divergence follows by exchanging p and q . Pinsker’s inequality gives $\text{TV} \leq \sqrt{\text{KL}/2} \leq \varepsilon/2$. For sequences, the chain rule gives $\text{KL}(P \parallel Q) = \sum_t \mathbb{E}_P[\text{KL}(p_t \parallel q_t)] \leq \sum_t \varepsilon_t^2/2$, and Pinsker’s inequality again gives the total-variation bound. $\square$

To guarantee a sequence total variation of at most 0.01 over $N = 1,024$ tokens with a uniform bound, the proposition needs a logit error, up to a common shift, of at most $\varepsilon = 6.25 \times 10^{-4}$, far tighter than any int8 envelope of Section 3 of the paper. An approximate head is therefore a quality trade-off to be measured (Section 3.7); these bounds cannot certify it as harmless over long outputs.

7.6 A demand frontier over a fixed protocol

For a chain of L proposals each head row is accepted, rejected or unresolved. The first certified rejection at position r proves that no row after r can affect the cycle’s output, while earlier unresolved rows still matter because one of them may be the true first rejection.

Proposition 7.5 (Safe suppression after a certified rejection). *If every issued certificate equals the decision of a fixed reference verifier on its row, the first true rejection is no later than the first certified rejection. Suppressing head work strictly after the latter does not change the reference output, provided completion and state commit use the actual first rejection.*

Proof. At the certified rejection the reference decision is rejection; an earlier unresolved row may also reject, and no later row can be the first rejection. All accepted outputs, the replacement token and the state prefix are determined by rows up to the first rejection. $\square$

The statement is for chains. SGLang’s trees share one coin across siblings and choose the next coin by the accepted node, so a tree frontier must follow the tree’s paths. The target backbone has already run over every admitted position, so the frontier saves head work, not transformer layers.

7.7 A cheap decision is not a cheap verification

A CPU experiment makes the point of Appendix B.3 of the paper concrete. It constructs two families of 128×8 integer heads with 16-row tiles and 36 paired fixtures each: one clusters rows around separated centres, the control uses unstructured rows. For each fixture $h^t = h^d + d \cdot v$ with drift scale $d \in \{0, 1, 2, 4\}$; proposals are sampled from the anchor head’s law, and all probability arithmetic is exact rational. The experiment counts distinct target rows projected, which is work, not time [N-E19].

At zero drift the transported summaries describe the target exactly and the acceptance decision needs only its candidate row, an identity check rather than a forecast. At nonzero drift the unstructured head defeats greedy screening. More subtly, large drift makes rejection easy to certify, which lowers the rows needed for the decision while destroying acceptance itself, and completing the residual then erases the saving (Figure 3). At drift four the clustered family decides with 11.4% of the rows but needs 96.5% once the dense residual is charged. The experiment does not test transport on trained heads (Section 5.3 of the paper does); it refutes the inference that a cheap acceptance decision means a cheap verification. Residual races answer it and remain unmeasured; fixed-noise verification avoids the residual altogether.

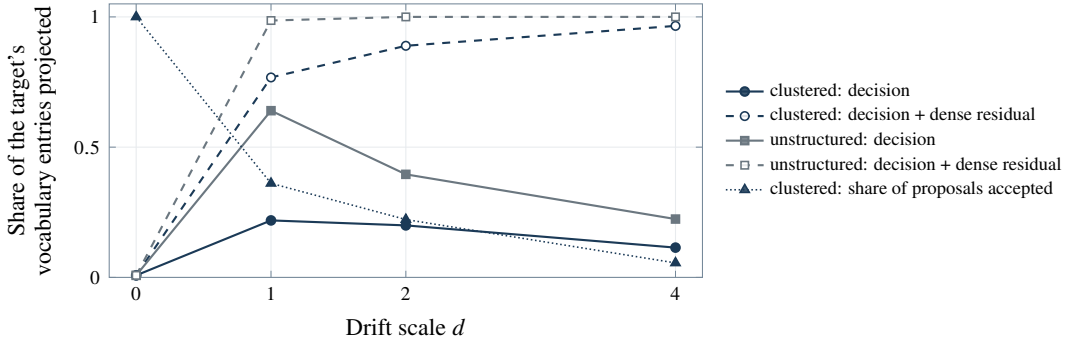


Figure 3. Work counts on constructed heads, not timings [N-E19]. Solid lines count the rows needed to decide acceptance; dashed lines add a dense residual completion after each rejection. The dotted line is the share of proposals accepted in the clustered family. Evidence production, bounds, bonus sampling and the backbone are excluded.

7.8 A summary that supports both selection and mass

For a set of labelled logits, keep its top k items under descending score and then ascending token index, together with the mass of every discarded item. Two such summaries merge by taking the top k of their retained union and adding the newly discarded mass to the two old tails.

Proposition 7.6 (Top- k plus tail is a compositional summary). *For disjoint finite domains the merge produces the same top- k items and tail mass as summarizing the union, and it is associative and commutative in exact arithmetic under the common total order.*

Proof. An item omitted from a local top- k has at least k locally superior items and cannot enter the union’s top- k , so every possible winner is retained. The original tails and the newly discarded items partition the final tail, so summing them gives its mass without subtracting nearly equal totals. Every parenthesization and order represents the same union. $\square$

This is a sufficient statistic built from familiar reductions, and floating-point tail masses are not associative. Its value is an interface: one epilogue can serve candidate generation, bounds and completion. The retained-tail mass bound that this interface supports is Equation (7) of the paper.

7.9 From skipped work to time

A skipped tile saves time only if the GPU avoids the work. For each mechanism the replay counts the weight bytes read (refined tiles including their union over the real batch, gathered candidate rows, the low-precision pass), the metadata and evidence bytes, the sequential stages and the fallbacks, and converts them to predicted time with the kernel’s measured primitive costs. The result would be a model prediction, not a timing, and it was not computed.

8 The exact rational reference

The second exact reference of Appendix A.10 of the paper, with bounded integer heads and base-2 masses represented as rationals, passes its 18 decision and 3 race test methods on the instances of Table 11 [N-E18].

Table 11. Instances checked by the exact rational reference for decisions and races [N-E18]. It also contains counterexamples for biased depth selection, graph-cost cliffs, batch unions, invalid numerical bounds, support truncation and stale namespaces. No GPU code is exercised.

Check	Instances	What agrees
Top- k plus tail composition	250	Exact union and associative merge (Proposition 7.6)
Transported tile bounds	300 heads, 1,048 tiles	3,787 weight rows enclosed (Theorem 5.1 of the paper, Hölder form)
Greedy certified screening	250	Same winner under a deterministic total order
Retained-tail tightening	565 tiles	Exact mass enclosed; bounds never looser (Equation (7) of the paper)
Acceptance guards	1,600	Certified bits agree with the complete mass (Theorem B.5 of the paper)
Unresolved-interval measure	160	Exact agreement with $P_?$ of Theorem B.5 of the paper
Lazy acceptance	500	With and without retained seeds; finite completion
Coupled one-step sampling	200	Same output with exact residual fallback
Rejection-correction identity	300 pairs	Includes zero-probability support entries
Demand frontier	1,092 partial states	5,460 complete decision assignments (Proposition 7.5)
Exclusion-aware race summaries	300	Exact maxima after support exclusions
Target priority race	250	Same token as a dense priority race
Sparse-support residual race	250	Bounded and dense races agree (Theorem B.6 of the paper)
Finite-map execution	400	Scan, walk and composition agree (Proposition 5.2)
Gated-delta replay	150 trajectories	868 accepted-prefix replay checks (Proposition 6.1)
Lease reuse	300 schedules	Sequential epoch and base-prefix guards

9 Evidence register

9.1 Evidence register

Every committed result these notes cite is listed below with the claims it supports, the files that hold it and the program that produced them. All paths are relative to the repository at <https://github.com/SamMausberg/verified-progress>. The exact command behind each evidence file is in the README.md of the file’s directory, and the files record the commits of this repository and of the engine that produced them where the generator records them. The text cites an entry as [N-En]. The paper numbers its own register (Appendix F.1 of the paper) separately, as En ; an entry that both documents cite lists the same files in both.

Entry	What it supports; files and the program that produced them
<i>The certified head on captured engine states</i>	
N-E1	Candidate sets, envelope checks, head-input norms, batch unions, FP32 re-scoring and cascades on 6,005 held-out plain-decode positions (the paper’s Section 4.2, Appendix A.8) evidence/head_geometry/selfevidence_plain4b.json; produced by experiments/head_geometry/analyze_selfevidence.py.
N-E2	Rows and batches left to the stock kernel by rule and error model; no certified token differed from the engine’s (the paper’s Table 5) evidence/head_geometry/rstock_plain4b.json; produced by experiments/head_geometry/analyze_rstock.py.
N-E3	Candidate sets and fallback shares on 4,010 MTP-4B verify rows and 4,024 draft rows (the paper’s Appendix A.8) evidence/head_geometry/selfevidence_mtp4b.json, selfevidence_mtp4b.csv, selfevidence_mtp4b_ccdf.csv; produced by experiments/head_geometry/analyze_selfevidence.py, export_candidate_ccdf.py.
N-E4	Kill test of the certified decoder tail, P1 (Section 7.2) evidence/head_geometry/tail_killtest.json; produced by experiments/head_geometry/tail_killtest.py.

Entry	What it supports; files and the program that produced them
N-E5	P12’s static screen on the head compiled through the last FFN, with the winner’s score known (an upper bound on the rows any screen with this bound skips), and the compiled argmax against the model’s (Section 7.3) evidence/repair/p12_static_screen.json, p12_static_screen.provenance.json, README.md; produced by experiments/repair/runs/p12_screen.sh, experiments/repair/p12_static_screen.py, write_provenance.py.
<i>The certified head kernel</i>	
N-E6	The cuBLAS kernel of the stock head per batch size; turning off reduced-precision reduction changes neither the kernel nor any output bit; the stock FP32-output kernel’s largest observed accumulation error, 2.07×10^{-6} of $\sum_j w_{ij}h_j $ over 2,048 real rows, 295 and 58 times inside the conservative model and the Hopper model; the BF16-output kernel’s accumulator is not observable evidence/certified_head/stock_invariance.json; produced by experiments/certified_head/stock_invariance.py.
N-E7	Which tensor-core instruction the two stock head kernels whose SASS could be read issue (HGMMA, warpgroup wgmma, at $M = 1$ and 32) and how the stock head kernels and Triton’s wgmma accumulate on crafted inputs (16-product blocks, 25 fractional bits, truncation; BF16 epilogue to nearest even) evidence/profiles/wgmma_precision.json, head_tensor_instructions.json; produced by experiments/profiling/wgmma_precision.py, tensor_instructions.py.
N-E8	60,000 real decode rows replayed through the kernel at their engine batch shapes (at most 16 rows): rows and steps that need the fallback under both error models, and certified rows plus the fallback equal to the reference on every row, greedy and seeded evidence/certified_head/replay_decisions.json, fallback_vs_model.json; produced by experiments/certified_head/replay_decisions.py, fallback_vs_model.py.
N-E9	Cost of the per-position fallback, which completes an undecided position from its gathered candidates, against the whole-batch fallback and the stock head chain at $M = 1$ to 256, under both error models (the head alone) evidence/certified_head/micro_head.json, head_path_time.csv, head_path_table.md; produced by bench/micro_head.py, summarize_head.py.
N-E10	Head-path microbenchmark: the certified head with probes on and off, both fallback modes and both error models, the W8A8 and BF16 passes and seeded sampling, against the stock head chain at $M = 1$ to 256; the head alone, not decoding evidence/certified_head/micro_head.json, head_path_time.csv, head_path_table.md, micro.hostload.json; produced by bench/micro_head.py, summarize_head.py.
N-E11	Primitive costs (the stock chain, the W8A16 pass alone, tile GEMMs, re-scoring and the draft summary), Nsight Compute of the production W8A16 launches, and the fastest TMA and pointer-load configurations per pass evidence/certified_head/head_primitives.json, ncu_gemv_summary.json, pointer_vs_tma_w8a16.json, pointer_vs_tma_w8a8.json, pointer_vs_tma_bf16.json; produced by bench/head_primitives.py, tune_gemv.py, experiments/certified_head/ncu_summary.py.
N-E12	The certified head inside SGLang in check mode, with SGLang’s own head run beside it at the same batch shape: eleven arms on 64 public prompts (at most 256 new tokens, the conservative model, the whole-batch fallback and in one arm the per-position fallback) covering greedy decoding, MTP and DFlash greedy verification, MTP draft steps and draft extend, the DFlash draft projection and MTP fixed-noise sampled verification at $T = 0.7$ under deterministic inference; 0 of 196,734 certified positions differed from the stock token; certified and stock outputs identical on 64 of 64 prompts one request at a time, for plain decoding and MTP; the merged package’s 167 GPU and CPU tests; correctness only, nothing timed evidence/certified_head/engine_v2.json, engine_v2_arm_checks.log, engine_v2_gpu_tests.log, x9e2_commit.txt, README.md; produced by experiments/certified_head/engine_validate.sh.
<i>Transport</i>	
N-E13	Drift on 40,000 DFlash-4B pairs, by outcome, position, domain and context, with the per-row thresholds (the paper’s Figure 5, Table 9) evidence/head_geometry/rho_dflash4b.json, rho_dflash4b_quantiles.csv, rho_dflash4b_pairs.csv; produced by experiments/head_geometry/analyze_rho.py.
N-E14	Transport on DFlash-4B by bound family and tiling: skips, partition widths, $P_?$, batch unions, drift scaling (the paper’s Tables 12, 13 and 14) evidence/head_geometry/transport_dflash4b.json, transport_dflash4b.csv; produced by experiments/head_geometry/analyze_transport.py.

Entry	What it supports; files and the program that produced them
N-E15	The same on 16,016 MTP-4B pairs, with their drift evidence/head_geometry/transport_mtp4b.json, transport_mtp4b.csv; produced by experiments/head_geometry/analyze_transport.py.
N-E16	Drift on 40,000 aligned MTP-4B pairs, with the per-row thresholds (the paper’s Figure 5) evidence/head_geometry/rho_mtp4b.json, rho_mtp4b_quantiles.csv, rho_mtp4b_pairs.csv; produced by experiments/head_geometry/analyze_rho.py.
<i>Exact references and formal checks</i>	
N-E17	Lean 4.19.0 elaboration of the two formal files of the paper’s own lemmas (the paper’s Appendix A.10) and of three files on the drafting proposals P-A to P-C (Section 5.8) formal/STATUS.md, evidence/precision/lean_check.log; produced by scripts/check_lean.sh.
N-E18	Exact rational reference for decisions and races (Table 11); the replay witness of Proposition 6.1 evidence/decision_tests.json, race_tests.json and their logs; produced by tests/test_decisions.py, tests/test_races.py, via scripts/verify_artifact.py.
N-E19	Work counts on constructed heads (Figure 3) data/synthetic_drift.csv, evidence/synthetic_drift.json; produced by experiments/synthetic_drift.py.
N-E20	Witnesses that separate the exactness contracts; the regrouped recurrence; the transition norm (the paper’s Appendix D.1) evidence/contracts/contract_witnesses.json, contract_witnesses.log; produced by tests/test_contracts.py.
N-E21	Witnesses for the structure of the recurrent state and for sharing across requests (Section 3.5) evidence/state_structure/state_structure_tests.json, state_structure_tests.log; produced by tests/test_state_structure.py.
<i>The stock engine</i>	
N-E22	First differing module outputs and divergence classes under both error models (the paper’s Figure 6; Table 10) evidence/state_safety/mechanism_plain_c1_vs_mtp_s3_c1.json, mechanism_plain_c1_vs_c32.json, first_difference_by_module.csv, divergences.csv; produced by experiments/state_safety/mechanism.py, analyze_all.sh, compare.py.
N-E23	Divergence rates, BF16 top-two ties, same-server and fresh-server repeats and the deterministic-mode checks; the server settings of every run evidence/state_safety/noise_floor.csv, noise_floor.json, run_meta.json; produced by experiments/state_safety/run_all.sh, compare.py.
N-E24	Positive controls of the bit-exact tap; the method; the tap-changed prompts and their module signature in two tapped sessions evidence/state_safety/tap_control_down9.json, tap_control_core13_decode.json, tap_control_core13.json, tap_check.json, tap_signature.json, README.md; produced by experiments/state_safety/mechanism.py, analyze_all.sh, tap_signature.py.
N-E25	Cache-level checks with the cache-hashing tap: at batch 1, plain decoding against MTP with three steps (40 prompts), the two history pairs at the level of the KV, the prefix cache on against off (40 prompts), one prompt’s same-server repeats with and without a flush; plain decoding at concurrency 1 against 32 with fixed pools (40 prompts); and the tapped runs against untapped ones (the paper’s Appendix D.4) evidence/state_safety/cachecheck_v4_plain_c1_vs_mtp_s3_c1.json, cachecheck_v4_plain_c1_radix_vs_noradix.json, history_v4_mt_bench-0056.json, history_v4_humaneval-0008.json, repeats_v4_h44_flush.json, repeats_v4_h44_noflush.json, cachecheck_v4_plain_c1_vs_c32.json, README.md; produced by experiments/state_safety/mechanism.py, from the tapped sessions of run_tap_v4.sh and run_tap_v4_batch.sh (tap_runs.py).
N-E26	The pools each server allocated for every comparison of the paper’s Appendix D.4 and Section 6.1, with whether the two servers were the same and the pools identical evidence/state_safety/pools.json, run_meta.json, noise_floor.csv; produced by experiments/state_safety/pools.py, analyze_all.sh.

Entry	What it supports; files and the program that produced them
N-E27	The matrix with fixed pools (the cap-8 runs): divergence rates and classes for plain and MTP pairs with identical pools, self-consistency, the pools and token counts of every run, divergences per fragile position by the previous cycle's commit length with the first cycle after prefill, the cap-8 against cap-16 batch-1 comparison, MTP with three steps and plain decoding under the prefix cache off, the overlap scheduler off, deterministic inference and the FP32 head, a second MTP session, and the cap-8 runs against the bench's runs with pools sized at start-up, without the prefix cache (Section 6.2; the paper's Appendices D.4 and E.3) evidence/state_safety/noise_floor_pinned.json, noise_floor_pinned.csv, divergences_pinned.csv, run_meta_pinned.json, cycles_mtp_s1_pinned.json, cycles_mtp_s3_pinned.json, cycles_mtp_s5_pinned.json, cycles_mtp_tree_pinned.json, cross_regime.json, cross_bench.json, divergences_cross_bench.csv, README.md; produced by experiments/state_safety/analyze_all.sh, compare.py, cycles.py.
N-E28	Targeted state tests: truncation and stop tokens inside MTP verify cycles and for the tree and plain decoding, warm against cold continuation, history dependence through the prefix cache with and without it, and checkpoint reuse (Section 6.2; the paper's Appendix D.4) evidence/state_safety/targeted.json; produced by experiments/state_safety/run_targeted.sh, targeted.py, summarize_targeted.py.
N-E29	Aborts on a small recurrent-state pool (four requests; slot assignment not recorded), run-to-run repeats and chunked prefill, with an exploratory alignment check of the largest prompt drifts (H5) evidence/state_safety/targeted.json, README.md; produced by experiments/state_safety/targeted.py, run_targeted.sh, summarize_targeted.py, compare_prefill.py.
N-E30	The declared first-cycle test on 960 fresh prompts: inconclusive under its rule, with the validity checks; a post hoc descriptive upper bound (amendment of 2026-10-02) evidence/state_safety/first_cycle_fresh.json, README.md; produced by experiments/state_safety/first_cycle.py, attest_runner.py.
<i>Where the time goes</i>	
N-E31	Attribution of the plain step and the MTP cycle by component; unprofiled throughput; the state-slot reservation (Table 3; the paper's Tables 20 and 21) evidence/profiles/tables.md, breakdown.csv, step_share.csv, label_structure_check.json, attribution/, windows/; produced by experiments/profiling/run_profiles.py, attribute.py, summarize.py.
N-E32	The head chain's share of a plain step and of an MTP cycle; the head microbenchmark and the kernels cuBLAS selects evidence/profiles/head_microbench.json, head_microbench_kernels.json, attribution/; produced by experiments/profiling/attribute.py, head_microbench.py.
N-E33	Read bandwidth: 3.83 TB/s over 4 GiB for the best of nine launch configurations, 3.79 TB/s over the head's footprint with that configuration evidence/profiles/hbm_bandwidth.json; produced by experiments/profiling/hbm_bandwidth.py.
N-E34	Bytes per step by component, checked against measured kernels; bytes per output token of plain decoding and MTP over the batch size evidence/profiles/bytes_per_step.json, bytes_per_step_sweep.csv, bytes_per_step_sweep_wide.csv; produced by experiments/profiling/bytes_model.py.
N-E35	Nsight Compute of the head GEMM and the GDN decode and verify kernels: DRAM bytes and throughput against the DRAM peak, SM throughput, occupancy and stalls; achieved bandwidth by batch and source with the regime each kernel falls in (Section 3.2) evidence/profiles/ncu_key_kernels.json, kernel_bandwidth.csv, README.md; produced by experiments/profiling/run_ncu.sh, ncu_summary.py, kernel_bandwidth.py.
N-E36	Host functions running while the GPU idles in the MTP cycle evidence/profiles/diagnostics/; produced by experiments/profiling/host_gaps.py, pypspy_summary.py.
N-E37	The DFlash cycle by phase on the two tuned DFlash arms at batch 1 to 64, traced and untraced: the head chains' share, the extend-attention and GDN verify costs, the host gap, and the profile's acceptance (Section 5.2, Section 3.6) evidence/profiles/README.md, dflash_cycle.json, dflash_cycle.csv, dflash_breakdown.csv, step_share.csv, attribution/, windows/; produced by experiments/profiling/run_profiles.py, collect_run.py, dflash_cycle.py, attribute.py, summarize.py.

Entry	What it supports; files and the program that produced them
N-E38	Share of layer 0's GDN input projections in a plain step (P5) evidence/profiles/p5_layer0_in_proj.json; produced by experiments/profiling/layer0_share.py.
N-E39	Derived ceilings per lever stack under both execution models (Table 4, Figure 1) evidence/moonshot/ceilings.json, ceilings.csv; produced by experiments/moonshot/ceilings.py.
N-E40	First served lever measurements, single runs evidence/moonshot/lever_sweeps_quick.csv; produced by experiments/moonshot/lever_sweep.py, levers.py, summarise.py.
N-E41	First engine-only decode-step times by state precision evidence/moonshot/decode_ceiling_try1.csv; produced by experiments/moonshot/decode_ceiling_sweep.py.
N-E42	Engine patches for the reduced draft head, the relaxed greedy rule and rounding-preserving replay, with their tests; code, not results engine/sglang/patches/moonshot/, tests/test_moonshot_levers.py, tests/test_gdn_exact_replay.py.
N-E43	Rounding-preserving replay (P4) against the packed decode, per word, and its kernel time by ring length (Section 3.5) evidence/moonshot/gdn_exact_replay_check_L4.json, gdn_exact_replay_check_L16.json, gdn_exact_replay_check_L4_layer20.json, gdn_exact_replay_bench.json; produced by experiments/moonshot/gdn_exact_replay_check.py.
N-E44	P4's served test: the design, metric and decision rule declared before its first run, the prompt check corrected after it, the three void attempts, the cause of their 127-request plateau checked against every admission wave, the amendments declared before each further run, and the amended admission preflight that showed 128 running requests (declarations and a diagnosis, not results) evidence/moonshot/README.md, p4_admission_plateaus.csv; produced by experiments/moonshot/run_p4b.sh, run_p4_admission.sh, validate_p4_ab.py, output_probe.py, check_admission.py, admission_plateaus.py.
N-E45	P4's served test: replay against dense decoding in four pairs at exactly 128 running requests, the validity checks, the decode-rate and client-throughput ratios with 95% t intervals and the verdict; the server output probe at concurrency 1 with its dense control (Section 3.5) evidence/moonshot/README.md, p4_ab_verdict.json, p4_ab_arms.csv, p4_output_probe.csv; produced by experiments/moonshot/run_p4b.sh, validate_p4_ab.py, output_probe.py, check_admission.py, summarise.py.
N-E46	Triton attention for the MTP cycle's target and draft at concurrency 1 and 4, single runs evidence/moonshot/host_levers.csv, README.md; produced by experiments/moonshot/lever_sweep.py, levers.py, summarise.py.
N-E47	Backbone weight GEMMs and RMSNorm in isolation: the kernels cuBLAS runs per projection and M , faster kernels reading the same weights and their bitwise agreement with cuBLAS, the packed GDN input projection, the norm and SiLU prologues, layer skeletons and the folding variants, each run's foreign CPU load; served: each switch's greedy outputs against stock plain decoding, bitwise checks on the raw runs, one session each of paired serving on tuned plain decoding and on MTP with FlashInfer and with Triton attention, with the streamed text of the routed and switches-off engines compared on the Triton arm, and an Nsight trace of the GEMM kernels a served decode step runs; the GPU tests; the opt-in engine patch series is code, not a result (Section 3.6) evidence/backbone/gemm_microbench.json, merge_microbench.json, norm_microbench.json, chain_microbench.json, chain_fold_variants_microbench.json, gpu_tests_hold2.log, tables.md, README.md, hostload/, served/, engine/sglang/patches/backbone/; produced by experiments/backbone/gemm_bench.py, summarize.py, make_table.py, bitwise_runs.py, experiments/state_safety/compare.py, bench/pareto.py, experiments/backbone/insitu_gemm.py, stream_text_identity.py, experiments/profiling/run_profiles.py.
N-E48	Host-side idle in the MTP and DFlash cycles: stock host traces, the derived untraced idle, plan and attention-output equivalence of the sync-free plans, in-engine validation, greedy output equality with stock, and untraced cycle times before and after; for all five patches on tuned MTP, stock host traces with the same window driver, held cycle times and an interleaved served A/B; for all five on DFlash block 8 with FlashInfer draft attention, pinned-pool output equality, held cycle times and an interleaved served A/B (Section 3.6) evidence/hostgap/README.md, stock_traces_hold1.json, cycle_profiles.json, plan_equivalence.json, equality/, hold4/, hold5/, engine/sglang/patches/hostgap/; produced by experiments/hostgap/profile_arms.sh, gap_analysis.py, hold_equality_profiles.sh, plan_equivalence.py, equality.py, summarize.py, hold_mtp.sh, hold_dflash.sh, ab_sweep.sh, ab_summary.py.

Entry	What it supports; files and the program that produced them
N-E49	The two declared approximations, FP16 recurrent state and the int4 DFlash drafter, against the tuned non-lossy arms: three pre-registered sessions, GSM8K and logit-probe quality holds and the verdicts against the lossy budget; the exploratory W4A16 GEMM microbenchmark (Section 3.6) evidence/lossy/decisions.json, ratios.csv, README.md, sessions/, quality/, probes/, gemm_w4a16.json; produced by experiments/lossy/analyze.py, run_hold.py, gsm8k_partial.py, gemm_w4a16_bench.py.
N-E50	P13: the GDN state projected to rank r on offline key-energy, query and product bases, against the same rule at full rank: full-vocabulary KL, top-1 agreement and a delayed-retrieval probe under long-context teacher forcing (offline, training-free; a decision, not a pre-registered test) evidence/moonshot/gdn_state_rank_study.json, README.md; produced by experiments/moonshot/gdn_state_rank_study.py.
N-E51	SGLang’s chunked GDN kernel as a block-parallel verify path: BF16 disagreement and time by block width evidence/moonshot/p7_fast_verify_check.json, p7_verify_width_bench.json; produced by experiments/moonshot/gdn_fast_verify_check.py.
N-E52	Coverage of hot draft-vocabulary maps, built on the tuning split, on the confirmation split’s plain outputs evidence/moonshot/token_map_coverage.csv, README.md; produced by experiments/moonshot/token_map_coverage.py, bench/token_map.py.
N-E53	Byte and FLOP decomposition of the MTP cycle at concurrency 128, from the tuning points (derived) evidence/moonshot/README.md, evidence/bench/tuning/points.csv.
<i>Drafting and serving</i>	
N-E54	Command, environment and resolved settings of the drafter’s acceptance runs evidence/drafter/launch/; produced by experiments/drafter/run_trace.sh.
N-E55	DFlash acceptance by block position and domain (Table 8, Figure 2) evidence/drafter/acceptance_summary.csv, acceptance_by_position.csv; produced by experiments/drafter/summarize_acceptance.py.
N-E56	Acceptance by text segment and the class of the first rejected draft evidence/drafter/trace_segments_b16.csv, trace_segments_b16_failures.json (and the block-8 files); produced by experiments/drafter/trace_analysis.py.
N-E57	Card-reproduction gate on the 80 MT-Bench first turns with the card’s settings; chat acceptance by prompt source evidence/drafter/card_gate_mtbench.json, README.md, evidence/drafter/launch/zlab_b16_card_gate.json; produced by experiments/drafter/run_card_gate.sh.
N-E58	DFlash and three-step MTP acceptance on the second panel, by position and domain evidence/drafter/panel_v2/acceptance_summary.csv, acceptance_by_position.csv, evidence/drafter/launch/panel_v2_zlab_b16.json, panel_v2_mtp3.json, panel_v2_plain.json; produced by experiments/drafter/run_equality.sh, summarize_acceptance.py.
N-E59	Divergences of DFlash and MTP outputs from plain decoding, with the plain run’s margins evidence/drafter/panel_v2/equality_zlab_b16.json, equality_mtp3.json; produced by experiments/drafter/run_equality.sh, compare_outputs.py.
N-E60	The P6 candidate-support screen: oracle bound per cycle for the top 2 to 16 candidates evidence/drafter/support/zlab_b16_panel_v1_summary.json, zlab_b16_panel_v1_survival.csv; produced by experiments/drafter/support_screen.py.
N-E61	SGLang’s fold-every-commit GDN verification enabled for DFlash and MTP: kernel parity with the stock verify; served greedy outputs against stock with matched pools and fixed batch composition; the per-phase split at batch 8 and 16 that decides P10’s gate; one session of paired serving on the two tuned DFlash arms (Section 5.9) evidence/drafter/buffered_verify/gdn_verify_parity.json, exactness.csv, localize/, evidence/drafter/phase_timing/summary.csv, evidence/drafter/fold_timing/summary.json, evidence/drafter/README.md, engine/sglang/patches/drafter/; produced by experiments/drafter/gdn_verify_parity.py, run_replay_check.sh, run_fold_localize.sh, run_phase_timing.sh, run_fold_timing.sh, ab_timing_summary.py.

Entry	What it supports; files and the program that produced them
N-E62	Narrow value tiles for the fold’s ring-writing verify (engine patch drafter/0005): kernel parity with the stock verify, served bitwise checks with matched pools, DFlash with decode-only ReplaySSM against stock, and one session of paired serving at concurrency 1 to 8 on the two tuned DFlash arms with its protocol check (Section 5.9) evidence/drafter/fold_narrow_tiles/gdn_verify_parity.json, off-p1_vs_fold-p1.json, dflash-w4-fold-vs-dflash-w4-off.json, mtp-w8-fold-vs-mtp-w8-off.json, dflash-w4-replayssm-decode-vs-off.json, evidence/drafter/fold_narrow_tiles/timing/check.json, summary.json, evidence/drafter/README.md, engine/sclang/patches/drafter/0005-gdn-ring-verify-narrow-tiles.patch; produced by experiments/drafter/run_fold_check.sh, gdn_verify_parity.py, fold_localize.py, compare_outputs.py, run_fold_timing.sh, fold_timing_check.py.
N-E63	GPU time per layer of the fold’s ring-writing GDN verify at value tiles 4, 8, 16 and 32, blocks 16 and 8, batches 1 to 64, read by a threshold rule declared before the run (one exclusive hold, four repeats, each tile checked bitwise against tile 32), and the derived change in the fold’s served throughput by tile and concurrency (Section 5.9) evidence/drafter/ring_tile_sweep/sweep.json, cycle_estimate.json, provenance.txt, cpu_load.json, evidence/drafter/README.md; produced by experiments/drafter/run_ring_tile_sweep.sh, gdn_ring_tile_sweep.py, ring_tile_rule.py, ring_tile_cycle_estimate.py.
N-E64	A training prompt set disjoint from every benchmark and quality set evidence/drafter/data/prompts-v2.manifest.json; produced by experiments/drafter/build_train_prompts.py.
N-E65	Repair Stage A: verify-pass, commit and draft times per block width with forced full acceptance; the DFlash baseline; derived oracle speedups S_a and S_b (Section 5.6) evidence/repair/stage_a_timing.json, stage_a_oracle.json, stage_a_oracle.csv, README.md; produced by experiments/repair/analyze_timing.py, stage_a.py, experiments/repair/runs/stage_a.sh.
N-E66	Verify-pass decomposition at $B = 16, 64$ and 256: per-position state writes dropped, SGLang’s Triton GDN verify kernel, and Stage A recomputed on it (exactness not classified); a same-session control of the three variants at $B = 16$ and 256; kernel time per verify cycle at $B = 256$ by category evidence/repair/stage_a_timing.json, stage_a_oracle_triton.json, stage_a_oracle_triton.csv, stage_a_oracle.csv, verify_control.json, verify_kernels_b256.json, README.md; produced by experiments/repair/runs/decomposition.sh, verify_control.sh, verify_nsys.sh, experiments/repair/analyze_timing.py, nsys_kernels.py, stage_a.py.
N-E67	The per-position acceptance a block drafter would need for 5× end to end at concurrency 1, from Stage A’s cycle periods under both verify kernels and the block-16 trace’s acceptance and top-16 support by position (derived, no new runs; Section 5.6) evidence/drafter/drafting_requirement.json, README.md; produced by experiments/drafter/drafting_requirement.py.
N-E68	One-step recycling of the target’s suffix predictions against fresh DFlash drafts evidence/repair/one_step_recycling.json, draft_source_accuracy.csv; produced by experiments/repair/analyze_jacobi.py.
N-E69	Repair Stage B: the anchored residual evaluator on real block-16 DFlash blocks (operator error, decision agreement, R_i , progress per sweep, exact Jacobi) and the economic gate evidence/repair/residual_eval_b16.json, residual_eval_b16.agree_by_distance.csv, p3_gate_b16.json, README.md; produced by experiments/repair/runs/residual_b16.sh, experiments/repair/residual_eval.py, summarize_residual.py, p3_gate.py.
N-E70	Exact Jacobi sweeps (recycle and keep) from real DFlash windows in the engine at $B = 16$ and 32: accepted drafts and committed tokens per exact pass after each sweep, P3’s ceiling, the effect of one correction downstream and the replay check (Section 5.6) evidence/repair/jacobi_probe_progress.csv, jacobi_probe_summary.json, jacobi_probe_provenance.json, README.md; produced by experiments/repair/runs/jacobi_probes.sh, experiments/repair/serve_probe.py, analyze_jacobi.py, write_provenance.py.
N-E71	P9 support oracle at concurrency 1: supported rates, oracle and fresh second-cycle commits and Δ with bootstrap intervals per candidate-set size, the unchanged cached control, the bound with a free reused verify, the omniscient-gate oracle with a padded and a free verify, and both with the measured remainder verify; verify times at widths 2 to 16; the reuse program’s time; draft shares and the oracle at concurrency 8 and 16 (upper bounds, not served results) evidence/repair/p9_support_oracle.json, p9_verify_widths.json, p9_program_cost.json, p9_draft_share.json, p9_support_oracle_c8.json, p9_support_oracle_c8_marginal.json, p9_support_oracle_c16.json, p9_support_oracle_c16_marginal.json, README.md; produced by experiments/repair/runs/p9_verify_widths.sh, p9_draft_share.sh, experiments/repair/p9_support_oracle.py, p9_program_cost.py, analyze_timing.py.

Entry	What it supports; files and the program that produced them
N-E72	Oracle upper bounds for the three external drafting proposals under rules committed before the oracles ran: the per-cycle requirement for 5× with the Triton verify kernel, P-A’s innovation density, perfect-event oracle and candidate support, P-B’s first-gap bound by anchor count (with the post-run correction of its extension past slot 15), and P-C’s one-sweep bound (Section 5.8) evidence/frontier/README.md, frame.json, innovation_oracle.json, anchor_bound.json, defect_oracle.json; produced by experiments/frontier/frame.py, interpreters.py, innovation_oracle.py, anchor_bound.py, defect_oracle.py.
N-E73	The certified head served on the four tuned arms against the same arms without it: three pre-registered sessions, paired throughput ratios with the four primaries Holm-adjusted, the served envelope, check-mode counters, token identity at concurrency 1, closed-loop divergences, the wave and seeded controls, and BF16 and FP32 reference paths at the large divergences (the paper’s Appendix E.5) evidence/certified_head/served/README.md, summary.json, ratios.csv, pairs.csv, check.csv, equality.csv, classes.csv, control_waves_mtp.json, control_waves_mtp64.json, control_waves_dflash16.json, control_seeded_mtp.json, control_seeded_stats.json, control_seeded_tokens.json, reference_paths.json, drain_579ae7ce.csv, stress_summary.json, capture_memory.csv, experiments/benchcert/README.md; produced by experiments/benchcert/analyze.py, run_session.py, rescore.py, control_waves.py, paths.py, drain.py, fallback_stress.py.
N-E74	The frozen serving workload: sources, revisions, splits and token statistics bench/workloads/mixed-v2/manifest.json; produced by bench/build_workload.py.
N-E75	Where greedy outputs stop on 576 tuning prompts evidence/bench/workload/natural_lengths_tune.json, natural_requests_tune.csv; produced by bench/sweep.py, bench/lengths.py.
N-E76	Feasibility probes of capacity and draft trees (what launches, not performance) evidence/bench/probes/; produced by bench/campaigns/capacity_probes.sh, bench/pareto.py.
N-E77	Frontend diagnosis at concurrency 256 on a quiet host (single points) evidence/bench/frontend/; produced by bench/campaigns/frontend_diagnostic.sh, bench/pareto.py.
N-E78	Configuration search on the tuning split, one launch per configuration, and the chosen arms (Table 6); derived snapshot traffic evidence/bench/tuning/points.csv, frontier.csv, launches.csv, evidence/bench/README.md, bench/arms.toml; produced by bench/campaigns/tuning_depth.sh, tuning_knobs_dflash.sh, tuning_backend_dflash.sh, bench/pareto.py.
N-E79	Confirmed serving frontier: nine tuned arms on the confirmation split, three sessions, paired ratios, run-to-run variance and the envelope over points with at least three sessions evidence/bench/confirm/frontier.csv, envelope.csv, envelope-all.dat, pairs.csv, points.csv, launches.csv, evidence/bench/README.md; produced by bench/campaigns/confirm.sh, confirm_supplementary.sh, confirm_evidence.sh.
N-E80	Greedy outputs of every arm that changes the target’s arithmetic, and of stock MTP and DFlash, against matched stock references without the prefix cache: first-divergence classes, rates and ratios to the batch-shape floor, each server’s pools, and the first classification against the prefix-cache-on plain run (Section 4) evidence/bench/equality/classes.json, report.json, divergences.csv, table.csv, pools.csv, evidence/bench/README.md; produced by bench/campaigns/equality_tuned.sh, bench/divergence.py, experiments/state_safety/run_matrix.py, compare.py.
N-E81	GSM8K test accuracy of six runs (two of plain decoding, buffered MTP, MTP with snapshot verification, DFlash at block 8, replayed plain decoding) with paired exact McNemar tests evidence/bench/quality/comparisons.json, evidence/bench/quality/; produced by bench/campaigns/quality.sh, quality_compare.py.

A file name listed without a directory is in the directory of the entry’s first file. Raw captures, traces and per-row arrays are not in the repository; each evidence README gives the command that regenerates them on the same hardware.

10 Glossary

This table defines the terms these notes use that the paper’s glossary (Appendix G of the paper) does not, and the paper’s earlier vocabulary that moved text keeps.

Term	Definition
Accepted length	Committed tokens per verify cycle, the bonus token included: τ pooled over cycles, $\bar{\tau}_{\text{req}}$ averaged over requests as the DFlash model card does (Section 5.3).
Common mass α^*	The largest probability with which an exact sampler can emit a token before more is known, $\sum_i \inf_p p_i$ over the compatible laws (Proposition 7.2).
Dead, deferred, soundly enclosed	The three classes of skipped work: no remaining consumer; a later consumer with a reconstruction path; or bounded well enough for every consumer that remains (Section 7.2).
Demand frontier	Suppression of head work after the first certified rejection in a chain (Proposition 7.5).
Emitted and materialized prefix	The tokens a cycle has returned and the tokens already consumed into every cache; the bonus token is emitted before it is materialized (Section 6.2).
Envelope β_i	The paper’s “error bound”: a proved bound on the error of the low-precision copy’s score for vocabulary entry i (Theorem 3.1 of the paper).
Exact and lossy lever, exact and lossy stack	A lever is exact when every emitted token remains the target’s decision under the stock-kernel contract, lossy otherwise; the lossy stack admits lossy levers under a quality budget declared before measuring (Sections 3.6 and 3.7).
First matrix, pinned matrix	The two sets of divergence runs: servers capped at 16 running requests with pools sized at start-up, and servers capped at 8 with identical, fixed pools (the paper’s cap-16 and cap-8 runs, Appendix D.4 of the paper).
H1–H7, P1–P13, P-A to P-C	The programme’s questions, the later proposals and three external drafting proposals (Table 2), all but P13 with a pre-registered kill test; P7a and P7b are the two parts of P7, and a proposal is listed once it has a result or a decision.
Kill test	A test fixed before a proposal is built, designed to reject it cheaply if its best case cannot pay; a pass does not support the proposal, it only lets it go on.
Layer-serial and overlapped ceilings	Throughput bounds under two execution models: per-request kernels after the weight GEMMs, as the engine runs, or overlapped with them (Equations (2) and (3)).
Pinned commit	SGLang commit bd66ce343e4f, the engine version the paper pins (its Appendix F.3); “code reading at the pinned commit” means read in that commit’s source.
Replayed and snapshot verification	Two ways SGLang verifies a draft chain on the GDN layers: snapshot verification writes one FP32 state per draft token and copies the accepted one back at commit; replayed verification (<code>--enable-linear-replayssm-spec</code>) keeps each draft token’s inputs and folds the accepted prefix into the state at commit (Section 3.4).
Rounding flip, order flip, ambiguous	How a divergence reaches the token: BF16 rounding merges an unchanged accumulator order into a tie; the accumulator order reverses; or the order cannot be read from the data under the named error model (Section 6.1).
Rounding-preserving replay	Decoding that keeps a dense state anchor and the recent FP32 operands and replays them in the stock kernel’s order, writing the state less often (P4, Section 3.5).
Row	In moved text, either a vocabulary entry (a row of the head’s weight matrix W) or a position (one head input of a batch); the paper keeps the two apart.
Second panel	The 80 prompts of the drafter’s equality and acceptance runs: the first 16 chat, code and GSM8K training prompts of the serving workload’s confirmation split, and 32 MATH-500 problems (Section 5.3).
Self-evidence	The low-precision copy of the head evaluated at the target’s own head input, as opposed to transport from a drafter’s summaries (Section 3 of the paper).
Survival $S(k)$, conditional acceptance α_k	The share of verify cycles that accept at least k drafted tokens, and $S(k)/S(k-1)$ (Section 5.3).
Verify pass, draft step, cycle	The target’s forward pass over a drafted block; one drafter forward; a draft, its verification and the state commit (Appendix E.2 of the paper; Section 5).

References

- An, Z., Liu, T., Liu, Z., Li, D., Liu, R., and Barsoum, E. PARD-2: Target-aligned parallel draft model for dual-mode speculative decoding. arXiv:2605.08632v1, 2026. URL <https://arxiv.org/abs/2605.08632v1>.
- Austin, J., Odena, A., Nye, M., Bosma, M., Michalewski, H., Dohan, D., Jiang, E., Cai, C., Terry, M., Le, Q., and Sutton, C. Program synthesis with large language models. arXiv:2108.07732v1, 2021. URL <https://arxiv.org/abs/2108.07732v1>.
- Cai, T., Li, Y., Geng, Z., Peng, H., Lee, J. D., Chen, D., and Dao, T. Medusa: Simple LLM inference acceleration framework with multiple decoding heads. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pp. 5209–5235. PMLR, 2024. URL <https://proceedings.mlr.press/v235/cai24b.html>.
- Cai, Y., Liang, X., Wang, X., Ma, J., Liang, H., Luo, J., Zuo, X., Duan, L., Yin, Y., and Chen, X. FastMTP: Accelerating

- LLM inference with enhanced multi-token prediction. arXiv:2509.18362v1, 2025. URL <https://arxiv.org/abs/2509.18362v1>.
- Chen, C., Borgeaud, S., Irving, G., Lespiau, J.-B., Sifre, L., and Jumper, J. Accelerating large language model decoding with speculative sampling. arXiv:2302.01318v1, 2023. URL <https://arxiv.org/abs/2302.01318v1>.
- Chen, J., Liang, Y., and Liu, Z. DFlash: Block diffusion for flash speculative decoding. In *International Conference on Machine Learning (ICML 2026)*, 2026a. URL <https://arxiv.org/abs/2602.06036v2>.
- Chen, M., Tworek, J., Jun, H., Yuan, Q., Pinto, H. P. d. O., Kaplan, J., Edwards, H., Burda, Y., Joseph, N., Brockman, G., Ray, A., Puri, R., Krueger, G., Petrov, M., Khlaaf, H., Sastry, G., Mishkin, P., Chan, B., Gray, S., Ryder, N., Pavlov, M., Power, A., Kaiser, L., Bavarian, M., Winter, C., Tillet, P., Such, F. P., Cummings, D., Plappert, M., Chantzis, F., Barnes, E., Herbert-Voss, A., Guss, W. H., Nichol, A., Paino, A., Tezak, N., Tang, J., Babuschkin, I., Balaji, S., Jain, S., Saunders, W., Hesse, C., Carr, A. N., Leike, J., Achiam, J., Misra, V., Morikawa, E., Radford, A., Knight, M., Brundage, M., Murati, M., Mayer, K., Welinder, P., McGrew, B., Amodei, D., McCandlish, S., Sutskever, I., and Zaremba, W. Evaluating large language models trained on code. arXiv:2107.03374v2, 2021. URL <https://arxiv.org/abs/2107.03374v2>.
- Chen, Z., May, A., Svirschevski, R., Huang, Y., Ryabinin, M., Jia, Z., and Chen, B. Sequoia: Scalable and robust speculative decoding. In *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)*, pp. 129531–129563, 2024. URL https://proceedings.neurips.cc/paper_files/paper/2024/hash/ea1f5f0878d43ff4fb8bf64ef4a2326c-Abstract-Conference.html.
- Chen, Z., Xu, D., Zhang, Y., Wang, C., Xu, M., and Ma, Y. NanoSpec: Accelerating speculative decoding using minimalist in-context vocabularies. arXiv:2605.26444v2, 2026b. URL <https://arxiv.org/abs/2605.26444v2>. Version 1 was titled “MicroSpec: Accelerating Speculative Decoding with Lightweight In-Context Vocabularies”.
- Cheng, X., Yu, X., Shao, C., Li, J., Xiong, Y., Qian, Y., Zhu, J., Ma, S., Zhang, X., Ye, J., Chen, Q., Deng, C., Yu, J., Dai, D., Zhang, Z., Wei, Y., Tan, Y., Yang, W., Xu, R., Wu, Y., Xu, Z., Wang, X., Chen, M., Tian, R., Bi, X., Hao, Z., Chen, S., Cao, H., Zhang, W., Xu, A., Zhang, H., Zhao, D., and Liang, W. DSpark: Confidence-scheduled speculative decoding with semi-autoregressive generation. arXiv:2607.05147v1, 2026. URL <https://arxiv.org/abs/2607.05147v1>.
- Cobbe, K., Kosaraju, V., Bavarian, M., Chen, M., Jun, H., Kaiser, L., Plappert, M., Tworek, J., Hilton, J., Nakano, R., Hesse, C., and Schulman, J. Training verifiers to solve math word problems. arXiv:2110.14168v2, 2021. URL <https://arxiv.org/abs/2110.14168v2>.
- DeepSeek-AI. DeepSeek-V3 technical report. arXiv:2412.19437v2, 2024. URL <https://arxiv.org/abs/2412.19437v2>.
- Dutson, M., Li, Y., and Gupta, M. Eventful Transformers: Leveraging temporal redundancy in vision transformers. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pp. 16911–16923, 2023. URL https://openaccess.thecvf.com/content/ICCV2023/html/Dutson_Eventful_Transformers_Leveraging_Temporal_Redundancy_in_Vision_Transformers_ICCV_2023_paper.html.
- Fu, Y., Bailis, P., Stoica, I., and Zhang, H. Break the sequential dependency of LLM inference using lookahead decoding. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pp. 14060–14079. PMLR, 2024. URL <https://proceedings.mlr.press/v235/fu24a.html>.
- Ghantasala, S. M. TreeWY: Speculative verification for Gated DeltaNet hybrids. arXiv:2608.20961v1, 2026. URL <https://arxiv.org/abs/2608.20961v1>.
- Gloeckle, F., Idrissi, B. Y., Rozière, B., Lopez-Paz, D., and Synnaeve, G. Better & faster large language models via multi-token prediction. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pp. 15706–15734. PMLR, 2024. URL <https://proceedings.mlr.press/v235/gloeckle24a.html>.
- Goel, R., Agrawal, S., Gagrani, M., Park, J., Zao, Y., Zhang, H., Liu, T., Yang, Y., Yuan, X., Lu, J., Lott, C., and Lee, M. VOCABTRIM: Vocabulary pruning for efficient speculative decoding in LLMs. arXiv:2506.22694v2, 2025. URL <https://arxiv.org/abs/2506.22694v2>. ICML 2025 Workshop on Efficient Systems for Foundation Models.
- Gu, G., Heo, B., Jun, H., Kang, Y., Lee, S., Yun, S., and Han, D. Verification-aware training for speculative decoding. arXiv:2608.30135v1, 2026. URL <https://arxiv.org/abs/2608.30135v1>.
- Hu, L., Kou, S., Fu, Y., Rajbhandari, S., Rosing, T., He, Y., Deng, Z., and Zhang, H. Fast and accurate causal parallel decoding using Jacobi Forcing. In *International Conference on Machine Learning (ICML 2026)*, 2026. URL <https://arxiv.org/abs/2512.14681v1>.

- Huang, K., Guo, X., and Wang, M. SpecDec++: Boosting speculative decoding via adaptive candidate lengths. In *Second Conference on Language Modeling (COLM 2025)*, 2025. URL <https://arxiv.org/abs/2405.19715v3>.
- Inco AI. DFlash 2: Keep drafting parallel. Inco AI blog, August 2026a. URL <https://inco.ai/blog/dflash2/>.
- Inco AI. Qwen3.8-27B-DFlash2 model card. Hugging Face, revision 015e795645c74b1a0eeef3b570031fb62e769bc5, 2026b. URL <https://huggingface.co/incoai/Qwen3.8-27B-DFlash2/tree/015e795645c74b1a0eeef3b570031fb62e769bc5>.
- Jiang, W., Chitty-Venkata, K. T., Flynn, M., Meyerson, R., Qi, Z., Wu, T., Kurtic, E., Yu, M., and Marques, A. H-Spec: Parallel speculative decoding without a drafter-side KV cache. arXiv:2609.24197v1, 2026. URL <https://arxiv.org/abs/2609.24197v1>.
- Karimi, A., Gao, C., and Hassanpour, N. DBLAST: Dependent block drafting for stochastic speculative decoding. arXiv:2608.05448v1, 2026. URL <https://arxiv.org/abs/2608.05448v1>.
- Khattak, F. A. and Mikaitis, M. Accurate models of NVIDIA tensor cores. *ACM Transactions on Architecture and Code Optimization*, 23(3):1–25, 2026. doi: 10.1145/3830409.
- Kim, J., Lee, J., and Kim, B.-K. Quantize the target, quantize the drafter: Efficient inference with Qwen3.5-4B. arXiv:2607.04244v2, 2026a. URL <https://arxiv.org/abs/2607.04244v2>. Submission to the Efficient Qwen Competition, ICML 2026 Workshop on AdaptFM.
- Kim, S., Koo, J., Kim, H., Bang, I., Lee, S., Oh, H., Park, B., and Lee, D. BV Loss: Block verification-aware loss for block diffusion speculative decoding. arXiv:2609.34832v1, 2026b. URL <https://arxiv.org/abs/2609.34832v1>.
- Koo, J., Woo, S., Kil, J., Lee, J., Lee, S., Jung, K., and Kim, M. Carryover drafting: Recycling rejected states for speculative decoding. arXiv:2609.14717v1, 2026. URL <https://arxiv.org/abs/2609.14717v1>.
- Köpf, A., Kilcher, Y., von Rütte, D., Anagnostidis, S., Tam, Z.-R., Stevens, K., Barhoum, A., Nguyen, M. D., Stanley, O., Nagyfi, R., ES, S., Suri, S., Glushkov, D., Dantuluri, A., Maguire, A., Schuhmann, C., Nguyen, H., and Mattick, A. OpenAssistant conversations – democratizing large language model alignment. In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023), Datasets and Benchmarks Track*, pp. 47669–47681, 2023. URL https://proceedings.neurips.cc/paper_files/paper/2023/hash/949f0f8f32267d297c2d4e3ee10a2e7e-Abstract-Datasets_and_Benchmarks.html.
- Kou, S., Hu, L., He, Z., Deng, Z., and Zhang, H. CLLMs: Consistency large language models. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pp. 25426–25440. PMLR, 2024. URL <https://proceedings.mlr.press/v235/kou24a.html>.
- Kumar, T., Dao, T., and May, A. Speculative speculative decoding. In *International Conference on Learning Representations (ICLR 2026)*, 2026. URL <https://arxiv.org/abs/2603.03251v3>.
- Leviathan, Y., Kalman, M., and Matias, Y. Fast inference from transformers via speculative decoding. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pp. 19274–19286. PMLR, 2023. URL <https://proceedings.mlr.press/v202/leviathan23a.html>.
- Li, Y., Wei, F., Zhang, C., and Zhang, H. EAGLE: Speculative sampling requires rethinking feature uncertainty. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pp. 28935–28948. PMLR, 2024a. URL <https://proceedings.mlr.press/v235/li24bt.html>.
- Li, Y., Wei, F., Zhang, C., and Zhang, H. EAGLE-2: Faster inference of language models with dynamic draft trees. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pp. 7421–7432, 2024b. doi: 10.18653/v1/2024.emnlp-main.422.
- Li, Y., Wei, F., Zhang, C., and Zhang, H. EAGLE-3: Scaling up inference acceleration of large language models via training-time test. In *Advances in Neural Information Processing Systems 38 (NeurIPS 2025)*, 2025. URL <https://neurips.cc/virtual/2025/poster/119930>.
- Lin, L., Zeng, Z., Peng, S., Liu, S., and Ji, R. ReTrace: Rejected-trajectory conditioning for speculative decoding. arXiv:2608.29748v2, 2026. URL <https://arxiv.org/abs/2608.29748v2>.
- Liou, Z.-W. and Dao, T. ReplaySSM: Cache SSM inputs, not state. Dao AI Lab blog, June 2026. URL <https://dao-lab.ai/blog/2026/replayssm/>.

- Liu, A., Meng, J., Liu, F., and Chen, Y. CURE: Local uncertainty repair for block-parallel speculative decoding. arXiv:2608.00531v1, 2026a. URL <https://arxiv.org/abs/2608.00531v1>.
- Liu, T., Shen, Y., Cen, R., Shi, J., Zhang, J., Qin, G., Liu, H., Liu, S., Yu, G., and Zhu, J. D-cut: Adaptive verification depth pruning for batched speculative decoding. arXiv:2607.14647v1, 2026b. URL <https://arxiv.org/abs/2607.14647v1>.
- Luo, X., Wang, Y., Zhu, Q., Zhang, Z., Zhang, X., Yang, Q., and Xu, D. Turning trash into treasure: Accelerating inference of large language models with token recycling. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 6816–6831, 2025. doi: 10.18653/v1/2025.acl-long.338.
- Ma, Z. GDN Tree-Scan: Served tree verification for recurrent-hybrid language models. arXiv:2609.23900v1, 2026. URL <https://arxiv.org/abs/2609.23900v1>. Version 2 (1 October 2026) is titled “LumoTree: Path-Parallel Speculative Verification for Hybrid Language Models”.
- NVIDIA. AIPerf metrics reference. AIPerf documentation, 2026a. URL <https://docs.nvidia.com/aiperf/reference/ai-perf-metrics-reference>. Accessed 30 September 2026.
- NVIDIA. Parallel Thread Execution ISA, version 9.4. CUDA documentation, 2026b. URL <https://docs.nvidia.com/cuda/parallel-thread-execution/index.html>. Accessed 30 September 2026.
- Pan, Y., Xi, H., Zhu, K., Li, X., Wu, Y., Mishra, M., Zhang, H., Zheng, W. X., Kasikci, B., Han, S., Keutzer, K., Iyer, R., and Stoica, I. LeapQuant: Efficient linear attention with accurate recurrent state quantization. arXiv:2609.38166v1, 2026. URL <https://arxiv.org/abs/2609.38166v1>.
- Parger, M., Tang, C., Twigg, C. D., Keskin, C., Wang, R., and Steinberger, M. DeltaCNN: End-to-end CNN inference of sparse frame differences in videos. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 12497–12506, 2022. URL https://openaccess.thecvf.com/content/CVPR2022/html/Parger_DeltaCNN_End-to-End_CNN_Inference_of_Sparse_Frame_Differences_in_Videos_CVPR_2022_paper.html.
- Qwen. Qwen3.5-4B model card. Hugging Face, revision 851bf6e806efd8d0a36b00ddf55e13ccb7b8cd0a, 2026. URL <https://huggingface.co/Qwen/Qwen3.5-4B/tree/851bf6e806efd8d0a36b00ddf55e13ccb7b8cd0a>.
- Rusanovsky, M., Miron, Y., Uziel, R., Belhasin, O., Guo, H., Zilberstein, R., Ashkenazi, M., and Elad, M. LiLiCorr: Lightweight likelihood correlation of parallel drafts for speculative decoding. arXiv:2608.20530v2, 2026. URL <https://arxiv.org/abs/2608.20530v2>.
- Santilli, A., Severino, S., Postolache, E., Maiorca, V., Mancusi, M., Marin, R., and Rodolà, E. Accelerating transformer inference for translation via parallel decoding. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 12336–12355, 2023. doi: 10.18653/v1/2023.acl-long.689.
- SGLang contributors. GDN ReplaySSM speculative fold (python/sglang/kernels/ops/attention/fla/gdn_replayssm_spec_fold.py). Source code, commit bd66ce343e4f6e2f2b75d7e820fe4d0718a8d824, 2026a. URL https://github.com/sgl-project/sglang/blob/bd66ce343e4f6e2f2b75d7e820fe4d0718a8d824/python/sglang/kernels/ops/attention/fla/gdn_replayssm_spec_fold.py.
- SGLang contributors. [Feature] Dynamic verification for DFlash2. SGLang pull request 36136, opened by GitHub user EthanWang555, head commit c0235e700214f1de5044be25f795faf5b755e450, August 2026b. URL <https://github.com/sgl-project/sglang/pull/36136>.
- SGLang contributors. DFlash model and candidate-head implementation (python/sglang/srt/models/dflash.py). Source code, commit bd66ce343e4f6e2f2b75d7e820fe4d0718a8d824, 2026c. URL <https://github.com/sgl-project/sglang/blob/bd66ce343e4f6e2f2b75d7e820fe4d0718a8d824/python/sglang/srt/models/dflash.py>.
- SGLang contributors. DFlash speculative worker (python/sglang/srt/speculative/dflash_worker_v2.py). Source code, commit bd66ce343e4f6e2f2b75d7e820fe4d0718a8d824, 2026d. URL https://github.com/sgl-project/sglang/blob/bd66ce343e4f6e2f2b75d7e820fe4d0718a8d824/python/sglang/srt/speculative/dflash_worker_v2.py.
- So, J., Kook, H., Jang, C., and Park, E. Speculative coupled decoding for training-free lossless acceleration of autoregressive visual generation. In *International Conference on Machine Learning (ICML 2026)*, 2026. URL <https://arxiv.org/abs/2510.24211v2>. Version 1 of the arXiv preprint was titled “MC-SJD: Maximal Coupling Speculative Jacobi Decoding for Autoregressive Visual Generation Acceleration”.

- Song, Y., Meng, C., Liao, R., and Ermon, S. Accelerating feedforward computation via parallel nonlinear equation solving. In *Proceedings of the 38th International Conference on Machine Learning*, volume 139 of *Proceedings of Machine Learning Research*, pp. 9791–9800. PMLR, 2021. URL <https://proceedings.mlr.press/v139/song21a.html>.
- Teng, Y., Shi, H., Liu, X., Ning, X., Dai, G., Wang, Y., Li, Z., and Liu, X. Accelerating auto-regressive text-to-image generation with training-free speculative Jacobi decoding. In *International Conference on Learning Representations (ICLR 2025)*, 2025. URL <https://arxiv.org/abs/2410.01699v2>.
- Triton contributors. triton.autotune. Triton documentation, 2026. URL <https://triton-lang.org/main/python-api/generated/triton.autotune.html>. Accessed 30 September 2026.
- Urbán, O., Kwon, Y. D., Venieris, S. I., and Mascolo, C. Margins, not windows: Training-free per-step lossy speculative decoding. arXiv:2609.02897v2, 2026. URL <https://arxiv.org/abs/2609.02897v2>.
- Vo, Q., Nguyen, C.-D., Srey, P., Tuan, L. A., and Nguyen, T. TIGER: Text-conditioned visual gated routing with acceptance alignment for multimodal speculative decoding. arXiv:2607.11131v1, 2026. URL <https://arxiv.org/abs/2607.11131v1>.
- Wang, J., Paliotta, D., May, A., Rush, A. M., and Dao, T. The Mamba in the Llama: Distilling and accelerating hybrid models. In *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)*, pp. 62432–62457, 2024. URL https://proceedings.neurips.cc/paper_files/paper/2024/hash/723933067ad315269b620bc0d2c05cba-Abstract-Conference.html.
- Wang, J., Su, Y., Li, J., Xia, Q., Ye, Z., Duan, X., Wang, Z., and Zhang, M. OPT-Tree: Speculative decoding with adaptive draft tree structure. *Transactions of the Association for Computational Linguistics*, 13:188–199, 2025. doi: 10.1162/tacl_a_00735.
- Wang, L., Su, Y., Wu, X., You, C., Liu, Y., Qiu, Z., Zhang, J., Zheng, J., Liu, F., Zhang, J., Tian, C., and Huan, C. Bole: Efficient tree speculation for hybrid-attention language models. arXiv:2608.01651v1, 2026a. URL <https://arxiv.org/abs/2608.01651v1>.
- Wang, Z., Han, X., Yang, Z., Liu, F., Li, X., Gu, R., Zhong, S., and Tian, C. SpecLA: Efficient speculative decoding for linear-attention models. arXiv:2607.16673v1, 2026b. URL <https://arxiv.org/abs/2607.16673v1>.
- Williams, M., Kwon, Y. D., Li, R., Kouris, A., and Venieris, S. I. Speculative decoding with a speculative vocabulary. In *Findings of the Association for Computational Linguistics: ACL 2026*, pp. 40240–40254, 2026. doi: 10.18653/v1/2026.findings-acl.2000.
- Wu, T., Yao, Y., Qi, Z., Zheng, H., Wang, Z., Ma, H., Liao, L., Lakkaraju, H., Li, J., and Du, Y. D-PACE: Dynamic position-aware cross-entropy for parallel speculative drafting. arXiv:2605.18810v1, 2026. URL <https://arxiv.org/abs/2605.18810v1>.
- Wu, Y., Qin, Z., Wong, A., and Soatto, S. STree: Speculative tree decoding for hybrid state-space models. In *Advances in Neural Information Processing Systems 38 (NeurIPS 2025)*, 2025. URL <https://neurips.cc/virtual/2025/poster/117297>.
- Xia, T., Ganesan, M., Feng, Y., Wang, H., Egger, M., and Zhang, S. Q. Acceptance-aware draft model training for speculative decoding. arXiv:2609.24150v1, 2026. URL <https://arxiv.org/abs/2609.24150v1>.
- Yang, S., Wang, B., Zhang, Y., Shen, Y., and Kim, Y. Parallelizing linear transformers with the delta rule over sequence length. In *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)*, pp. 115491–115522, 2024. URL https://proceedings.neurips.cc/paper_files/paper/2024/hash/d13a3eae72366e61dfdc7eea82eeb685-Abstract-Conference.html.
- Yang, S., Kautz, J., and Hatamizadeh, A. Gated delta networks: Improving Mamba2 with delta rule. In *International Conference on Learning Representations (ICLR 2025)*, 2025. URL <https://openreview.net/forum?id=r8H7xhYPwz>.
- Yao, J., Li, H., Liu, Y., Ray, S., Cheng, Y., Zhang, Q., Du, K., Lu, S., and Jiang, J. CacheBlend: Fast large language model serving for RAG with cached knowledge fusion. In *Proceedings of the Twentieth European Conference on Computer Systems (EuroSys ’25)*, pp. 94–109, 2025. doi: 10.1145/3689031.3696098.
- Ye, Z., Chen, L., Lai, R., Lin, W., Zhang, Y., Wang, S., Chen, T., Kasikci, B., Grover, V., Krishnamurthy, A., and Ceze, L. FlashInfer: Efficient and customizable attention engine for LLM inference serving. In *Proceedings of the Eighth Conference on Machine Learning and Systems (MLSys 2025)*, 2025. URL https://proceedings.mlsys.org/paper_files/paper/2025/hash/dbf02b21d77409a2db30e56866a8ab3a-Abstract-Conference.html.

- z-lab. Qwen3.5-4B-DFlash model card. Hugging Face, revision 9a1996ccf887b79ab3af4fcbf8c1d1f4b5658bcf, 2026. URL <https://huggingface.co/z-lab/Qwen3.5-4B-DFlash/tree/9a1996ccf887b79ab3af4fcbf8c1d1f4b5658bcf>.
- Zhang, J., Ullah, N., Schultheis, E., and Babbar, R. DynaSpec: Context-aware dynamic speculative sampling for large-vocabulary language models. arXiv:2510.13847v3, 2025. URL <https://arxiv.org/abs/2510.13847v3>.
- Zhang, S., Qiu, H., He, H., and Dai, Y. Cost-aware diffusion draft trees for speculative decoding. arXiv:2606.01813v1, 2026a. URL <https://arxiv.org/abs/2606.01813v1>.
- Zhang, T., Tan, J., Sun, P., Yu, Y., Su, Z., Jiang, Z., Xie, Y., Cai, X., and Zeng, Z. DAMP: Decay-aware mixed-precision recurrent-state quantization. arXiv:2608.27513v2, 2026b. URL <https://arxiv.org/abs/2608.27513v2>.
- Zhang, Y., Zhang, L., Cui, B., and Miao, X. DFlow: Enabling verifier information flow in block diffusion speculative decoding. arXiv:2609.06498v1, 2026c. URL <https://arxiv.org/abs/2609.06498v1>.
- Zhao, W., Pan, T., Han, X., Zhang, Y., Sun, A., Huang, Y., Zhang, K., Zhao, W., Li, Y., Zhou, J., Zhou, H., Wang, J., Liu, Z., and Sun, M. FR-Spec: Accelerating large-vocabulary language models via frequency-ranked speculative sampling. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 3909–3921, 2025. doi: 10.18653/v1/2025.acl-long.198.
- Zheng, L., Chiang, W.-L., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., Lin, Z., Li, Z., Li, D., Xing, E. P., Zhang, H., Gonzalez, J. E., and Stoica, I. Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023), Datasets and Benchmarks Track*, pp. 46595–46623, 2023. URL https://proceedings.neurips.cc/paper_files/paper/2023/hash/91f18a1287b398d378ef22505bf41832-Abstract-Datasets_and_Benchmarks.html.
- Zhou, E., Sheng, K., Chen, H., and He, X. CARD: A cache-assisted parallel speculative decoding framework via query-and-correct paradigm for accelerating LLM inference. arXiv:2508.04462v2, 2025. URL <https://arxiv.org/abs/2508.04462v2>. Version 1 was titled “CARD: Cache-Assisted Parallel Speculative Decoding for Efficient Large Language Model Inference”.
- Zou, X., Li, J., Huang, J., and Zhou, P. Variational speculative decoding: Rethinking draft training from token likelihood to sequence acceptance. arXiv:2602.05774v6, 2026. URL <https://arxiv.org/abs/2602.05774v6>.